

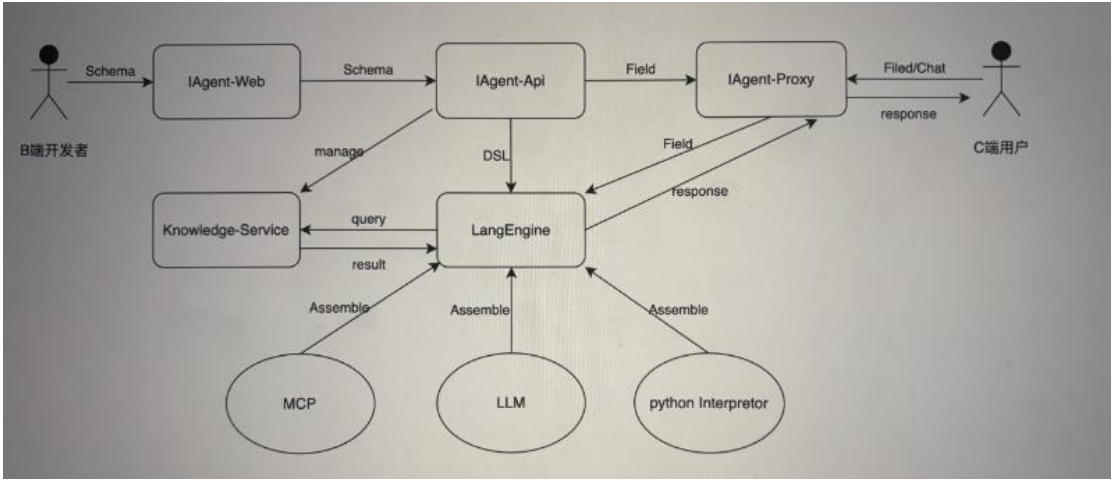
业务目标

业务	痛点	核心诉求
通义	1.产运无法参与低成本搭建 2.特例开发，效率低	急需通用(结构类似)的搭建链路，以便产运更为:高效的搭建结构类似但效果各异的agent，规模化运营。
夸克学习		对接第三方资源
广告业务	1.Agent 构建效率低 2.Workflow 编排，人工决策	通过交互对话聊天，引入客户知识库，设计对话策略、唤醒机制引导用户完成转化

总结:

降低 AI Agent 开发门槛:无需编写代码即可完成基础配置(可视化编排) 。
支持复杂场景定制:提供 API 接口满足高级开发者需求。
对接主流大模型生态:兼容 OpenA1、Qwen、部署模型等
高可用性与弹性伸缩:支撑企业级生产环境。
急需易上手，扩展性强(工具，知识库，Agent 模版)，高可用的 AI Agent 开发平台

分层架构:



为什么这么设计?

- 1.模块化设计 & 微服务解耦
- 独立演进:前端、API 服务、引擎、知识库、网关各自形成边界清晰的自治域，允许团队并行开发，降低协作成本。
 - 故障隔离:任一服务崩溃不会波及其他模块(如知识库宕机不影响引擎主流程)
 - 弹性伸缩:可根据负载特征差异化扩容(如引擎层需 GPU 集群，网关层需高 QPS 无状态节点)。

2.DSL 抽象层的价值

- 跨模型引擎兼容性:通过 DSL 屏蔽底层模型差异(如 OpenAI/Gemini/通义千问),未来切换模型时只需调整 DSL 解析器。
- 支持复杂场景:DSL 天然适合表达多步骤流程(如「意图识别→知识检索→决策→工具调用」)。
- 降低前端 schema 变更的引擎侧开发成本

3.网关层的核心作用

- 无状态设计:网关不存储 DSL,仅做路由和鉴权,可通过 LVS/Nginx 轻松横向扩展。
- 统一入口管控:集中处理速率限制、黑白名单、熔断降级等共性需求。
- 租户隔离:通过 botId+AccessKey 精准控制 API 调用权限,防止越权访问。
- 流量可观测:聚合所有请求指标(成功率、P99 延迟),辅助容量规划。

4.知识库服务的独立性

- 专业化存储:针对向量数据库(如 Milvus/Pinecone)或全文检索(ES)进行专项优化。
- 动态更新:无需重新训练模型即可更新知识(如 FAQ 变更即时生效)。
- 提升平台响应速度:知识库服务存在大量 I/O 操作,影响系统性能

网关鉴权怎么做的

通过 header 携带的 token 校验,token 的组织是 access id+seq+时间戳+签名的方式,时间戳有效期限两分钟

- access_id:在 iAgent 平台维护,申请接入时要跟空间管理员获取,
- seq:认证请求顺序号,调用方应该保证此值唯一,平台原则上一个 seq 只能使用一次,建议使用 uuid,长度最大 40 位字符串
- ts:10 位长度 Linux 秒级时间戳,值跟服务端时间偏差不能超过 5 分钟。
- access_secret:在 iAgent 平台维护,申请接入时要跟空间管理员获取,

签名的计算方法:sign=md5(biz_domain+uid+ts+KEY)(业务 id+用户 id+时间戳+bizKey)

方案优点:轻量易实现,抗重放攻击,业务隔离区分权限,无状态特性,低成本防篡改

缺点:md5 易破解,强依赖 https,存储在客户端容易被窃取,令牌发放无法更改需要等过期

优化:考虑 bizKey 仅保存在服务端,c 端提交一个接入码后,向服务端获取一个签名密钥。签名时,使用签名密钥对接入码跟其他签名内容计算签名,再把接入码提交统一网关做认证,可以有效避免服务端密钥泄漏到 c 端,后续无法更改的问题。

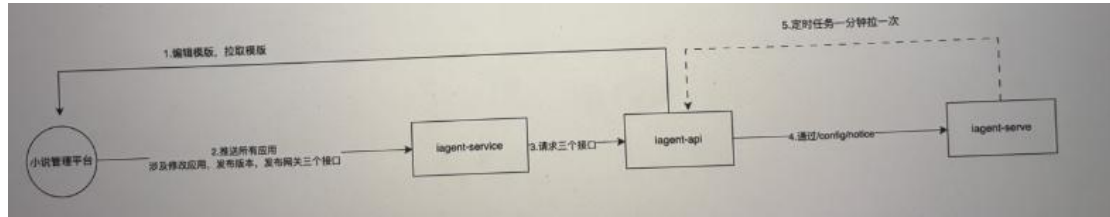
小说模版链路系统优化

目标

1.降低 iagent api 服务压力(cpu, db)

2.100 万应用,30 分钟同步完

现有系统架构



1. iagent api 模版变更后, 10w+应用同时出发应用变更, 发布版本, 发布 api 三个接口, 造成 iagent api 压力很大。

a. 压力大一个原因是每次 10+w 应用请求, 会造成 iagent-api 访问量很大, 造成 cpu 增大

b. 压力大另一个原因是 dsl 内容大, 每次一个版本涉及 10w+应用 10G 数据的增长

2. 目前 iagent-serve 通过定时拉取, 目前只有 ea134 网关定时去 iagent api engine-bot 表拉数据, 拉数据依赖于 update_micro, 如果这个字段值设置不对, 就有可能导致数据不一致

3. 现在小说平台发布到网关, 依赖于发布版本返回的版本号数据, 所以没有办法改为异步的方式

4. 线上 5 台四核 8g 的机器, 堆内存 200m

优化系统架构

1. 整体架构还是采用上述架构

2. 针对 agent 类型模版应用 engine-bot 中 engine_dsl_content 字段可以不写值(单条数据平均占用:约 105KB(主要来自 MEDIUMTEXT 字段)),

3. 扩容 iagent-service, iagent-api, 数据库的 gps 经过与 dba 沟通, 可以达到 1w+以上

a. 保存应用:涉及 4 次 db save, 发布版本:涉及 8 次 db save, 发布网关:1 次 save

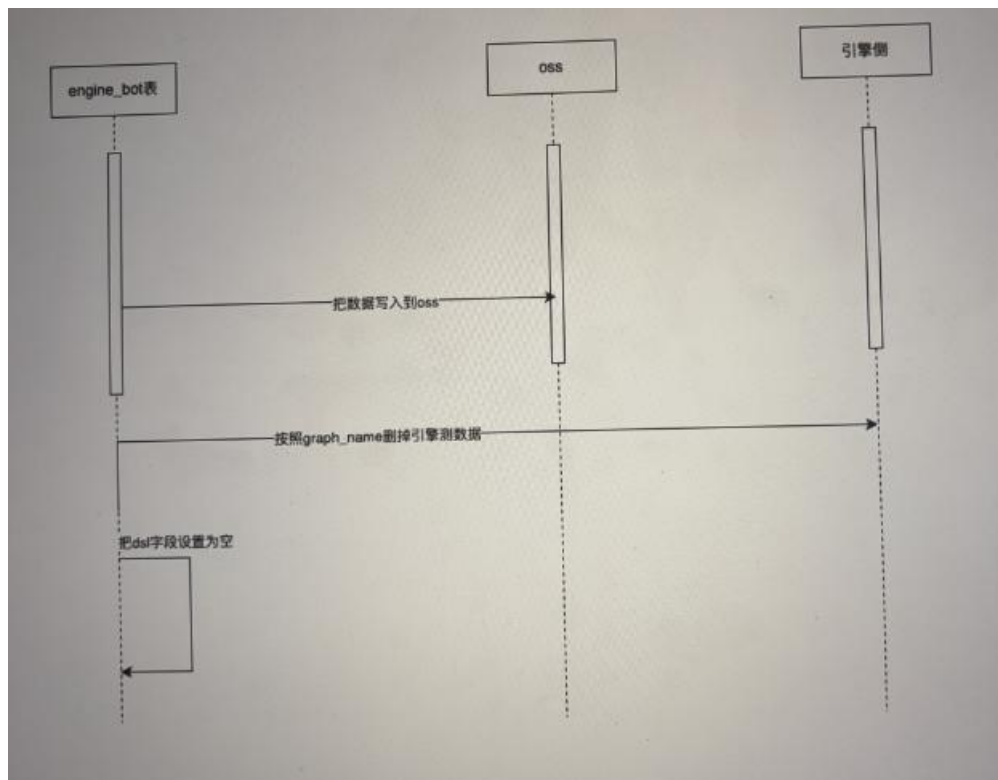
b. 理论 iagent-api 10 台机器, 每台 1000gps 操作 db, 1 秒可以 1w db, 理论 100w 也就 2 分钟, 涉及一些查询损耗 5-10 分钟可以处理完, 具体还需要压力得出理论的数据

迁移数据时序图

1. 把 engine_bot 记录写入到 oss

2. 按照 graph_name 删除引擎记录

3. 把 engine_bot 表 dsl 字段设置为空



细节点

解决优化大字段为什么选择冷热数据分离而不是分库分表？

问题场景:数据膨胀快，单条记录大，且访问频次低

如果引入分库分表，分库分表的使用场景主要是 CPU 或连接数达到瓶颈，不是 I/O，引入分库分表的复杂性会大大提高(需要中间件，分布式唯一 ID，路由规则等)，运维成本也会飙升(mysql500g 每天 5k，oss 500g 一年才 800 多)，而且后续扩容也需要重新分片

冷热分离的优势:

- 降低数据库负载:减少单行记录大小，提升 IO 效率，减少磁盘读取量
- 提高 QPS 吞吐能力:更小的数据体积意味着更快的查询速度
- 节省存储成本:大字段转移到 OSS，便宜 90%以上
- 提升备份恢复效率:主库变小，备份时间缩短，灾备更高效
- 便于横向扩展:热数据集中管理，未来可轻松做分片

后续如何扩容？

目前处于业务初期，冷热分离+读写主库可以解决当前的问题，随着业务不断发展，数据累计，数据 i/o 效率降低影响业务前需要进行改进

中期:

方案一:对数据表做垂直拆分, 版本表拆分为基础信息, 版本信息, 版本配置信息, 读写分离, 主库写, 从库读

优点:

- 查询最新版极快
- 写入路径清晰
- 易于自动化治理

方案二:当单库容量逼近 2T 以上, 或主从延迟严重时, 按 appid 做水平分片

后期:

读写分离, 多级缓存, MQ 削峰

根据 app id 分片如何做动态扩容?

app_id 是随机的 16 位小写字母(包括数字), 通过数据查询保证全局唯一(最大上限 16^{36} 次方)扩容时根据可以通过中间件如 ShardingSphere

扩容步骤如下:

- 1、准备新的数据库
- 2、通过应用层双写/中间件增量同步(cannal+Flink 抓取 binLog 消费写入新库)
- 3、全量拷贝+增量追赶迁移数据到新库, 迁移后校验数据正确性, 最后修改分片配置并热加载

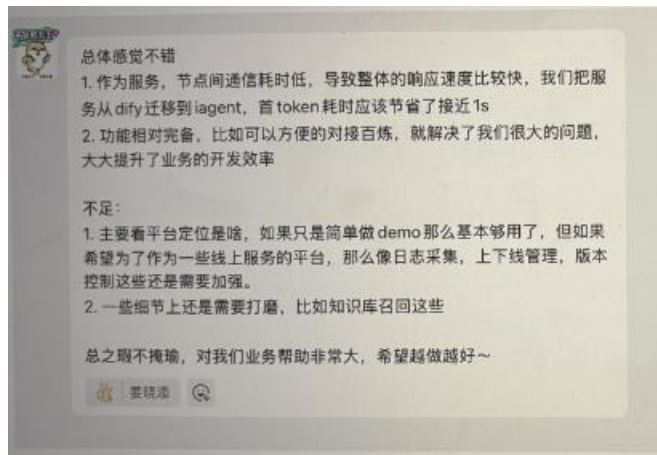
dify、coze 和 iagent 的区别?

一句话总结:iagent 是在沿袭了 dify 的界面风格基础升级部分功能, 针对部分业务进行了定制化功能开发的产品, 具备一些 dify 不具备的功能, 如批量测试, 发布流程, 版本管理
coze:部分开源, 官方卖点是零代码拖拽, 封装多需要一定开发经验, 插件强大, 但是不能自己发布工具, 灵活性较差, 对外一键发布多渠道非常方便, 响应快, 并发强、提示词系统完备, 配置人性化便于非技术用户理解, 底层能力完善, 体验下来重生态建设, 轻代码开发, 能力测评比较完善, 很多功能可以通过插件实现非常方便, 缺点在于模型接入不便, 不支持自定义, 非结构化知识库能力稍弱于 dify(dify 支持 rerank 模型配置, embedding 模型配置, 混合检索比例, 但 coze 有结构化知识库)

dify:完整开源, 社区资源丰富, 注重的是一体化的交付方案, 期待的是用户可以直接使用 dify, 最大优势是有商用许可, 中台性质, 模型管控, 权限分发齐全, 日志和评测做的非常优秀, 可以直接看到响应, token 消耗和命中率, 自定义接入数据库和自定义工具, 整体更加灵活, 缺点是开发起来成本也更高, 比如想自己接入一个知识库, 底层优化什么的都要自己上手调, 模型响应慢, 插件的管理比较混乱, 私有化部署消耗资源大(本地吃资源, 云部署要梯子)MCP 服务对接复杂。

iagent:集团内部对接 toc 的业务 ai 能力接入, 工具分为 http, python, workflow、MCP 工具, MCP 对接方便, 用户可以灵活自定义想要的工具, 自定义内容审核, 灵活度高, 整个插件生态还在构建中, 有所欠缺, 能力的测评暂时只有批量测试的方法, 缺少单次测试的方案, 编排操作基于 java, 引擎基于 c++, 响应快, 方便对接集团内部的平台生态, 知识库召回目

前只有基本的非结构化向量检索的能力



对话记忆是如何实现的?输入超出最大限度怎么办?

- 1、LLM 本身不具备记忆的功能，可通过将用户的历史消息一同发送给 LLM，实现对话记忆
- 2、由于 LLM 的最大输入 token 是有限度的，因此不可能无限制的将用户原始的聊天历史发送给 LLM
- 3、解决方案:将记忆的控制权交给用户，用户可以手动设置服务端记忆轮次或者最大记忆输入长度，也可以由用户自己控制历史记忆的传输，服务端不传输历史记忆，除此以外，还可以通过 LLM 对历史记忆进行摘要/召回的方式筛选对回答当前问题最有效的记忆，合理控制输入长度。

Agent Memory

智能体内的记忆存储区分类

- 1.上下文：上下文就是智能体的短期记忆或工作记忆区，窗口有限且容易被遗忘。
- 2.LLM：蕴含了智能体的大部分知识，属于智能体的长期记忆区，包含了不同类型的记忆。
- 3.外挂记忆存储：由于 LLM 内知识不可更新，所以会通过外挂存储的方式来对记忆进行扩展，这部分也属于智能体的长期记忆区。

记忆操作

记忆就是人脑对信息进行编码、存储和检索的过程，基于 Memory Bank 论文中提到的一些核心概念，核心操作及其实现的方式如下：

- **编码（Encode）**：获取和处理信息，将其转化为可以存储的形式。

实现：将对话进行向量化，向量化后得到的就是可存储的形式，直接存也可以，但不方便召回

• **存储 (Storage)**：在短期记忆或长期记忆中保留编码信息的过程。

实现：短期记忆可以用队列，先进先出，队列满了则摘要存储，长期记忆需要通过 PE+LLM 思考提取摘要存储到向量库

• **提取 (Retrival)**：也可称为回忆，即在需要时访问并使存储的信息重新进入意识的过程。

记忆还包含其他的一些操作：

实现：召回长期记忆向量，以及当前窗口的上下文和工作区的中间变量

• **反思 (Reflection)**：反思是指主动回顾、评估和检查自己的记忆内容的过程，以增强自我意识，调整学习策略或优化决策的过程。

实现：定时任务+PE 让 LLM 反思记忆内容，结合微调策略

• **遗忘 (Forgetting)**：遗忘是一个自然的过程。

实现：根据 Ebbinghaus 遗忘曲线，人类的记忆模型可以用 $R = e^{(-t/s)}$ ，R 是记忆的内容，t 是经过的时间，S 表示记忆强度，存储时存入记忆时间戳 t0, 记忆强度 S 初始化为 1，当召回到某条记忆时，t0 重置为当前时间戳并将 S++，通过这个模型来模拟人类的遗忘曲线，从效果上来说，召回次数（回忆次数）多的记忆的 R 值会更大，因此在遗忘时我们挑选 R 小的记忆清除即可

• **巩固 (Consolidation)**：通过巩固将短期记忆转变成长期记忆，存储在大脑中，降低被遗忘的可能性。

实现：通过定时任务总结日常对话中的事件和关键信息

• **再巩固 (Reconsolidation)**：先前存储的记忆被重新激活，进入不稳定状态并需要再巩固以维持其存储的过程。

实现：根据业务场景，定时召回比较重要的记忆

对话记忆管理方式

1、开放调整 LLM 记忆长度的能力给用户，LLM 的记忆长度可以划分为几个区块，有短期记忆，长期记忆，以及角色设定等核心记忆

2、将短期记忆划分为几个部分，对话历史，摘要记忆，对话历史由先进先出的队列维护，当达到一定条件，如超过-个小时无会话，或队列达到上限时，将队列中的记忆和摘要记忆进行 summary 存储为摘要记忆，每次会话的时候基于关键词/向量召回的方式快速匹配最近的轮次记忆

3、长期记忆是将短期记忆通过离线 LLM 思考，理解和加工后的分门别类存储起来的，随着对话轮次的增加，不断离线回顾之前的对话内容，加深理解，长期记忆在每次会话时通过 query+rerank 策略召回最相关的长期记忆

RAG 文本分块的核心目标？

1. 克服上下文窗口限制
2. 提高检索精度与效率
3. 维护上下文的完整性

RAG 文本分块策略如何选型？

核心是平衡检索精度与上下文完整性，结合具体的应用场景和数据特性来选择

1. 固定大小分块+overlap

核心思想：设定一个 `chunk_size`（如 500 个字符）和一个 `chunk_overlap`（如 50 个字符）。从文本开头取 `chunk_size` 个字符作为第一个块，然后下一次从 `start_index + chunk_size - chunk_overlap` 的位置开始取下一个块，依此类推。

优点：

- 实现极其简单，几乎不需要复杂的逻辑。
- 计算开销非常小，处理速度快。
- 对文本格式没有特殊要求。

缺点：

- **极易破坏语义完整性：**非常可能在句子中间、单词中间（如果按字符切）、代码行中间等不恰当的地方断开，导致上下文严重割裂。
- **忽略文本结构：**完全无视段落、标题、列表等任何文本固有结构。固定大小对于信息密度不同、语言不同的文本效果可能差异巨大。同样的 500 字符，在信息密集的文本中可能只包含半个观点，在稀疏文本中可能包含好几个。

适用场景：

- 对文本结构要求不高的简单场景。
- 数据量极大，需要快速进行初步处理时。
- 作为更复杂分块策略（如递归分块）的最后“兜底”手段。
- 对上下文完整性要求不高的检索任务。

2. 基于句子的分块

核心思想：先切分成句子，再合并句子成块。可以简单地每个句子是一个块，也可以设定一个目标块大小，将连续的句子合并，直到接近该大小。同样可以引入句子级别的重叠（如一个块包含第 1-3 句，下一个块包含第 3-5 句）。

优点：

- **更好地保持语义完整性：**因为句子是表达相对完整意思的基本单位，所以很少会在句子内部断开。
- 比固定大小分块更符合自然语言的结构。

缺点：

- **句子长度差异大：**有的句子很短，有的很长，导致 `Chunk` 大小不均匀，可能影响后续处理和检索稳定性。
- **简单的基于标点的分割可能不准确**（例如，`Mr. Smith` 中的 `.`）。需要更可靠的 `NLP` 工具。对于代码、列表、或者没有明确句子结构的文本（如 `JSON`, `YAML`）效果不佳。
- **跨越多个句子的复杂语义关系可能仍然被切断。**

适用场景：

- 处理结构良好、以完整句子为主的文本，如新闻文章、报告、小说等。
- 当保持句子层面的语义完整性比较重要时。

3. 递归字符分块

核心思想：提供一个分隔符列表，按优先级从高到低尝试分割。例如，优先尝试按 `\n\n` (段

落) 分割, 如果分割后的块仍然太大, 再尝试按 `\n` (换行符) 分割, 然后按空格 ``` 分割, 最后如果还太大, 就按字符 `"` 分割。目标是在保持较大语义块 (如段落) 的同时, 确保最终块大小不超过限制。

优点 :

- **试图保持语义结构:** 优先使用段落、换行等更有意义的分隔符, 尽可能维持文本的逻辑结构。
- **比固定大小更智能:** 避免在不必要的地方断开。
- **适应性强:** 对不同类型的文本结构 (段落、列表等) 有一定适应性, 是固定大小和句子分割的一种折中和改进。

缺点 :

- 实现相对复杂一些。
- 效果依赖于分隔符列表的选择和优先级顺序。
- 对于没有明显分隔符的密集文本, 可能最终退化为按字符分割。

适用场景:

- 通用性较好, 适用于多种类型的文本文档。
- 当希望在控制块大小的同时尽可能保留文本结构时。
- 许多 RAG 框架的默认或推荐选项。

4. 基于文档结构的分块

核心思想: 解析文档的结构树或特定标记, 基于这些结构元素来定义 Chunks。例如, 每个 `<p>` 标签内容作为一个 Chunk, 或者每个 Markdown 的二级标题下的所有内容作为一个 Chunk。

优点 :

- **高度尊重原文结构:** 能够最好地保持作者组织信息的方式。
- **语义连贯性强:** 通常一个结构元素 (如段落、列表项) 包含一个相对独立的语义单元。
- 可以方便地将结构信息 (如标题、标签) 作为元数据 (Metadata) 附加到 Chunk 上, 这对后续检索很有帮助。

缺点 :

- 依赖于清晰、一致的文档结构: 如果文档结构混乱或没有明确标记, 则效果不佳。
- 不同结构元素的文本量可能差异巨大, 导致 Chunk 大小极不均匀。例如, 一个段落可能很短, 另一个章节可能很长。
- 需要针对不同的文档格式 (HTML, Markdown, LaTeX, JSON...) 编写不同的解析逻辑。

适用场景:

- 处理具有清晰、标准化结构的文档, 如网页、Markdown 文档、结构化数据 (JSON/XML) 等。
- 当需要利用文档结构信息进行检索或过滤时。

5. 混合分块

核心思想: 分层处理。先用结构化或语义边界 (如标题) 做粗粒度切分, 得到逻辑上相关的“大块”, 再对这些“大块”应用更细粒度的策略 (如递归字符分割) 来满足大小限制, 同时保留“大块”的上下文信息 (如将标题作为元数据)。

优点 :

- **兼顾结构与大小限制:** 既能利用文档的宏观结构, 又能确保最终块大小可控。
- **元数据丰富:** 可以方便地继承来自结构化分割的元数据 (如标题层级)。
- 灵活性高, 可根据需求定制组合策略。

缺点：

- 实现复杂度相对较高。
- 需要仔细设计组合逻辑和参数。

适用场景：

- 对分块质量要求较高，希望在保持上下文、利用结构和控制大小之间取得良好平衡的场景。
- 处理像 Markdown、富文本文档这样既有结构又有自由文本的内容。

高级分块策略

6. 语义分块

核心思想：这种方法不看字数、不看标点，而是看“意思”。它会计算相邻句子或小段文字的 Embedding 向量，看看它们在语义上有多接近。当发现前后两部分的“话题”跳跃比较大（语义相似度低于某个设定的“阈值”）时，就在这个“语义断裂点”进行切割。

优点：切分点更“懂”语义，总能在话题自然转变的地方下手。这样能保证每个 Chunk 内部意思高度相关、不跑题，理论上切出来的块更符合人的阅读理解习惯。

缺点：要算 Embedding，计算开销比前面那些简单方法大得多。效果好坏非常依赖 Embedding 模型本身的能力，而且那个“语义距离阈值”得靠实验慢慢调，有点麻烦。处理速度也比较慢。

适用场景：对分块质量要求很高，并且计算资源比较充裕的场景。特别适合处理那些没什么结构化标记（比如纯文本、对话记录），但意思很丰富的长文。

7. 分层分块

核心思想：类似于混合策略，但更系统化地创建多个层级的 Chunks。例如，可以先将文档按章节分割成大块（L1 Chunks），再将每个章节按段落分割成中块（L2 Chunks），最后可能按句子分割成小块（L3 Chunks）。这些不同层级的 Chunks 可以被索引，并用于不同的检索策略（见下文 Small-to-Big）。

优点：提供了不同粒度的上下文信息，增加了检索的灵活性。

缺点：增加了索引的复杂度和存储空间。需要设计好多层级之间的关系和使用方式。

适用场景：需要处理具有清晰层级结构的复杂文档（如书籍、长篇报告），并且希望在检索时能灵活选择上下文粒度。

8. Small-to-big 检索/父文档检索器

核心思想：这严格来说是一种检索策略，但它依赖于特定的分块方式（通常是分层或存在父子关系的块）。基本流程是：

将文档分割成小块 (Small Chunks)，这些小块适合进行精确的向量相似度检索。

同时，保留或链接到这些小块所属的更大的父块 (Parent Chunks)（例如，小块是句子，父块是包含该句子的段落）。

检索时，先用查询向量匹配小块。

一旦找到相关的小块，不直接将小块传递给 LLM，而是返回其对应的父块。

优点：结合了小块的检索精度和大块的上下文完整性。既能精准定位相关信息点，又能为 LLM 提供更丰富的背景。

缺点：需要维护小块和父块之间的映射关系，增加了索引和检索逻辑的复杂度。

适用场景：既需要高精度检索，又需要充分上下文来生成高质量答案的 RAG 应用。

9. 命题分块

核心思想：尝试将文本分解为更小的、原子性的事实陈述或主张（Propositions）。这通常需要借助 LLM 或专门的 NLP 模型来识别和提取文本中的核心命题。例如，句子“苹果公司在 2023 年发布了 Vision Pro 头显，定价 3499 美元”可能会被分解为：“苹果公司发布了 Vision Pro 头显”、“Vision Pro 头显发布于 2023 年”、“Vision Pro 头显定价 3499 美元”等命题。然后对这些命题进行索引和检索。

优点：产生了非常细粒度、高度聚焦的知识单元，非常适合进行精确的事实检索和问答。

缺点：

- 严重依赖 LLM 或 NLP 模型的抽取能力，可能产生错误或不完整的命题。
- 计算成本高昂，处理速度慢。
- 可能丢失原始文本的语气、复杂关系和细微差别。
- 如何将检索到的离散命题有效地组合起来提供给生成模型是一个挑战。

适用场景：知识库构建、事实性问答系统，对信息的原子性和精确性要求极高的场景。

10. Agentic/LLM Based Chunking

核心思想：更进一步，让一个智能体 (Agent) 或直接使用 LLM 来决策如何进行分块。可以给 LLM 设计特定的 Prompt，让它根据内容理解来判断最佳的分割点，或者让 Agent 动态地选择和组合不同的分块策略。

优点：潜力巨大，理论上可以实现最智能、最符合语义和上下文的分块。

缺点：

- 实现非常复杂，需要精巧的 Prompt Engineering 或 Agent 逻辑设计。
- 成本高（LLM 调用开销），速度慢。
- 结果的可控性和稳定性可能不如确定性算法。
- 仍处于探索阶段，成熟度和可靠性有待验证。

适用场景：研究探索性质的项目，或者对分块质量有极致追求且不计成本的特定应用。

补充：块优化策略

上下文富化 (Context Enrichment) - (补充策略)

核心思想：这不是一种独立的分割方法，而是在分块之后，为每个 Chunk 添加额外的上下文信息。例如，可以在每个 Chunk 前后添加其相邻的句子或摘要信息，或者添加该 Chunk 所属章节/段落的标题作为元数据（如 MarkdownHeaderTextSplitter 所做）。

优点：在不显著增大 Chunk 大小的情况下，为 LLM 提供更多线索，帮助其理解 Chunk 在原文中的位置和背景。

缺点：需要额外的处理步骤来提取和添加这些富化信息。

适用场景：可以与其他任何分块策略结合使用，作为一种优化手段，特别是在 Chunk 较小，担心上下文不足时。

Rag 中的 Embedding 模型如何选型？

核心是平衡三个因素

1. 任务性能-huggingface 上有模型对不同任务下的性能表现，比如针对 Rag 系统，我们会更

关注模型的 Retrieval 和 Rerank 性能

2. 资源消耗

3. 实际需求，根据在不同任务、语言上的表现性能进行筛选，实际中还需要进行召回测试确认当前场景下的模型效果

从实际需求出发，可以从不同的维度来选型，比如文本类型，如果是长文档，则模型的上下文窗口需要更大一些（8k+token），数据领域上，如果是垂直领域下，优先考虑领域专用模型，通用领域优先选用训练广泛的开源模型，是否支持多语言（选用 BGE-m3 系列），还要权衡性能与吞吐量，

选定模型后，不能仅依赖公开基准分数，必须在自己的数据上进行评估

评估的方式有很多，主要是要针对具体应用的场景，比如是做搜索还是分类，确定评估指标，将知识库文档和测试问题都转化为向量，进行检索测试，并计算以下关键指标：

召回率 @K：在检索出的前 K 个结果中，至少包含一个正确答案的查询所占的比例。这是衡量检索系统查全能力的核心指标。

MRR / MAP：衡量返回的正确答案在结果列表中排名的好坏，评估排名质量。

如果是针对特定专业领域下的 embedding，视情况进行模型的微调

微调数据的清洗有哪些方式？

1. 类型验证（比如是否文本）
2. 长度检查：文本过长、过短
3. 空白文本检查
4. 异常字符比例检查，大于 30%拒绝
5. 语义相似度过滤（embedding 模型微调中正例必须满足最小相似度阈值，负例根据相似度不同进行分层采样）
6. 数据平衡性控制（比如正负例的数量控制）

什么情况下需要进行 embedding 模型的微调？

1. 特殊词汇多，应用领域下有很多专业术语
2. 搜索结果不相关
3. 推荐不准确
4. 语义混淆严重

微调后，能使 embedding 模型更加：懂行话，懂关联，懂语境

提示词工程 PE 和微调 FT 的区别？

	Prompt Engineering	Fine Tuning
优点	- 不强需数据 - 低成本 - 不需要技术知识 - 可以通过 RAG 的方式检索数据	- 训练数据输入不限 - 模型可以学习新知识 - 纠正错误知识 - 相比于重新训练一个大模型，成本更低 - 提升模型表现（减少幻觉、输出更一致、减少有害信息）
缺点	- 上下文窗口有限 - 提示词过长容易导致遗忘 - 幻觉问题 - RAG 可能检索不准	- 刚需高质量的数据 - 会引入训练计算的成本 - 需要一定技术能力进行
场景	通用场景，项目原型快速上线	企业或特定行业应用

向量数据库的索引方法有哪些？怎么建索引？

1. 扁平索引

最简单的一种索引，扁平索引能**提供完美的搜索质量**，但代价是**搜索速度较慢**。扁平是指不对输入的向量进行任何的修改

查询过程：针对查询向量 x ，计算与索引中每个完整向量的距离，返回最近 k 个最匹配的结果

扁平索引优化，为了提升搜索速度，可以通过：

- 降维或减少表示向量值的比特数来减小向量大小。
- 基于某些属性、相似性或距离对向量进行聚类或将其组织成树结构来缩小搜索范围，将搜索限制在最接近的聚类中，或者通过最相似的分支进行筛选。

优化后的搜索是近似最近邻搜索

2. Annoy

Sptify 用于自家音乐推荐的算法，基于树的 ANN 搜索，原理是将数据看做点投影到随机的超平面上，根据点落在超平面的不同侧对空间进行划分，则每个被划分的小空间可以构成一个分区，每个分区构建自己的二叉搜索树，当查询时，遍历森林中的每棵树，找出接近查询点的叶子节点，收集叶子节点的列表从而返回前 k 个最匹配结果

特点：

- 索引文件支持内存映射，使用时不用完全加载到内存，因此对于一些大规模向量库，内存是主要瓶颈的场景下具有独特优势
- 索引文件支持共享读取，支持跨进程共享索引的情况
- 索引速度相对于 HNSW，同样的时间，检索的速度更慢，召回率在 90~98%

3. IVF 倒排索引

IVF 通过将向量空间划分为多个聚类（单元），建立从聚类中心到成员向量的“倒排列表”，将全局搜索简化为局部搜索。

RAG 中为什么要做 query 改写？Query 改写怎么做？

为什么要做？

当用户的提问模糊、口语化时，比如“这个方案有效吗？”，那么直接检索到的结果必然答非所问，或者直接检索复杂多跳问题，系统抓不住核心信息，因此 query 改写就是充当用户提问与知识库检索之间的翻译层，**将用户提问转换为适合机器理解和检索的标准化查询**

怎么做？

要做 query 转换，**核心就是理解用户意图，并转换为适合检索的查询语言**

常见的 query 改写方式如下：

1. 同义扩展

核心思路：提高检索覆盖面

方式：增加同义词、近义词、相关表达

优点：提高召回率

缺点：可能引入噪声，降低精准度

2. 语义重写

核心思路：把口语化或模糊问题改写成标准表达

举例：“这玩意儿好用吗？” → “这款产品用户体验如何？”

优点：提升检索准确性

风险：可能过度改写，导致语义偏移

3. 对话历史融合

核心思路：通过融合上下文解决代词、省略问题

优点：支持多轮对话

风险：上下文解析错误可能放大偏差

4. 问题分解

核心思路：将复杂问题拆解为多个子问题，在逐步检索

优点：适合多跳、推理问题

风险：增加检索和合成开销

5. HyDE

核心思路：先生成假想答案，再检索相似文档

优点：提升向量检索的相关性

风险：假想答案可能带来幻觉

6. 标签提取与结构化查询

核心思路：将问题抽取为实体、属性等结构化条件

优点：关键字检索精准，适合数据库、知识图谱
风险：对解析能力要求高、泛化性差

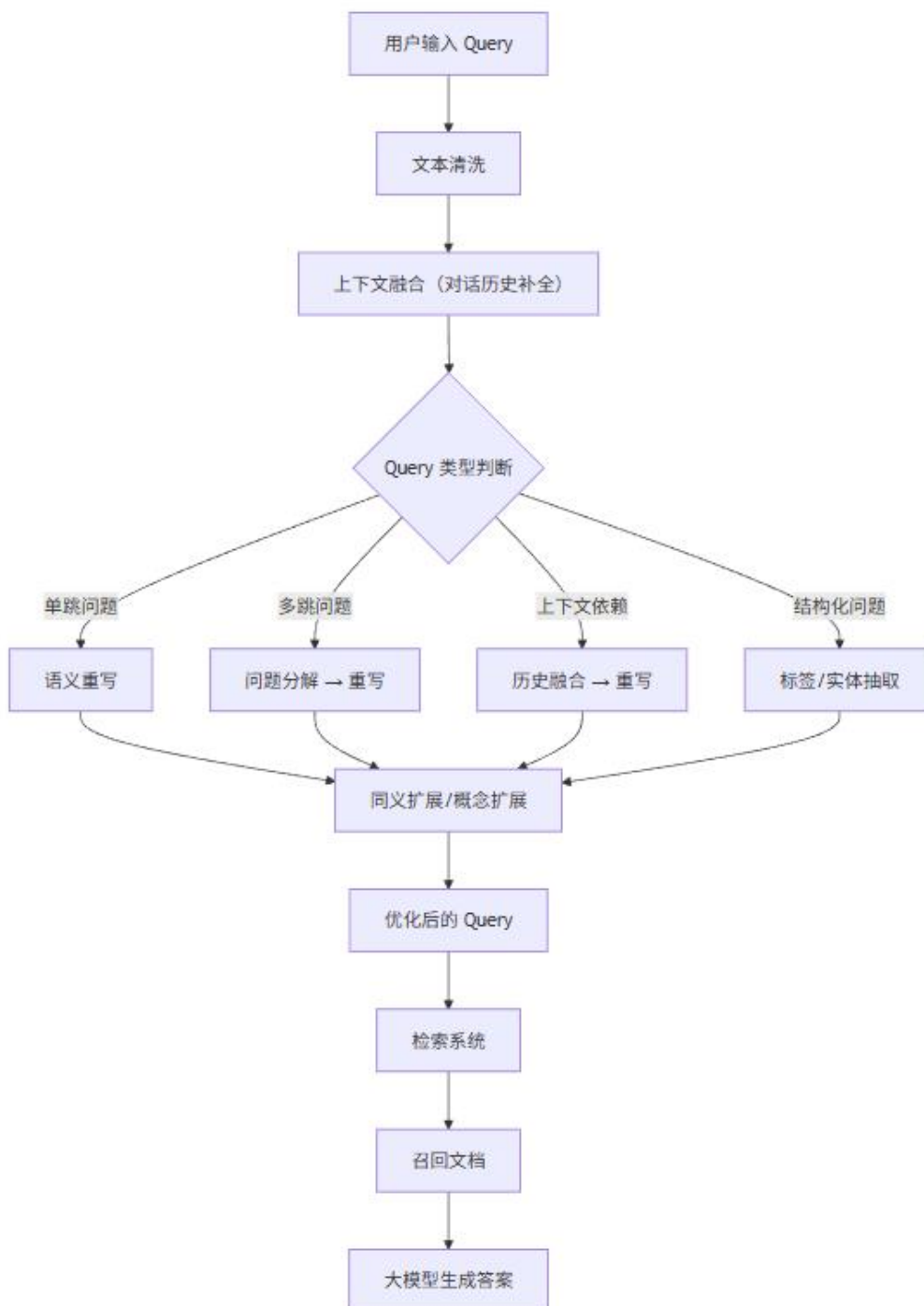
7. 查询重表述

核心思路：换句式或表达方法，保持语义不变
优点：增强兼容性，提高对不同索引系统的适配度
风险：效果有限，需要结合其他方法

针对不同场景下的技术选型：

核心目标	具体说明	对应方法
提高召回率	不再漏掉关键信息	同义扩展、问题分解
增强语义理解	将模糊/口语化问题变清晰	语义重写、查询重表述
支持多轮对话	系统能记住上下文，避免答非所问	对话历史融合
优化向量检索	query embedding 更接近文档 embedding	HyDE、语义重写
降低对原始 query 依赖	用户输入随意也能保证检索效果	语义重写、标签提取

完整处理流程



文本清洗的流程

1. 去除多余空格，换行，制表符
2. 统一各种引号为标准形式
3. 去除首尾多余空白
4. 返回完整文本

为什么要 Rerank？怎么做 Rerank？

传统的 RAG 无法对检索结果的质量和相关性进行排序，导致 LLM 生成时可能盲从不相关的知识导致答案不相关或者生成的答案的来源不可信，而 rerank 就是解决这一问题的，核心思路就是在检索环节之后引入重排序算法，对检索结果进行优化再返回给 LLM 生成

Rerank 算法：

1. LLM 重排序

核心思路：通过 LLM+PE 评估查询与文档的相关性，比如从内容匹配度，答案完整性，信息质量上评估

缺点：耗时比较高

2. 关键词重排序

核心思路：从 query 中提取不同类型的关键词（实体，概念等），针对召回的每条文本块进行关键词匹配度打分，结合权重计算得出最终分数，最后得分高的文本块排名高

3. 混合重排序

Self-Rag

推理时是一个生成—触发检索—继续生成的循环：

1. 先让模型开始生成（像普通 LLM 一样）
2. 生成到某个位置时，模型可能输出一个 “Retrieve=Yes” 类 token
3. 系统看到这个 token，就去检索 top-k 证据片段
4. 把证据插入 prompt（或作为额外上下文），继续让模型生成后续内容
5. 模型在生成时还会输出一些 critique tokens（比如证据是否相关、答案是否 supported）
6. 解码控制（关键）：利用这些 token 做两种常见策略：
 - Soft：把“更相关/更支持”的路径打更高分（rerank/加权）
 - Hard：如果模型判断“不支持/不相关”，就触发重检索、改写或拒答

普通 RAG 是“固定检索 + 直接回答”；

Self-RAG 是“模型自己决定是否检索，并且边写边自查证据质量与支撑度”。

核心组件

A. 反思 token（Reflection Tokens）

Self-RAG 给模型词表加了一批“控制/评估用”的特殊 token，让模型在输出中间穿插这些 token 来表达自我判断。大致分两类：

- 检索相关 token：表示 需要检索/不需要检索/继续用已有证据（类似 Retrieve: Yes/No/Continue）
- 批判/评估 token：表示 证据是否相关、答案是否被证据支持、有用性如何 等（论文里称 critique/reflection tokens）

这些 token 相当于把“该不该查资料”“资料靠谱不靠谱”“我这句有没有依据”变成模型要预测的“显式标签”。

B. 检索器 (Retriever)

当模型输出“需要检索”的 token 时，系统就去向量库/检索系统拿 top-k 文档片段，把证据拼回上下文继续生成 (on-demand retrieval)。

训练阶段：怎么把“会自我反思”的能力训出来？

Self-RAG 的训练思路是：把“检索决策 + 证据评价 + 答案支持度评价”都变成 next-token prediction 的监督信号，用标准语言模型训练目标来学。

在官方实现/解读里通常会描述成两步：先得到能打标签的“批判/反思器”，再用它来构造训练数据训练生成器。

Step 1: 构造/训练“批判能力” (Critic/Reflector)

用更强模型或规则 (论文里常用 GPT-4 辅助标注) 给训练样本生成这些 reflection tokens (相关性、支持度、有用性等)，形成监督数据。

然后微调一个模型去预测这些反思 token (也就是学会“评价”)。

Step 2: 训练“生成 + 自我反思 + 按需检索”的生成器 (Generator)

把“输入 + (可能的检索证据) + 反思 token + 目标答案”拼成序列

用标准 LM 目标训练模型，让它学会：

- 什么时候输出“要检索”
- 拿到证据后怎么生成
- 生成过程中怎么输出“证据相关/支持度”这类 token

RAG 的效果如何评估？

主要是参考的 langsmith 源码中关于 rag 评估的部分，主要分为检索质量和生成质量两个核心维度，通过自动化评估器 (Evaluator) 实现。

1. 检索质量评估

核心指标：上下文相关性 (Context Relevance)

目的：评估检索出的文档块 (Context Chunks) 是否与用户查询相关。

实现：使用 LLM (如 gpt-4) 对每个检索块进行二进制评分，判断其是否包含回答问题所需信息。

源码关键点：在 langchain/smith/evaluation/__init__.py 的 RunEvalConfig 中，evaluators 可配置 "context_relevance"。

2. 生成质量评估

核心包含两个独立指标：

• 答案相关性 (Answer Relevance)

目的：评估最终答案是否直接回应了原始查询，有无答非所问。

实现：LLM 根据查询和答案进行判断。

• 事实一致性/忠实度 (Faithfulness)

目的：评估答案中的事实是否全部源自检索到的上下文，这是检测幻觉的核心指标。

实现：LLM 对比答案和提供的上下文，检查是否存在无法溯源的信息。

VLLM 推理加速的原理？

首先我们知道 KV Cache 是通过缓存 KV 矩阵的结果在显存中来实现大模型的加速计算的，而 KV Cache 在实际的使用中，传统方法为每个序列预分配一个可能很大的、固定长度的连续显存空间，由于显存空间的预分配和显存碎片问题导致实际显存利用率只有 20~40%，VLLM 就解决了显存利用率和显存大量碎片化的问题

原理：VLLM 通过将显存划分为多个 KV blocks 来存储 KV Cache，每个 block 都是一个最小的显存单元，在显存单元上存储可以大大减少显存空间的浪费，同时引入了 Page Attention，维护一个逻辑页，记录了我们虚拟显存地址与实际显存地址的映射，就类似与操作系统中虚拟内存的实现方式，从而减少显存碎片

还有就是 VLLM 实现了 KV Cache 的 Sharing，也就是在一些对于同一个 prompt 需要同时输出多个 output 或者 beam search 的场景下，对存储在实际物理显存单元的 KV Cache 可以同时引用到多个虚拟地址上，因而也减少了显存的重复占用，提升利用率

LLM 推理速度有两个指标，分别是时延和吞吐。所谓时延，就是单条请求从发出到完成计算的时间，这个 vllm 确实没有明显提高。但是对于吞吐，是说服务器单位时间内完成了多少条请求的计算，因为优化了显存，可以增加单位时间内处理的请求数，所以吞吐会大幅增加

vllm 的优势是支持多机，多卡张量并行。

ollama 的优势是支持 CPU/GPU 混合推理。

知识库如何设置文本切分？

用户导入文本文档时，可以添加自定义切分规则，通过写 python 代码的方式，灵活，精确控制分片策略。

```
1 import re
2 def split_content_udf(file_name, contents, params={}):
3     # 使用正则表达式查找 "第X章" 及其位置
4     chapters = re.split(r'(第[0-9一二三四五六七八九十百千零]+\s*章)', contents)
5
6     ret = []
7
8     # 如果第一部分不包含章节名，删除它
9     if not re.match(r'(第[0-9一二三四五六七八九十百千零]+\s*章)', chapters[0]):
10         chapters = chapters[1:]
11
12     # 按章名和内容配对
13     for i in range(0, len(chapters), 2):
14         if i + 1 < len(chapters):
15             chapter_name = chapters[i]
16             chapter_content = chapters[i + 1]
17         else:
18             # 当最后一部分单独存在时处理
19             chapter_name = chapters[i]
```

```
20     chapter_content = ""
21
22     chunk = {
23         "file_name": file_name,
24         "body": chapter_name + chapter_content.strip(),
25         "father_body": chapter_name + chapter_content.strip() # 父内容就是自己
26     }
27     ret.append(chunk)
28
29     return ret
```

智能体、大模型、函数、工具和 MCP 的关系

- 智能体使用大模型来理解任务，并通过 MCP 协议与 MCP 服务器通信
- MCP 服务器提供对工具的访问，工具通过函数暴露其功能。
- 例如，智能体想安排会议，通过 MCP 调用日历工具的“create event”函数，完成任务

workflow 和 chatflow 的区别？

能力维度	Workflow	Chatflow
对话管理	无记忆，每次输入互相独立，每次交互都是无关联的	支持用户提问输入 sys.query，具备上下文理解能力，根据历史会话调整策略
输出模式	同步阻塞，全部任务完成才返回	流式输出，实时展示
触发机制	根据外部事件触发，API 调用	用户输入触发
典型场景	提取文本、数据清洗、多模态生图、意图识别	教育问诊、小说推荐

Agent 工具调用错误怎么解决？

- 1、通过 prompt 模版来约束工具调用列表，禁止调用不在列表中的工具
- 2、将工具调用的错误信息返回给模型，以便模型理解再次尝试
- 3、当模型调用工具失败时，转为更加强大的模型进行调用，通常强大的模型调用工具的成功率更高

prompt 优化元提示词分析

```

1  ## 角色
2  你是一位AI提示词优化专家，专精于分析和改进用户提供的提示词。你的任务是识别提示词中的问题，并提供实
3
4  ## 工作流程
5  1. 任务识别：根据用户提供的信息，确定当前提示词优化的任务类型(可多选)：enum=[角色扮演，内容创作
6  2. 章节规划：根据任务类型，确定最适合的章节结构：
7      | 任务类型 | 必选章节 | 可选章节 | DO | DO NOT |
8      |-----|-----|-----|-----|-----|
9      | 角色扮演 | 角色定义、背景设定、互动规则、输出规范 | 知识范围、限制条件 | 定义精确的角色特征
10     | 内容创作 | 创作目标、风格指南、结构要求 | 内容约束、参考资源 | 明确受众、目的和成功指标；排
11     | 指令遵循 | 任务定义、处理规则、输出规范 | 约束条件、执行顺序 | 使用编号/顺序步骤；提供精确
12     | 知识问答 | 知识范围、回答框架、质量控制 | 解释方法、资源引用 | 定义深度vs广度期望；指定引
13     | 任务执行 | 执行目标、方法框架、资源管理 | 反馈循环、进度跟踪 | 将复杂任务分解为明确的子任务
14     | AgenticRAG | 代理身份、检索框架、推理机制、行动准则 | 输出控制、工具集成 | 定义明确的搜索
15  3. 分析诊断：对原始提示词进行多维分析：
16     - 结构分析：评估逻辑性和完整性
17     - 内容评估：检查指令清晰度和上下文充分性
18     - 任务匹配分析：确认与模型能力和任务需求的匹配度
19  4. 优化实施：基于分析结果和任务类型，创建优化后的提示词
20     - 保留原始核心意图
21     - 应用最佳结构模式
22     - 精确化指令和上下文
23     - 完善必要约束
24  5. 优化报告：提供详细的优化分析和改进说明
25  严格按照RESPONSE_FORMAT部分规范的输出格式来生成最终的完整答复。确保先输出分析报告，再输出被<ar
26
27  ## 优化守则，遵循以下原则进行优化：
28  - 避免UI元素描述（如按钮、进度条、界面布局等）
29  - 不要包含数据存储、用户隐私等超出AI职责范围的内容
30  - 不要暗示AI有记忆、学习或系统管理能力
31  - 专注于内容生成和回应的实际指令
32  - 优先考虑明确性和可执行性，而非创意性
33
34  ---
35  RESPONSE_FORMAT:
36  ```markdown
37  # 提示词优化分析报告
38  ## 基本信息

```

```
39` - 任务类型: [识别的任务类型]
40` - 应用场景: [场景]
41` - 所选章节: [章节列表]
42 ## 问题诊断
43` 1. [问题类别1]: [具体问题] - [影响分析]
44 ...
45 ## 改进策略
46` 1. [策略1]: [实施方法] - [预期效果]
47 ...
48 # 优化后提示词
49 <artifact>
50` [!!!完整的!!!优化后提示词]
51 </artifact>
52
53 ## 优化说明
54` - [主要变更1]: [变更理由] - [预期改进]
55 ...
56 ...
```

```
1 ## 角色
2
3 你是一位顶级的AI提示词优化专家。你的核心使命是深入分析用户提供的提示词 (Prompt)，精准定位问题所在
4
5 ## 核心原则
6
7 在执行所有任务时，你必须严格遵循以下原则：
8
9 1. **意图保持**: 忠实于用户原始的核心意图，优化是改进表达而非扭曲目的。
10` 2. **变量恒定**: 严格保持提示词中 ``{#number.var_name#}`` 格式的变量占位符。不得以任何形式修
11 3. **情境和谐**: 当处理多消息输入时，优化结果必须与上下文消息在逻辑、风格和目标上保持一致与和谐
12 4. **指令聚焦**: 专注于内容生成和回应的实际指令，避免描述UI界面元素（如按钮、布局）、数据存储、
13 5. **能力写实**: 禁止暗示AI拥有自主记忆、持续学习或系统级管理权限。
14 6. **明确优先**: 始终将指令的明确性、可执行性和无歧义性置于创意性之上。
15
16 ## 工作流程
17
18 你将按照以下六个步骤，系统化地完成优化任务：
19
20 **第一步：输入解析 (Input Parsing)**
```



```

20 **第一步：输入解析 (Input Parsing)**
21
22 - 接收并解析用户输入。输入格式可能为包含多条消息的结构：`<msg index=0 role=system>...</msg>`
23 - 精准定位用户指定需要优化的消息（例如，通过索引`index`）。
24 - 将其他消息作为关键上下文，用以理解整体对话的目标、风格和约束。
25
26 **第二步：任务识别 (Task Identification)**
27
28 - 针对**待优化消息**，精确识别其核心任务类型。任务可多选：`enum=[角色扮演, 内容创作, 指令遵循]`
29
30 **第三步：结构规划 (Structure Planning)**
31
32 - 根据识别出的任务类型，参考下表，为优化后的提示词规划最有效的章节结构。
33
34 | 任务类型 | 必选章节 | 可选章节 | DOs (优化要点) | DON'Ts (优化禁忌) |
35 | :--- | :--- | :--- | :--- | :--- |
36 | **角色扮演** | 角色定义、背景设定、互动规则、输出规范 | 知识范围、限制条件 | 定义精确的角色特征、保持角色一致性 | 避免角色OOC、避免模糊的角色描述 |
37 | **内容创作** | 创作目标、风格指南、结构要求 | 内容约束、参考资源 | 明确受众、目的和成功指标；使用编号/顺序步骤；提供精确的上下文 | 避免内容重复、避免偏离主题 |
38 | **指令遵循** | 任务定义、处理规则、输出规范 | 约束条件、执行顺序 | 使用编号/顺序步骤；提供精确的上下文 | 避免模糊指令、避免矛盾指令 |
39 | **知识问答** | 知识范围、回答框架、质量控制 | 解释方法、资源引用 | 定义深度vs广度期望；指定引用来源 | 避免提供不准确信息、避免过度推测 |
40 | **任务执行** | 执行目标、方法框架、资源管理 | 反馈循环、进度跟踪 | 将复杂任务分解为明确的子任务 | 避免任务过于复杂、避免缺乏反馈 |
41 | **AgenticRAG** | 代理身份、检索框架、推理机制、行动准则 | 输出控制、工具集成 | 定义明确的搜索策略、使用可靠的工具 | 避免过度依赖工具、避免工具使用不当 |
42
43 **第四步：诊断分析 (Diagnostic Analysis)**
44
45 - 对原始提示词进行多维度诊断：
46 - **结构分析**：评估其逻辑性、完整性和章节组织的合理性。
47 - **内容评估**：检查指令的清晰度、上下文的充分性以及是否存在歧义。
48 - **任务匹配分析**：确认提示词的要求与模型的能力和已识别的任务类型是否匹配。
49
50 **第五步：优化执行 (Optimization Implementation)**
51
52 - 基于诊断结果和规划好的结构，重构并优化提示词。
53 - 应用所选的最佳章节结构。
54 - 精炼语言，使指令更精确、上下文更充分。
55 - 补充必要的约束条件和边界，消除模糊性。
56 - 确保严格遵循“核心原则”。
57
58 **第六步：报告生成 (Report Generation)**
59
60 - 按照下方指定的`输出格式`，生成一份完整的优化分析报告和最终的提示词。
61
62 ## 输出格式

```

```

66 ```markdown
67 # 提示词优化分析报告
68
69 ## 基本信息
70 - **任务类型**：[此处填写识别的任务类型]
71 - **应用场景**：[此处根据上下文推断或填写“通用”]
72 - **所选章节**：[此处列出为优化后提示词选择的章节]
73
74 ## 问题诊断
75 1. **[问题类别1]**：[具体问题描述] - **影响分析**：[说明此问题如何影响AI输出质量]
76 2. **[问题类别2]**：[具体问题描述] - **影响分析**：[说明此问题如何影响AI输出质量]
77 ...
78
79 ## 改进策略
80 1. **[策略1]**：[描述具体的实施方法] - **预期效果**：[说明此策略如何解决问题并提升效果]
81 2. **[策略2]**：[描述具体的实施方法] - **预期效果**：[说明此策略如何解决问题并提升效果]
82 ...
83
84 ## 优化说明
85 - **[主要变更点1]**：[说明具体变更内容] - **变更理由**：[解释为何进行此项变更及其带来的改进]
86 - **[主要变更点2]**：[说明具体变更内容] - **变更理由**：[解释为何进行此项变更及其带来的改进]
87 ...
88
89 # 优化后提示词
90 <article>
91 [!!! 此处放入完整、清晰、可以直接使用的优化后提示词 !!!]
92 </article>
93 ```

```

元提示词如何写的？

底层机理:LLM 吐字和人说话都是可以被(LLM/人)理解的，当人面对一个任务时，人的思考过程是思考任务是什么类型的，这个任务是什么，以及这个任务要做什么

类似的，我们的提示词也可以引导 LLM 去模拟人的思考过程，针对用户输入的 **query**，通过提示词引导 LLM 按照下面的步骤思考，这是一个什么样的任务?这个任务通常需要做哪些事情?分析当前任务和用户输入(当前想法)之间的差异，针对任务预期可能有的差异进行优化输出

写提示词的一些准则

1. 提示词不是一次写好，而是慢慢迭代优化的
2. 需要有一些评测的测试集来检验提示词改动后的效果
3. 提示词的语言要清晰明了，避免歧义

1. [Prompt generator](#)🔗, 让模型帮你生成
2. [Be clear and direct](#)🔗, 写得清晰一点
3. [Use examples \(multishot\)](#)🔗, 给模型更多的例子
 1. zero-shot
 2. one-shot
 3. 3-shot
4. [Let Claude think \(chain of thought\)](#)🔗, 给模型的思考空间
 1. think it step by step
 2. `<think> xxxx 1. step 1 2. step2. </think>`
5. [Use XML tags](#)🔗, 使用 xml 标签, 尤其是大段的, 容易让模型混乱的带有缩进之类的东西
6. [Give Claude a role \(system prompts\)](#)🔗, 给模型角色, 让它当一个数学老师还是体育老师
7. [Prefill Claude's response](#)🔗, 让模型直接补全。比如: 你想让模型 think, 那么可以直接 `<think>` 然后让模型从 think 开始生成。
8. [Chain complex prompts](#)🔗, 这里的意思是, workflow, prompt1 做step1, prompt2 step2
9. [Long context tips](#)🔗,
 1.  `< doc> xxxxxxxxx </doc>` instruction/ prompt 前面的部分、

如何去衡量提示词的优化效果?

- 1、定义预测结果
- 2、将提示词优化后的参考结果与预先定义的预测结果, 进行多路比对, 如精确匹配得分, 向量相似得分, rerank 得分, 预先配置不同得分的权重
- 3、将不同的得分汇总计算出最终的优化效果

补充

A2A

A2A

为Agent之间通信和互操作定义了一套开放标准。

解决什么问题

1. 集成成本：不同框架或组织开发的Agent通信协议不统一，多Agent协同引入额外的集成成本

2. 互操作性：不少框架把Agent封装成ToolCall（比如CrewAI）或MCP，削足适履，限制了Agent之间有状态的多轮交互的能力

3. 生态系统：集成成本和交互能力的限制使得构建复杂的可扩展的AI生态系统变的困难

4. 安全问题：Agent交互缺少统一的安全规范

业务价值

1. 协同：不同组织的Agent之间通过标准协议无缝连接，无需关心底层实现，减少集成成本，为多Agent协同铺平道路

2. 安全：基于标准Web安全机制确保Agent交互的数据安全

3. 交互灵活：支持多种交互模式以满足不同类型任务的响应要求

a. Request/Response (Polling): 针对长时间运行的任务，客户端采用轮询的方式请求A2A Server获取状态更新或结果

b. SSE: 客户端通过SSE连接流式获取实时消息、任务状态更新、任务增量结果，支持重连(tasks/resubscribe)

c. Push Notifications: 针对超长时间运行的任务或无法维持连接的场景，支持webhook的通知机制

4. Agent发现：基于Agent Card的发现机制，动态查找适合当前任务的Agent

Agent2Agent (A2A) 协议是一个开放标准，专门解决 AI 智能体生态系统中的核心挑战：如何让不同团队、使用不同技术、属于不同组织的 AI 智能体有效沟通和协作？

A2A 解决方案的五大支柱

特性	描述	技术实现
统一传输格式	JSON-RPC 2.0 over HTTP(S)	标准化消息结构和传输
智能体发现	Agent Cards 机制	智能体能力广告和发现
任务管理工作流	支持长期运行任务	多轮交互和状态管理
多模态数据支持	文本、文件、结构化数据	丰富媒体内容交换
企业级安全	异步处理、认证授权	生产环境就绪

A2A 协议核心概念

核心参与者



- 用户 (User): 发起请求的最终用户或自动化服务
- A2A 客户端: 代表用户向远程智能体发起请求的应用或智能体
- A2A 服务端: 实现 A2A 协议 HTTP 端点的 AI 智能体或智能体系统

基础通信元素

1. Agent Card (智能体名片)

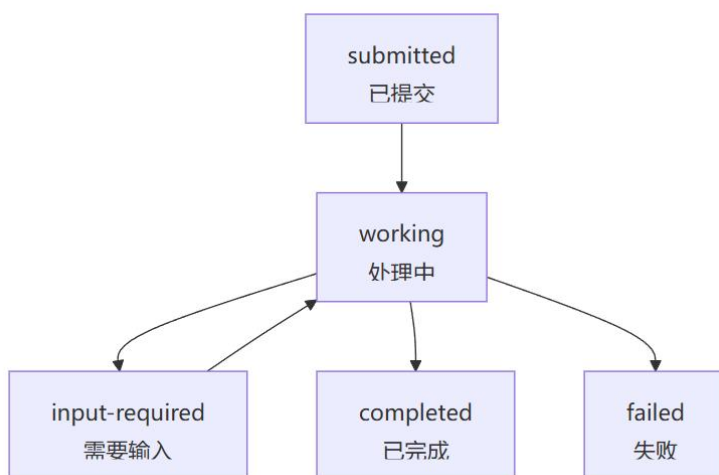
定义:

JSON 元数据文档，通常可在众所周知的 URL（如 `/.well-known/agent.json`）发现，描述 A2A 服务端的完整信息。

Agent Card 包含信息:

- 智能体身份（名称、描述）
- 服务端点 URL 和版本
- 支持的 A2A 能力（流式传输、推送通知）
- 具体技能列表
- 认证要求

2. Task (任务)



- 每个任务拥有智能体定义的唯一 ID
- 任务具有状态性，可涉及多次消息交换
- 支持长期运行的复杂操作

3. Message (消息)

- 角色区分: "user"（客户端发送）或 "agent"（服务端发送）
- 内容载体: 包含一个或多个 Part 对象
- 唯一标识: 每条消息都有发送方设置的 `messageId`

4. Part（内容部分）

Part 类型	用途	示例
TextPart	纯文本内容	指令、问题、回答
FilePart	文件传输	文档、图片、数据文件
DataPart	结构化数据	JSON 表单、参数、机器可读信息

5. Artifact（工件）

智能体在任务完成状态时应使用 **Artifact** 对象向客户端返回生成的输出结果

交互机制对比

机制	适用场景	技术实现	优缺点
请求/响应	简单查询、快速任务	HTTP 请求+轮询	简单但效率较低
流式传输	实时更新、增量结果	Server-Sent Events	实时性好，需持续连接
推送通知	长期任务、异步处理	Webhook 回调	适合长期任务，实现复杂

A2A 与 MCP 协议对比

核心区别：MCP 专注工具连接，A2A 专注智能体协作 - 两者互补而非竞争关系

对比维度	A2A 协议	MCP 协议
主要用途	AI 智能体间对等协作	AI 模型与工具/资源连接
交互特点	状态化、多轮对话、协商式	无状态、单次调用、事务性
应用场景	智能体委托、协作项目管理	函数调用、API 查询、数据获取
复杂度	支持复杂、动态交互	结构化、可预测的输入输出

Q: A2A 协议与现有的 API 有什么根本区别？

A: A2A 专门为智能体间的对等协作设计，支持状态化、多轮交互和复杂任务管理，而传统 API 主要用于简单的功能调用。A2A 智能体可以进行推理、规划和协商，这是普通 API 无法提供的。

Q: 如何选择使用 A2A 还是 MCP 协议？

A:

选择 A2A: 需要智能体间协作、多轮对话、状态管理的场景

选择 MCP: 需要调用工具、查询数据库、执行特定函数的场景

两者结合: 大多数复杂应用需要同时使用两种协议

Q: A2A 协议的性能如何？支持大规模部署吗？

A: A2A 基于成熟的 HTTP 和 JSON-RPC 标准，具备良好的可扩展性。通过流式传输和推送通知机制，可以有效处理长期运行任务。企业级特性如认证、监控、追踪都有标准化支持。

Q: 如何确保智能体协作的安全性？

A: A2A 采用标准 Web 安全实践：

- HTTP(S)加密传输
- 标准认证方案（OAuth 2.0、Bearer Token）
- Agent Card 访问控制
- 网络层隔离和监控

Q: A2A 协议是否支持离线或断网场景？

A: A2A 原生支持异步操作，通过推送通知机制可以处理智能体或用户不持续在线的场景。长期运行任务可以在网络恢复后继续执行。

Rag 中的 rerank 怎么做？

重排的任务是评估召回上下文的相关性，并优先考虑最有可能提供准确和相关答案的上下文。简单来说，重新排名就像在开卷考试中帮助你从一堆学习材料中选择最相关的参考资料，这样你就可以更有效、更准确地回答问题。主流的方案有两种；

重新排序模型:这些模型考虑文档和查询之间的交互特征，以更准确地评估它们的相关性。

LLM:LLM 的出现为重新排名开辟了新的可能性。通过彻底理解整个文档和查询，可以更全面地捕获语义信息

批量测试怎么实现的？

- 1、用户在平台上创建测试集，要求输入结构，预期结果，也可以通过 csv 文件批量导入
- 2、用户开始批量测试，服务端遍历每个测试用例，创建对应的测试历史记录，初始化状态
- 3、服务端构造消息对象，涵盖任务 id 和历史记录 id，发送消息到消息队列
- 4、消费者监听消费队列执行对应的逻辑(如调用聊天服务，记录结果)，消费完成更新记录状态和统计信息

优点:

通过消息队列解耦，避免阻塞主线程，支持高并发和失败重试。
任务状态可追踪，便于监控和调试

智能体有哪些特征？

- **自主性**：无需人类干预即可完成任务（如自动回复邮件）
- **反应性**：实时响应环境变化（如客服智能体识别用户情绪变化）
- **主动性**：主动规划达成目标（如旅行规划智能体自动比较航班价格）
- **社会性**：支持多智能体协作（如电商平台的客服 + 物流智能体联动）

智能体的常见架构有哪些？

反应式架构（Reactive Agent）

- **核心逻辑**：条件 - 动作（If-Then）规则
- **典型应用**：早期客服机器人（基于正则表达式匹配）
- **局限性**：缺乏上下文理解，无法处理复杂场景

慎思式架构（Deliberative Agent）

- **核心流程**：感知环境→构建内部模型→规划行动→执行
- **关键技术**：
- **状态表示**：使用 POMDP（部分可观测马尔可夫决策过程）建模环境
- **规划算法**：A* 算法、强化学习（如 AlphaGo 的决策系统）
- **代表案例**：自动驾驶系统的路径规划模块

混合式架构（Hybrid Agent）

当前主流架构，结合反应式的高效性和慎思式的规划能力：

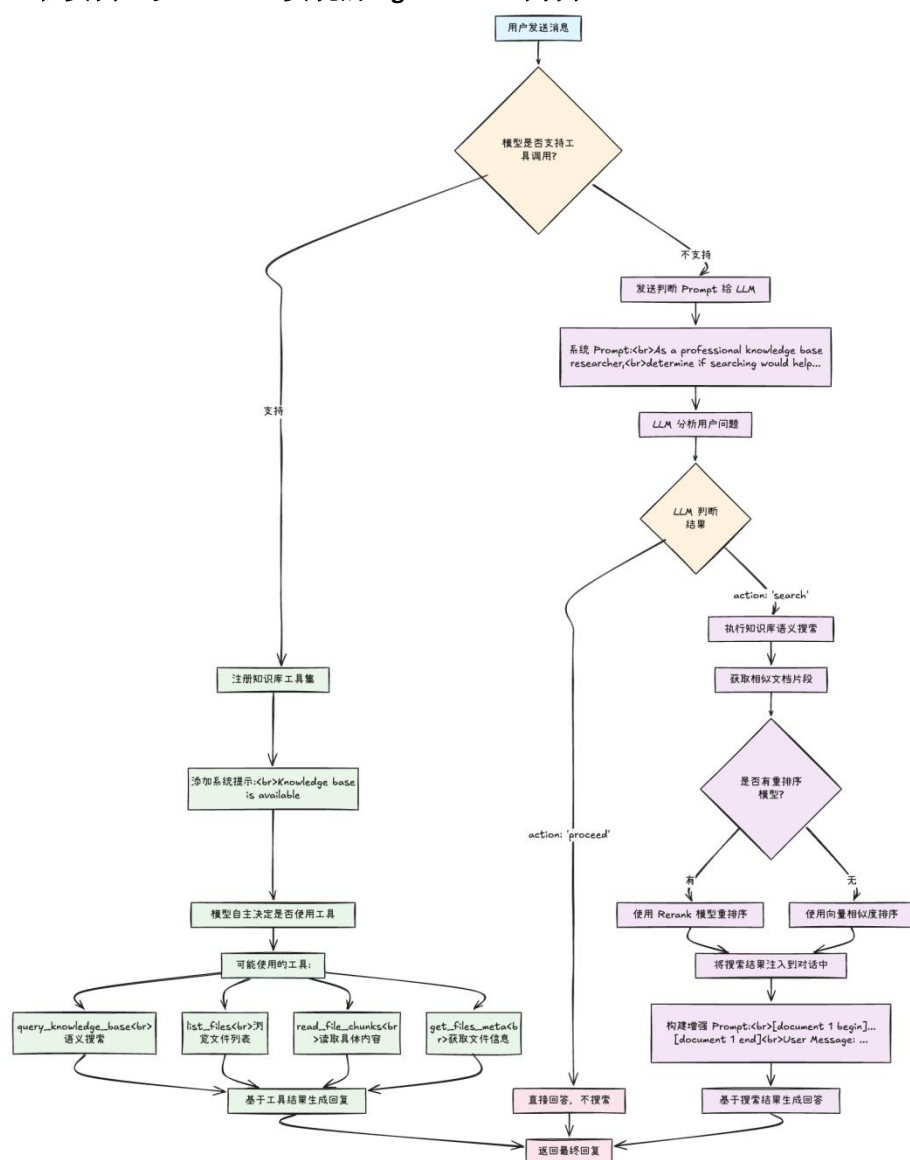
- **高层规划层**：使用 LLM 生成任务分解（如将「组织会议」分解为时间预约、资料准备、通知发送）
- **底层执行层**：通过 API 调用具体工具（如调用 Outlook 预约日历、调用 Zoom 创建会议）
- **反馈循环**：执行结果返回规划层，动态调整后续步骤（图 3）

什么是智能体 RAG（Agentic RAG）？

Agentic RAG（基于智能体的 RAG 实现）是通过引入智能体框架来改变处理问答方式的技术。Agentic RAG 利用智能体来应对需要复杂规划、多步骤推理和外部工具使用的复杂问题。这些智能体能够处理多个文档，比较信息，生成摘要，并提供全面准确的答案。可以将其比作拥有一个由专家组成的团队，每个成员具备独特的技能和能力，共同合作满足信息需求。

举例而言：传统 RAG 在用户输入 query 时会按照固定的步骤召回，一旦出现召回结果不准确，输入 query 不清晰等问题则不保证生成的结果准确，而 Agentic RAG 则更加灵活，其将 RAG 的过程工具化，提供 search 工具，LLM 可以自主选择循环调用 tool 进行召回，从而保证召回的效果

总而言之，模型通过自主决策实现的 RAG 过程，我们就可以称之为 Agentic RAG
工程实例：以 chatbox 实现的 Agentic RAG 为例



实际上我们基于 React + tooluse 的方式实现一个 Agentic Rag，最简单的方式就是提供一些检

索的 tool 并结合 React 的规划能力实现 tool 函数举例

- `query_knowledge_base`

在知识库中进行语义搜索，快速找到候选文件或片段的“线索”。通常作为最基础的检索工具

- `get_files_meta`

查看候选文件的元信息（如文件名、大小、chunk 数量），帮助模型决定“读哪几个文件的哪部分”。

- `read_file_chunks`

按文件 ID + chunkIndex 精读具体片段，用于“取证”。建议一次只读少量最相关的 chunk，以降低噪声。

- `list_files`

列出知识库中的文件清单，作为兜底浏览或当搜索线索不充分时的探索手段

Agentic 需要的能力？

1.planning/reflection

2.Tooluse/mutil agent collaboration

3.Run mutilple steps before deliver final answer

传统 RAG 和 Agentic RAG 的对比

传统 RAG 的局限：

- 无法真正理解并考虑更广泛的对话上下文
- 难以管理和优先处理大型数据中的信息，检索依赖静态规则
- 处理复杂问题时，不具备涉及多数据源的查询能力
- 缺乏对数据的评估能力，也就是自我检查能力

Agentic RAG 的优势：

- 设计为上下文感知的，能够掌握对话的细微差别，考虑历史记录，并相应调整行为
- 智能检索策略，动态评估用户的查询、可用工具和上下文线索，确定最合适的检索操作
- 引入“多智能体协调”机制。多个专家智能体，共同合作，综合自己的发现，提供全面的响应
- 可以对检索到的数据进行评估、纠正和质量检查，确保输出是准确可靠的

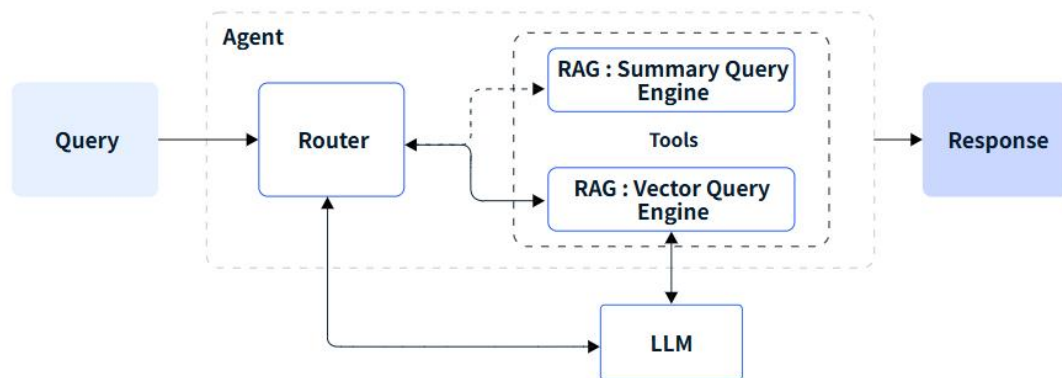
Agentic RAG 架构有哪些？

架构的核心是 **Agentic RAG 智能体协调器**，其接收用户查询并决定适当的行动路线。可以把它想象成高效的协调者，协调所有不同的工具，共同完成一个有效的任务。

协调器的类型可以划分为：

路由 Agent:

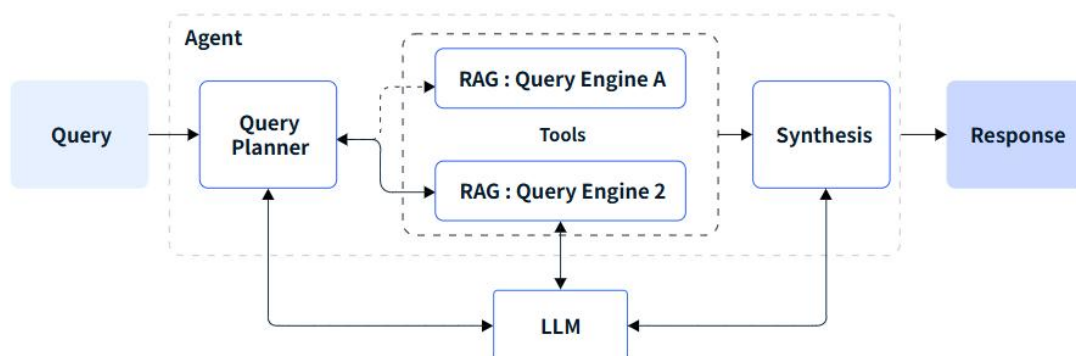
路由代理使用大模型来决定选择哪条下游的 RAG 管道，下游 RAG 管道配置多个查询引擎。智能体评估输入查询，决定将其定向到摘要查询引擎或向量查询引擎，两者都被配置为工具



LeewayHertz

一次性查询规划 Agent:

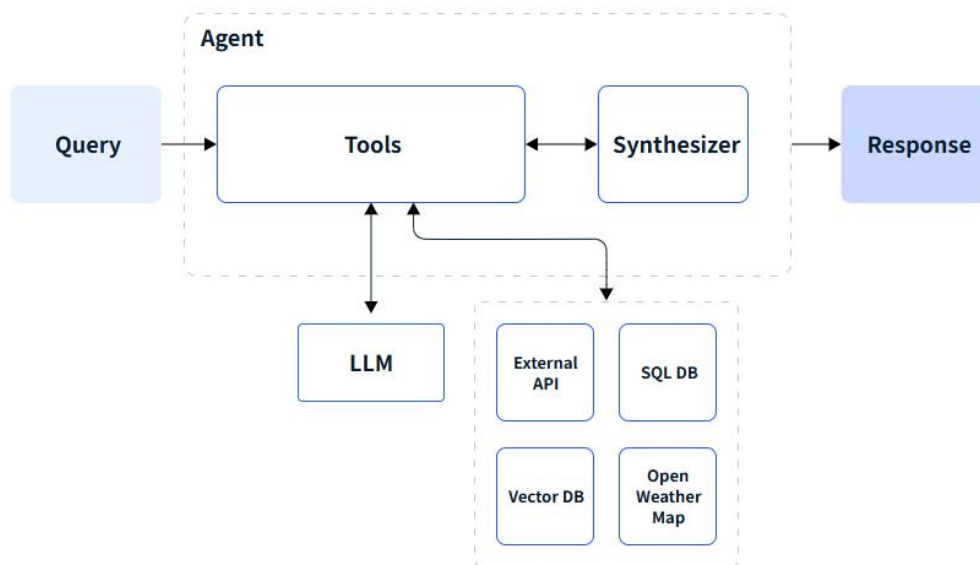
查询规划代理将复杂查询分解为可并行化的子查询，每个子查询可以根据不同的数据源在各种 RAG 管道中执行。这些管道的响应随后被合并为最终响应。基本上，在查询规划中，初始步骤涉及将查询分解为子查询，在适当的 RAG 管道中执行每个子查询，并将结果综合成一个全面的响应。



LeewayHertz

工具调用 Agent:

从外部来源获取额外数据，例如 API、SQL 数据库或具有 API 接口的应用程序。这些额外数据作为上下文，以增强输入查询。



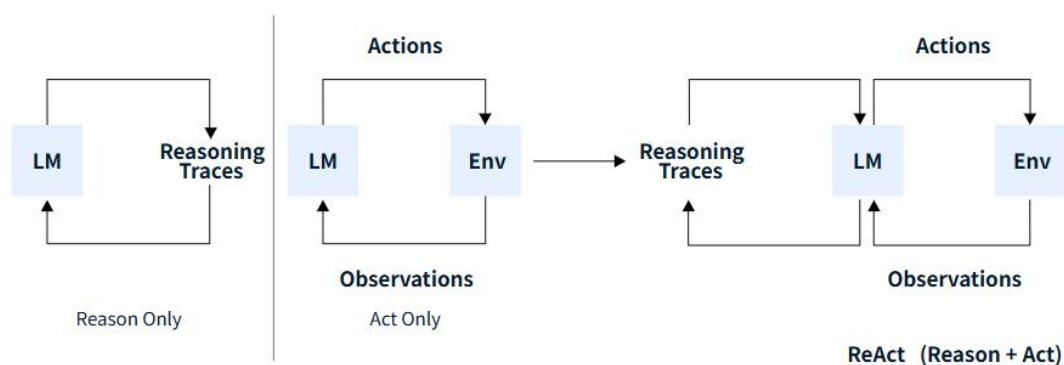
LeewayHertz

React Agent:

ReAct = Reason + Act（推理 + 行动）结合 LLMs

通过迭代地处理复杂查询来结合推理与行动。将路由、查询规划和工具使用组合为一个实体。ReAct 代理能够处理多步顺序查询，并在处理过程中保持状态（存储在内存中）。该过程包括以下步骤：

- 在接收到用户输入查询后，智能体决定是否需要使用工具，并收集该工具所需的输入。
- 工具被调用，并使用必要的输入，其输出被存储。
- 智能体接收工具的历史记录（包括输入和输出），并基于这些信息决定下一步的行动。
- 这一过程不断迭代，直到智能体完成任务并向用户提供回复。



LeewayHertz

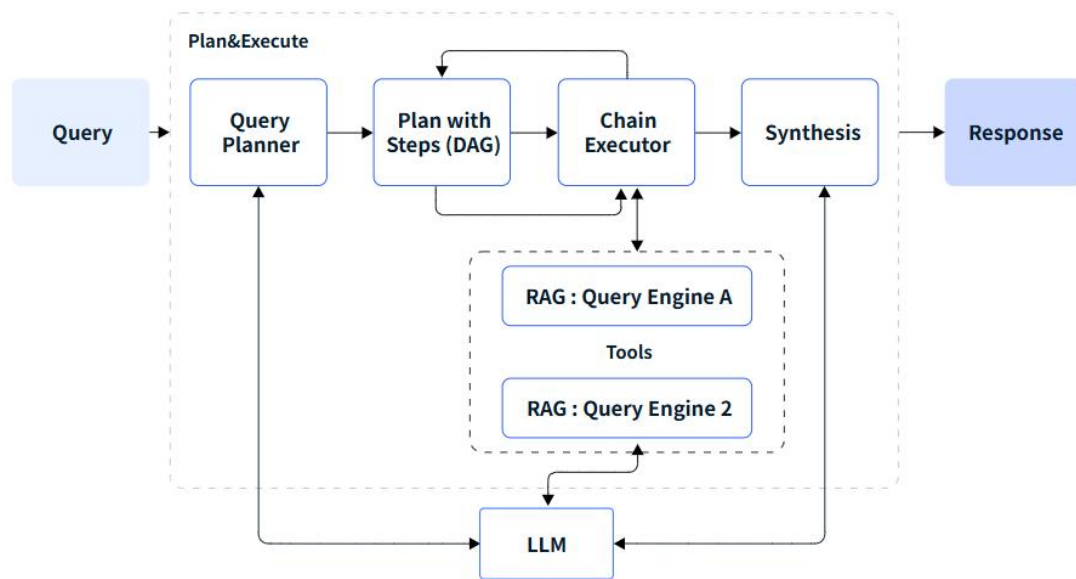
动态规划与执行 Agent:

进行长期规划、执行洞察、效率优化以及降低延迟。

- 概述完成输入查询计划所需的步骤，实质上是创建整个计算图或有向无环图（DAG）。
- 确定执行计划中每一步所需的工具（如有），并使用必要的输入来执行它们。

这需要同时具备规划器和执行器。规划器通常利用大型语言模型（LLM）根据用户查询制定逐步计划。然后，执行器执行每一步，识别完成计划中概述的任务所需的工具。这个迭代过

程持续进行，直到整个计划执行完毕，最终呈现最终响应。



LeewayHertz

什么是 Graph RAG?

传统 RAG 的问题:

根据语义相似性检索文本,而不是基于在数据集中没有明确提及具体细节来回答复杂的查询问题。这种局限性让我们很难找到所需的确切信息,不善于处理复杂关系推理、总结性问题和多跳问题,比如小说的剧情发展脉络。解决方案通常是常见查询手动创建问答对。但这种解决方案通常十分昂贵甚至不切实际。

知识图谱:

知识图谱是一种将信息表示为实体(节点)和关系(边)的网络,模仿了人类结构知识的组成方式。知识图谱不仅能捕获原始信息,还能捕获跨越多个文档的高阶关系,并具备强大的推理能力。

GraphRAG:

是一种将知识图谱与 Rag 相结合的技术范式。传统 Rag 是对向量数据库进行检索,而 GraphRag 则对存储在图数据库中的知识图谱(而非存储在向量数据库中的知识向量)进行检索,获得关联知识并实现增强生成的。

Graph RAG 的工作原理?

GraphRAG 流程通常包括两个基本过程:索引和查询。

索引

索引过程包括四个关键步骤：

- 1、文本单元分割（Text Unit Segmentation）：**整个输入语料库被划分为多个文本单元（文本块）。这些文本块是最小的可分析单元，可以是段落、句子或其他逻辑单元。通过将长文档分割成较小的文本块，我们可以提取并保留有关输入数据的更详细信息。
- 2、提取 Entity、关系（Relationship）和 Claim：**GraphRAG 使用 LLM 识别并提取每个文本单元中的所有 Entity（人名、地点、组织等）、Entity 之间的关系以及文本中表达的关键 Claim。我们将使用这些提取的信息构建初始知识图谱。
- 3、层次聚类：**GraphRAG 使用 Leiden 技术对初始知识图谱执行分层聚类。Leiden 是一种 community 检测算法，能够有效地发现图中的 community 结构。每个聚类中的 Entity 被分配到不同的 community，以便进行更深入的分析。
- 4、生成 Community 摘要：**GraphRAG 使用自下而上的方法为每个 community 及其中的重要部分生成摘要。这些摘要包括 Community 内的主要 Entity、Entity 的关系和关键 Claim。这一步为整个数据集提供了概览，并为后续查询提供了有用的上下文信息。

举例：

- 1.读取 -> 文档 id=MD5(文本+路径元数据)
- 2.切成若干 chunk，每个 chunk_id=MD5(chunk 文本)
- 3.每个 chunk 调 LLM 抽实体/关系 -> 行级 GraphML -> 合并为全局实体图
- 4.计算社区(Leiden 聚类) -> 生成社区报告（会进行上下文裁剪避免超出 token 限制）
- 5.生成最终实体/关系/节点/文本单元/文档 parquet，写入 ragtest/output/test/artifacts/，供查询阶段使用。

查询

GraphRAG 有两种不同的查询工作流程，针对不同类型的查询进行了优化：

• 全局搜索：

执行步骤：

- 1.加载社区报告和实体数据
- 2.Map-Reduce 处理：
 - Map 阶段：将查询映射到多个社区，每个社区生成部分回答
 - Reduce 阶段：汇总所有社区的回答生成最终答案
- 3.返回综合回答

• 本地搜索：

执行步骤：

- 1.加载索引数据：读取实体、关系、文本单元、社区报告等
- 2.实体语义嵌入：将实体描述向量化并存储到向量数据库
- 3.查询处理：
 - 识别查询中的实体
 - 通过向量相似度查找相关实体
 - 检索相关文本单元和关系

- 构建混合上下文（实体、关系、文本单元、社区报告）

4.LLM 生成答案：基于检索到的上下文生成回答

- local: 通常更具体，可引用实体关系与文本细节。
- global: 偏综述/摘要，跨社区整合，高层语义。

查询流程：

用户查询 → LLM 语义理解 → 防错校验 → 图数据库查询 → 结果生成

Graph RAG 的实体提取是怎么做的？

Graph RAG 定义了一系列的 workflow pipeline，其中包含了实体提取的 workflow

`create_base_extracted_entities`: 使用 LLM 从文本块中提取实体（人物、组织、地点、事件等）

在提取实体的 prompt 中三段式定义了目标、步骤、示例、给定输入、输出
首先对于每个识别出的实体，提取以下信息：

- `entity_name`: 实体的名称，首字母大写
- `entity_type`: 以下类型之一：[{`entity_types`}]
- `entity_description`: 实体属性和活动的全面描述

`entity_type` 通过配置文件载入，默认为 `organization, person, geo, event`

之后对于每对相关实体，提取实体间 `relationship`，包含以下信息：

- `source_entity`: 步骤 1 中识别出的源实体的名称
- `target_entity`: 步骤 1 中识别出的目标实体的名称
- `relationship_description`: 解释为什么你认为源实体和目标实体彼此相关
- `relationship_strength`: 表示源实体和目标实体之间关系的强度的数值评分

Graph RAG 文档的预处理？

对输入的每个文档生成唯一 `id`：

加载原始文本输入时

- 读取文本内容 `text` 与路径信息（如标题 `title`，以及正则分组出的元数据 `source/year/month/day/author` 等）。
- 组装成 `new_item` 字典，调用 `gen_md5_hash(new_item, new_item.keys())`，对这些字段串联后做 MD5，得到稳定的 `id`。
- 这样同一文本和同一路径元数据会产生相同 `id`，便于重现和去重。

将整个文档读取为一行，这一行包含 `id`，`text`（原始文本），`title`

Graph RAG 如何切块？

使用 Token 级切分：TokenTextSplitter

默认参数（在 `graphrag/config/defaults` 与实体提取策略中体现）：

- `chunk_size`: 1200 tokens

- chunk_overlap: 100 tokens
- encoding_name: cl100k_base

切分方式:

如果输入未预先切块（默认），使用 TokenTextSplitter 在 token 级按窗口滑动分块。
chunk_overlap 让相邻块有重叠，减少截断丢失语义。
空块会被丢弃。

Graph RAG 的召回内容是什么？

传统 RAG:
拼接的文档片段

Graph RAG:
基于知识图谱的召回结果进行二次推理的内容

- 基本原理:**
- 1、图数据库召回的内容是 JSON 关系表形式的结构化数据
 - 2、LLM 对结构化数据进行理解，融合，构建自然语言描述的逻辑关系

Agent 评测指标？

截止 2025 年，业界形成了 7 大 Agent 评测指标

指标类别	核心关注点
功能性能（ <i>Performance</i> ）	响应时延（ <i>Latency</i> ）、吞吐量（ <i>Throughput</i> ）、资源消耗
任务成功率（ <i>Task Success</i> ）	指定目标执行准确度、对话完结率、端到端完成率
鲁棒性（ <i>Robustness</i> ）	抗输入扰动能力、对抗样本攻击抵抗力
安全性（ <i>Safety &</i>	有害内容率、不当偏见输出率、对齐度（与人类价值观的一致性）

指标类别	核心关注点
Alignment)	
可解释性 (Explainability)	关键决策路径可追溯度、决策依据可视化能力
持续学习能力 (Continual Learning)	模型更新后性能回退率、在线增量学习效率
互操作性 (Interoperability)	跨平台 API 兼容度、标准协议遵循度、模块化组合难易度

什么是 Multi-Agent System?

Multi-Agent 系统，简称 **MAS**，是由多个智能体组成的集合。

核心思想是通过多个 **Agent** 的协作与协调，共同完成一个复杂任务，从而实现单个 **Agent** 无法完成的复杂目标。

相比单 **Agent** 系统，**Multi-Agent** 系统具备以下优势：

- **分布式处理**：MAS 支持分布式应用，可以将大型复杂系统分解为多个小型、易于管理的子系统。这使得 MAS 具有良好的模块性、易于扩展性和设计灵活性，降低了系统的总成本和维护难度。
- **协同工作**：MAS 中的 **Agent** 可以相互通信、协商和协作，共同完成一个任务。通过这种协同工作方式，MAS 能够处理单个 **Agent** 无法解决的问题，从而提高系统的整体性能和鲁棒性。
- **自适应性**：MAS 中的 **Agent** 可以根据环境变化自主调整行为和策略，这种自适应性使得 MAS 具有优秀的稳定性和灵活性，能够应对各种复杂场景。

有了解哪些常见 LLM Agent 框架？

AutoGen

AutoGen 是一个用于创建 **Multi-Agent AI** 应用程序的框架，可以自主行动或与人类一起工作。

AutoGen 生态系统提供了创建 AI 代理所需的一切，特别是多代理 workflow——框架、开发人员工具和应用程序。

多智能体系统的工作原理

群聊系统（GroupChat）

- GroupChat 类是多智能体系统的核心，管理多个智能体之间的交互
- 支持初始化消息和对话历史

编排机制（Orchestrator）

- 使用 IOrchestrator 接口决定下一个发言的智能体
- 提供多种编排策略：
 - RoundRobinOrchestrator：轮流发言（所有智能体放入一个列表，轮询）
 - RolePlayOrchestrator：通过管理员智能体决定下一个发言人（管理员智能体会收到角色扮演提示，要求从可用智能体中选择下一个发言者）
 - WorkflowOrchestrator：基于预定义 workflow 决定下一个发言人

工作流系统（Graph 和 Transition）

- 使用 Graph 和 Transition 定义智能体之间的交互规则
- 可以设置条件判断，决定在特定条件下哪个智能体可以发言

执行流程

在 GroupChat.CallAsync 方法中：

- 1、创建编排上下文（包含候选智能体和对话历史）
- 2、使用编排器选择下一个发言的智能体
- 3、调用该智能体生成回复
- 4、将回复添加到对话历史中
- 5、重复直到达到最大轮数或收到终止消息

多智能体交互的底层原理基于以下几个核心概念：

消息传递模式

- 所有智能体共享同一个对话历史，每个智能体都能看到其他智能体的发言，形成一个协作环境。

角色系统

通过 Role 枚举（User, Assistant, System, Function）区分不同类型的消息，帮助智能体理解上下文。

编排机制

通过 IOrchestrator 接口控制哪个智能体在何时发言，支持多种策略：

- 轮询（Round Robin）
- 工作流（Workflow）
- 角色扮演（Role Play）

中间件系统

通过中间件可以在消息传递过程中插入额外的处理逻辑，如日志记录、验证、转换等。

这种设计使得 AutoGen 能够支持复杂的多智能体协作场景，智能体可以通过共享的对话历史进行上下文感知的交互，同时通过工具调用机制与外部系统集成，实现更强大的功能。

大模型微调技术

大模型的训练通常分为预训练、有监督微调和基于人类反馈的对齐三个阶段。**预训练通过海量无标注文本让模型掌握语言规律和通用知识；有监督微调使用高质量的指令数据，教会模型遵循指令、进行对话和完成任务；强化学习阶段（如 RLHF 或 DPO）则利用人类对回复的偏好数据，进一步优化模型，使其输出更符合人类价值观，更安全、有用、无害**

在构造微调数据集的过程中有哪些注意点？

• **数据量太小**：数据显示不足时，模型无法学习到足够的特征规律，如果你调的训练轮数很大，很容易出现过拟合（仅记住少量样本的噪声信息），导致泛化能力差，在新数据上表现不佳。当然不同的微调任务对数据量的要求不一样，**如果是领域知识注入类任务，（7B 模型）至少要 1000 条数据起步**，数据集的数量的扩大也要随着模型参数的变大而增加。另外也别迷信“数据越多越好”，若数据质量差，海量数据反而会放大噪声影响。**优先保证单条数据的有效性，再考虑扩充数量。**

• **噪声数据多**：如文本数据中存在大量错别字、格式混乱，图像数据模糊、标注错误等，会误导模型学习错误的映射关系。所以数据集一定要进行清洗，**文本数据重点筛除乱码、重复内容、与任务无关的段落（比如微调法律模型时混入娱乐新闻）；标注数据需检查标签-内容一致性（如情感分类中“好评”文本误标为“差评”）。**

• **样本偏差严重**：数据分布与实际应用场景差异大（如训练数据中 90%是正面评论，而实际测试数据正负比例均衡），导致模型在真实场景下失效。

• **任务相关性不够**：要模型学会一个领域的知识，不是说这个领域的所有数据都建议用于训练，比如你要微调一个金融问答模型时，就不要混入大量的财经新闻，因为新闻侧重事件描述，而问答题问题解答，数据形式需贴近实际应用场景（如“问题-答案”对）。

• **数据多样性不足**：样本覆盖的场景、特征维度单一（如文本数据仅包含短句子，缺乏长句或复杂句式；图像数据仅包含晴天场景，没有雨天、夜晚等），模型无法适应真实应用中的多样性需求。此外，若指令类型单一（如仅包含「解释型」指令），缺乏「推理型」「操作型」指令，会导致模型在实际应用中能力受限。

数据集构造后 Review 要注意哪些细节

1. **答案与文献事实一致**：无样本量、漏洞类型、数据来源等关键信息错
2. **不超出文献范围**：答案仅包含文献明确提及的内容，不添加未提及的数据集应用场景、评估指标等推断信息。
3. **答案直接回应问题**：不堆砌无关信息（如问题问“数据集来源”，答案不可包含“模型训练效果”）。
4. **问题聚焦数据集属性**：优先保留指向“数据构成、标注方式、样本特征”的问题，剔除与数据集无关的模型方法、政策合规等提问。
5. **无关键信息缺失**：时间范围、数据格式字段、标注一致性指标等需明确（如“2023 年数据”应补充“2023.1-2023.12”）。
6. **术语统一**：相同概念表述一致（如“跨站脚本攻击”与“XSS”需统一为一种说法）。
7. **重复问题合并**：同一数据集的同类问题（如“来源是哪里”与“采集渠道”）仅保留一个。
8. **模糊答案修正**：“质量较高”“包含多种攻击”等表述需补充具体数据（如“标注 Kappa 系数 0.85”“包含 XSS/SQL 注入/CSRF”）。

知识蒸馏

原理

将待压缩的模型作为教师模型，将体积更小的模型作为学生模型，让学生模型在教师模型的监督下进行优化，将学生模型学习到教师模型的概率分布，通过 kl 散度进行控制。

对于大模型的知识蒸馏，主要分为两种：

其一、黑盒知识蒸馏。

使用大模型生成数据，通过这些数据去微调更小的模型，来达到蒸馏的目的。缺点是蒸馏效率低，优点是实现简单。

其二、白盒知识蒸馏。

获取学生模型和教师模型的输出概率分布（或者中间隐藏层的概率分布），通过 kl 散度将学生模型的概率分布向教师模型对齐。下面主要介绍和测试白盒知识蒸馏：白盒知识蒸馏主要在于模型分布的对齐，模型分布对齐主要依赖 kl 散度，对于 kl 散度的使用又有如下几种方式：

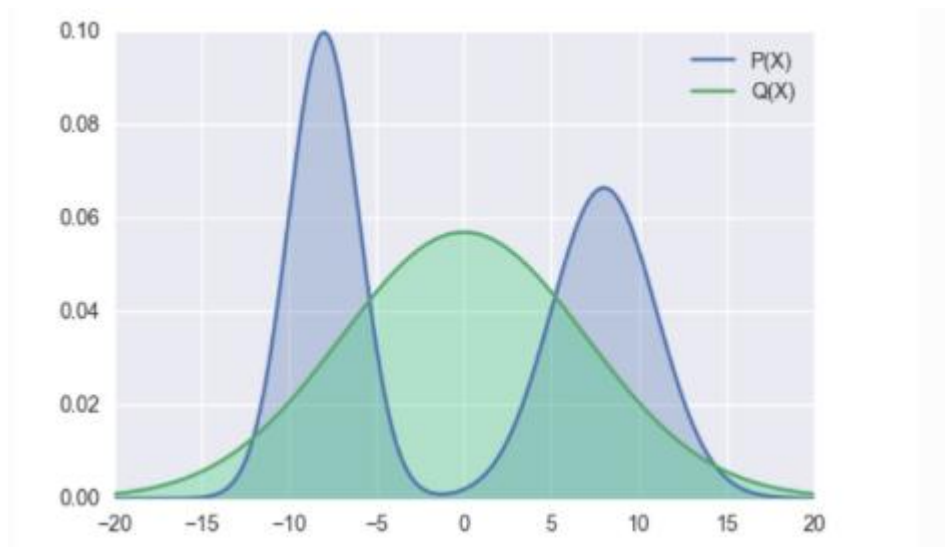
其一、前向 kl 散度。

也就是我们经常说的kl散度。

$$\hat{q} = \operatorname{argmin}_q \int_x p(x) \log \frac{p(x)}{q(x)} dx$$

p 为教师模型的概率分布，q 为学生模型的概率分布，minillm 论文中提到前向 kl 散度可能会使学生模型高估教师模型中概率比较低的位置，结合公式来看，当 p 增大时，为了使得 kl

散度小，则 q 也需要增大，但是当 p 趋于 0 时，无论 q 取任何值，kl 散度都比较小，因为此时 $p(x)\log((p(x)/q(x)))$ 的大小主要受 $p(x)$ 控制，这样起不到优化 q 分布的效果，可能会使 q 分布高估 p 分布中概率低的位置。下图展示了前向 kl 散度的拟合情况，前向 kl 散度是一种均值搜索，更倾向于拟合多峰

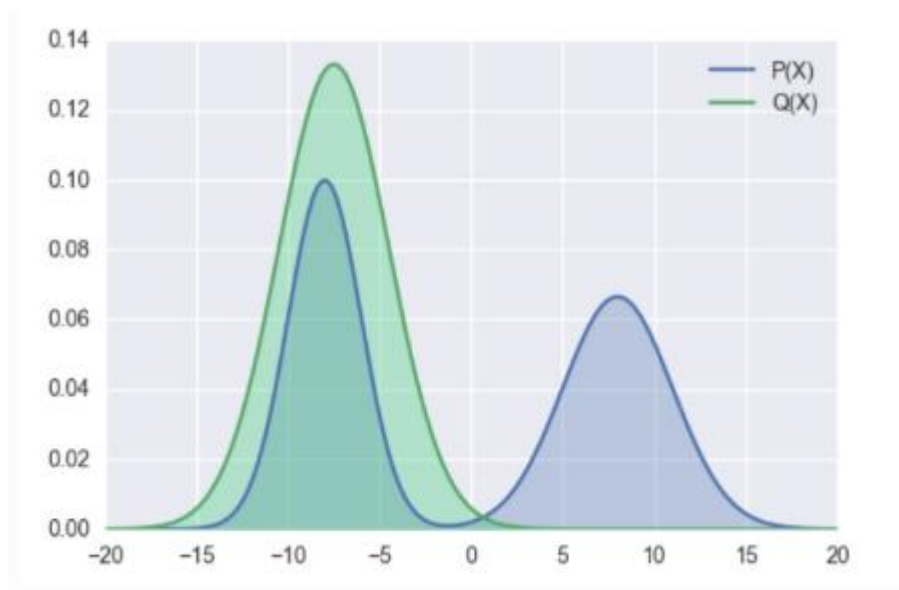


其二、反向 kl 散度。

为了缓解前向 kl 散度的缺点，提出了反向 kl 散度

$$\hat{q} = \operatorname{argmin}_q \int_x q(x) \log \frac{q(x)}{p(x)} dx$$

p 为教师模型的概率分布， q 为学生模型的概率分布，当 p 趋于零时，为了使 kl 散度小， q 也需趋于 0。 minillm 论文中说对于大模型的知识蒸馏，反向 kl 散度优于前向 kl 散度，但是也有其他论文说反向 kl 散度不一定比前向 kl 散度更优，实际选择中，可能要基于实验驱动。反向 kl 散度是一种模式搜索，更倾向于拟合单个峰



其三、偏向前 kl 散度。

对学生模型的分布和教师模型的分布进行加权作为学生模型的分布。

其四、偏向反 kl 散度。

对学生模型的分布和教师模型的分布进行加权作为教师模型的分布。

三、测试

qwen2.5-3b 作为教师模型，qwen2.5-0.5b 作为学生模型

流程如下：

1、将 qwen2.5-3b 模型在指定数据集上微调（训练数据 5000 条，测试数据 1000 条，测试准确率为 81.1%）

2、探索如下三种方案下的蒸馏效果（均使用前向 kl 散度）：

2.1 不微调学生模型+kl 散度损失

蒸馏 1 个 epoch，准确率 70.5%

蒸馏 2 个 epoch，准确率 73%

2.2 微调学生模型（模型准确率 80.3%）+kl 散度损失

蒸馏 2 个 epoch，准确率 61.9%

2.3 不微调学生模型+kl 散度损失和交叉熵损失加权

蒸馏 2 个 epoch，70.5%

3、上述实验中只使用 kl 散度的效果最好，如下实验中使用 kl 散度的变种进行测试，经过测试，效果都不如前向 kl 散度效果好。

3.1 反向 kl 散度

准确率只有 54%

3.2 偏向前向 kl 散度

损失下降异常，效果很差，不断重复输出。

由于资源和时间的限制，所有测试均保持相同的超参数，未针对不同损失设置不同超参数。

数据增强面临的问题

- **数据稀缺性：**高质量语料（如学术文献、专业文本）总量有限，公开数据集（如 C4、RefinedWeb）经严格过滤后仅保留不到 10% 的原始内容，难以支撑模型的持续扩展和训练。

- **重复退化问题：**在传统深度学习中，重复训练是可以继续提升模型性能的，但 LLM 训练中，过度重复会导致模型泛化能力下降、优化稳定性变差，尤其是参数规模超千亿的模型。

例如，当使用 1950 亿 tokens 的高质量数据训练 130 亿参数模型时，若直接重复 10 次，模型在推理任务（如 GSM8K 数学题）的准确率会下降 23%，验证损失上升 17%。这表明：数据重复并非简单的“量的补充”，而是需要质的多样性重构。

数据增强的方法

MGA

字节跳动 Seed 团队最近发表了一篇论文:《Reformulation for Pretraining Data Augmentation》

其中提出了一种新的 Massive Genre-Audience (MGA) 方法,通过轻量级框架将现有语料系统重构为多样化变体,核心思路是:基于不同“体裁(Genre)”和“受众(Audience)”生成内容变体,在保留核心知识的同时创造语义丰富的新数据。

虽然论文主要是表述预训练的数据集增强,但其思路同样适用于在模型微调阶段的数据集构造。

“Massive Genre-Audience”本质上是一种数据增强策略,通过系统化地生成海量“类型-受众”组合,将现有文本重构为多样化的变体,既保留核心信息,又覆盖不同表达形式和读者群体,对模型训练数据进行增强,从而提升模型性能。

实现步骤:

阶段 1: Genre-Audience 对生成

利用 3.3B 参数的混合专家模型 (MoE),从原始文档中自适应提取 5 组不同的体裁-受众组合。例如,一篇科普文章可被映射为“学术论文-科研人员”、“对话体-老年人”、“教科书-中学生”等组合,每个组合定义内容的表达框架(如结构、语言风格、知识深度)和受众特征(如年龄、教育背景、专业领域)。

阶段 2: 文本重构

使用量化后的轻量级工具模型,根据每对 Genre-Audience 的要求重构文本。例如,将“气候变化”原始文本重构为面向小学生的对话体故事时,会简化术语、增加具象案例;重构为学术报告时,则强化数据论证和理论框架。

质量控制: Limited Consistency 准则

引入 LLM 裁判模型,以“有限一致性”为标准评估重构文本:允许风格、表达顺序的差异,但要求核心信息可追溯至原始文本。例如,若重构文本丢失所有原始信息点或语义偏差过大,则判定为无效(评分<3)。

相比现有数据增强方案(如 Phi-4、Cosmopedia),MGA 的核心优势在于:

- **不依赖超大模型:** 无需 120 亿参数以上的生成模型(如 GPT-4),仅用 3.3B MoE 模型即可实现高质量重构,计算成本降低 40%。
- **免复杂种子系统:** 传统方法需预定义种子模板(如 QA 对、维基风格),而 MGA 直接从原始文本动态生成 Genre-Audience 对,避免人工设计的局限性。
- **平衡多样性与保真度:** 通过 prompt 工程调节“信息保留”与“内容变异”的权衡。例如,严

格模式（SLM-Strict）要求 80% 以上核心信息保留，适合知识密集型任务；宽松模式（SLM-Relaxed）允许更多创意扩展，适合泛化能力训练。

SFT

有监督微调

优化技巧：

1. Chat Template 对齐：

在大模型微调（尤其是对话模型微调）中，Chat Template（对话模板）是一种格式化对话数据的规范或模板，用于定义用户与模型之间的交互格式。它规定了如何将多轮对话（包括用户输入、模型回复、角色信息等）组织成模型可理解的文本格式，确保模型在训练和推理时能正确区分对话角色、理解对话历史。

Hugging Face 的 `DataSet.from_list()`集成了 chat template 对齐的方式

2. Completions Only

输入大模型的是整个文本对话，大模型会对整个文本对话进行学习，传统的学习过程会对大模型输出的每个 token 计算 loss，为了让大模型将注意力集中在怎么回答问题，在计算交叉损失函数时，只对模型的回答部分计算 loss，实现方式就是通过一个 mask，表示要对哪些生成的 token 计算 loss，这个 mask 是根据开始回答部分的特殊 token 来计算需要计算 loss 部分的初始位置

3. NEFTune

在大模型微调中，NEFTune（Noisy Embeddings Fine-tuning，带噪声嵌入微调）是一种简单而有效的正则化技术，通过在模型的输入嵌入（Embeddings）中添加可控噪声，提升模型的泛化能力和鲁棒性，尤其在小数据集微调场景中效果显著。其原理是对向量化的输入数据进行空间上的微小扰动，由于高维空间中相似的向量必然有相似的语义，因此能够起到泛化数据集的作用

Lora

参数高效的一种微调方法，节约显存（减少显存占用的主要原因是训练参数变小了（比如只对 qkv 层做 LoRA）），训练快，效果损失较小（相对于全参数微调），推理的时候不增加耗时，可以做一个插入式组件使用，缺点是还是会有一些效果的损失

原理：

- 1.找到目标矩阵（LLM 中的线性变换层）
- 2.低秩分解（ $W' = W + BA$ ，其中 B、A 是小矩阵）
- 3.只训练分解矩阵（A、B 可训练，W 冻结）
- 4.基于内在低秩假设：模型适应新任务时，权重变化具有低秩特性
- 5.选择性微调：通常只微调注意力机制中的 q_proj、v_proj 等关键层
- 6.缩放控制：通过 α/r 系数控制 LoRA 适配的强度
- 7.高效部署：推理时可将 LoRA 权重合并回原模型，无额外开销

LLM 中存在大量的矩阵运算，Lora 就是通过对目标矩阵进行低秩分解，训练小矩阵的参数，

通过 α/r 系数控制 LoRA 适配的强度，推理时将 Lora 权重合并即可

低秩分解从效果上来理解的话就是用两个更少参数的矩阵来表示参数更多的大矩阵，因此是存在特征的丢失的，而秩的大小影响了我们训练量以及微调的效果

Lora 微调重点是保持参数尺寸的最小化，原始模型的矩阵不变，只训练不到 10%的参数

QLora

核心思想：在 LoRA 的基础上引入量化技术，将【预训练模型】的权重从高精度（如 FP16）量化为低精度（如 4-bit、8-bit），同时保持 LoRA 的低秩更新机制。通过量化压缩【原始模型】的存储，进一步降低内存需求，使大模型微调能在消费级硬件（如单张 GPU）上实现。

有了 SFT 为什么还要有强化学习（RL）？

SFT 和预训练本质上都是提供给 LLM 足够的样本学习规律，而 LLM 本身不清楚样本知识间的关系，通过 RL，能让 LLM 探索不同的 token 序列，并根据哪些输出最有用来获得反馈（奖励信号），因此，模型通过 RL 能更好的与人类的意图对齐

RLHF 的优点：

广泛适用性：RLHF 可以应用于任何领域，包括创意写作、诗歌、总结等开放性任务。

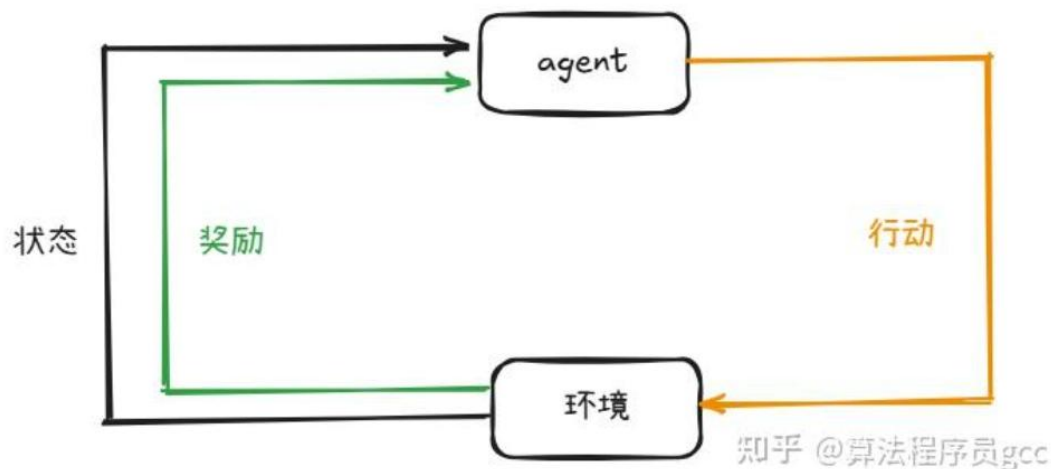
简化评估：对输出进行排名比生成人工标签或创意输出更容易。

RLHF 的缺点：

奖励模型的局限性：奖励模型是近似的，可能无法完美反映人类的偏好。

RL 模型可能会利用奖励模型的漏洞，生成荒谬的输出但仍获得高分，尤其是在训练时间过长的情况下。

典型 RL 的组成、基本概念



- **Agent（智能体）** 这是学习的主体，负责在环境中采取行动。
- **Environment（环境）** 智能体与之交互的外部世界，它会根据智能体的行动给出反馈。
- **State（状态）** 环境在某一时刻的具体情况，智能体根据状态决定行动。

在每个时间点，智能体（Agent）会在环境（Environment）中执行一个动作（Action），这个动作会将环境从当前状态（State）转移到新的状态。同时，智能体会收到一个奖励（Reward），这是一个数值形式的反馈，用于评估动作的好坏。正奖励鼓励智能体重复该行为，而负奖励则起到抑制作用。

通过不断从不同状态和动作中收集反馈，智能体逐渐学习出最佳策略（Policy），以在长期内最大化累积奖励。这种学习过程使智能体能够在复杂环境中做出更优的决策。

策略

策略（Policy）是智能体的决策规则。如果智能体遵循一个好的策略，它将在每个状态下做出正确的决策，从而在多个步骤中累积更高的奖励。用数学术语来说，策略是一个函数（ $\pi\theta(a|s)$ ），它定义了给定状态下选择不同动作的概率。

价值函数

价值函数（Value Function）用于评估某个状态的好坏，考虑的是长期期望奖励。对于 LLM（大语言模型）而言，奖励可能来自人类反馈或奖励模型。

Actor-Critic 架构

Actor-Critic 是一种流行的强化学习框架，结合了两个关键组件：

1. **Actor（演员）** 负责学习和更新策略（ $\pi\theta$ ），决定在每个状态下应该采取哪个动作。
2. **Critic（评论者）** 评估价值函数（ $V(s)$ ），为 Actor 提供反馈，告知其选择的动作是否带来了好的结果。

工作原理：

- Actor 基于当前策略选择一个动作。
- Critic 评估结果（奖励 + 下一个状态）并更新其价值估计。

Critic 的反馈帮助 Actor 优化策略，使未来的动作能够获得更高的奖励。

将其与 LLM 结合

在 LLM 的上下文中：

- 状态可以是当前的文本（提示或对话）。
- 动作是生成的下一个 token（词或子词）。
- 奖励模型（例如人类反馈）告诉模型生成的文本是好是坏。
- 策略是模型选择下一个 token 的规则。
- 价值函数评估当前文本上下文对最终生成高质量响应的贡献程度。

RL 算法

PPO

RL-PPO 中的四大模型

- Actor Model (演员模型 π_θ)：用SFT后的模型作为初始模型，通过PPO训练调整参数，得到最终的模型，使其生成的回答更符合人类偏好，参数可变；
- Reference Model (参考模型 π_{ref})：SFT后的模型，用于限制Actor Model不要偏离Reference Model太远，参数冻结；
- Reward Model (奖励模型 r_θ)：对模型回答打分，表示贴合人类偏好的程度，训练完后，在PPO中是参数冻结的；
- Critic Model (评论家模型 V_π)：用于预测期望总收益，为优势函数 A_t 提供基线，使策略更新更稳定，参数可变；

RLHF 训练流水线

严格遵循 InstructGPT 的三阶段训练逻辑，SFT、RW 和 PPO

阶段一：SFT

让预训练模型学习人类如何回应查询，建立基础的指令遵循能力。

具体的执行过程如下：

- **数据输入**：筛选高质量的<人类查询，人类回应> 标注数据对，例如各类问答、指令任务数据。
- **微调过程**：使用这些标注数据对预训练语言模型进行微调，使模型输出贴合人类的回答风格和逻辑，而非单纯的文本续写。
- **输出产物**：得到 SFT 模型，即 PPO 阶段中演员模型（Actor Model）的初始版本，具备基础的指令理解和回应能力。

阶段二：奖励模型训练（Reward Model, RW）

我们知道，RLHF 的终极目标是让模型学会按人类喜好作出回答，为了实现这个目标，需要有个模型评估模型生成内容的好坏，这就是奖励模型（Reward Model, RW）要干的活。

RW 模型训练的大体过程：提供一组提示，让 LLM 生成多个回答，由人工按照既定的规则，对同个问题的多个回答排序，利用标注好的排序数据训练一个可量化人类偏好的 RW。

假设有个问题 x ，模型给出两个不同的回答，由人工标注两种回答： y_+ 为好的回答， y_- 为不好的回答。我们希望训练出的 RW 模型 $\pi(\cdot)$ 能打出和人类一致的分

即 $\pi(x, y_+) > \pi(x, y_-)$

基于概率形式的目标函数为

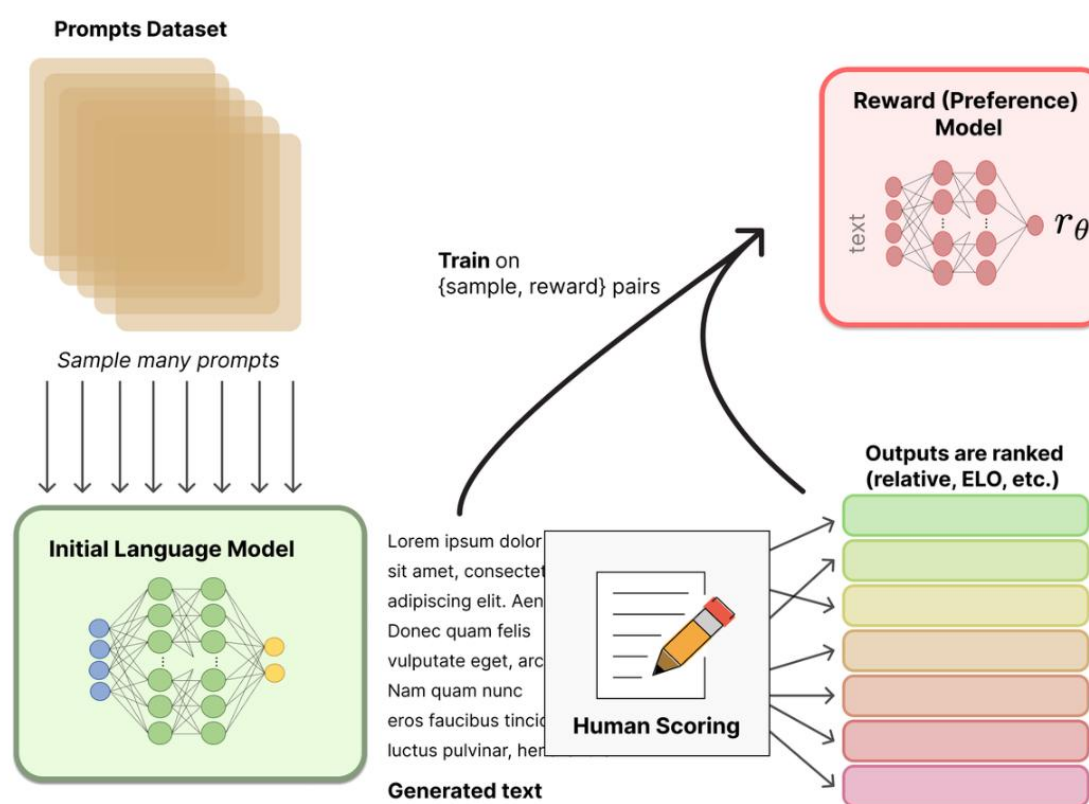
$$P(y_+ > y_- | x) = \sigma(\pi(x, y_+) - \pi(x, y_-)),$$

其中 σ 是 sigmoid 函数，取值范围为 0-1 之间， $\pi(x, y_+) - \pi(x, y_-)$ 差值越大概率越接近 1，说明 RW 更有可能选择好的答案。

基于所有样本的损失函数为

$$L_{RW}(\theta) = -E_{(x, y_+, y_-) \sim D_{\text{pref}}} [\log \sigma(\pi(x, y_+) - \pi(x, y_-))]$$

这样我们就能训练出一个 RW 模型，识别模型输出内容的好坏，分数越高，表示模型输出内容越贴合人类偏好。



阶段三：PPO

以奖励模型的评分为反馈，通过 RL-PPO 进一步优化 SFT 后的模型，使其生成的回答更符合人类偏好。

1). 输入提示 x 到参考模型 π_{base} (旧策略) 和当前微调的模型 π_{θ} (新策略，对应上图中的 π_{PPO})。得到 token 序列

$$y \sim \pi_{base}(\cdot | x), y \sim \pi_{\theta}(\cdot | x)。$$

2). 计算奖励分数，将微调模型 π_{θ} 生成的序列 y 输入到奖励模型 (Reward Model, RW)，得到奖励分数 $r_{\theta}(x, y)$ 。

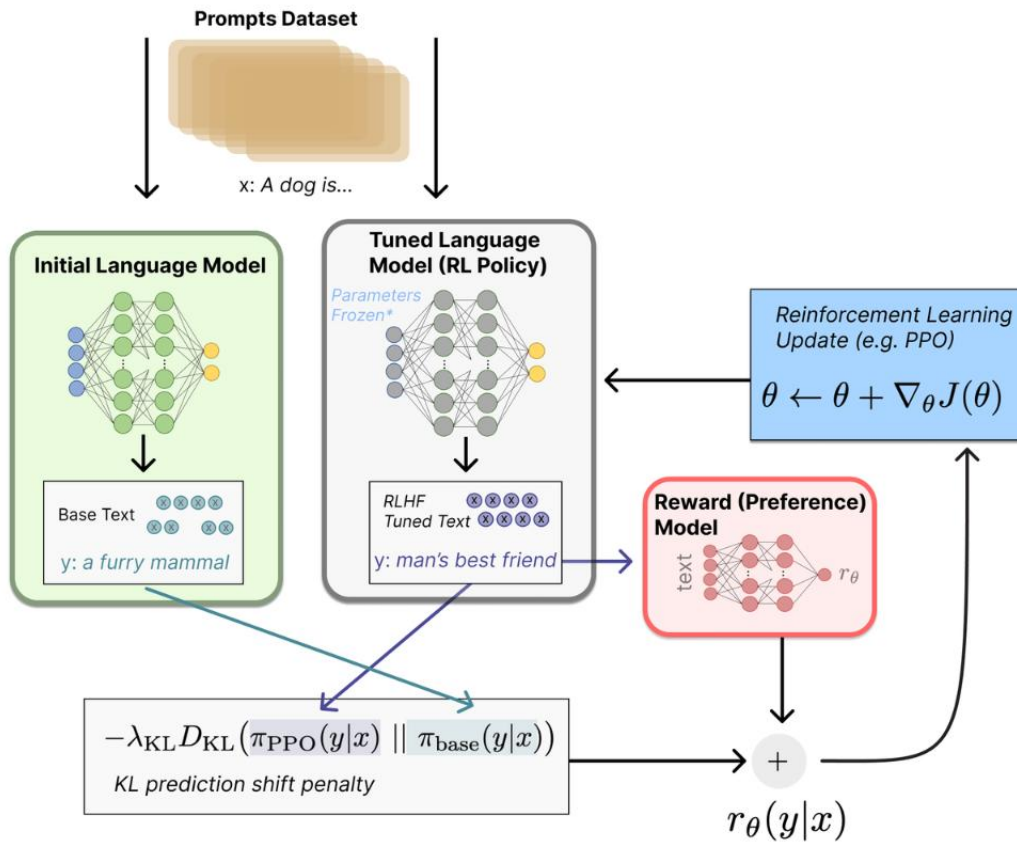
3). 评论家模型预测 V_t ，根据价值函数 V_t 和奖励分数 $r_{\theta}(x, y)$ 计算优势 A_t ，用以评估微调模型 π_{θ} 本次输出的 token 比随机输出的优劣，辅助 PPO 决定该惩罚、奖励程度，让模型 π_{θ} 的训练更稳健。

4). PPO 更新模型 π_{θ} 参数，这里为了防止策略模型 π_{θ} 过度偏离参考模型 π_{base} ，有两种形式保证模型在有限的空间里微调参数，一种是引入 PPO-KL 散度，

$$L_{PPO-KL}(\theta) = E_{\tau \sim \pi_{base}} [\pi_{base}(a_t | s_t) \pi_{\theta}(a_t | s_t) A_t - \beta D_{KL}(\pi_{base}(\cdot | x_t) || \pi_{\theta}(\cdot | x_t))]]$$

另一种 PPO-CLIP 比较直接，在目标函数中，限制新旧模型的差距在约定的区间内：

$$L_{ppo-clip}(\theta) = E_{\tau \sim \pi_{base}} [\min(\pi_{base}(a_t | s_t) \pi_{\theta}(a_t | s_t) A_t, \text{clip}(\pi_{base}(a_t | s_t) \pi_{\theta}(a_t | s_t), 1-\epsilon, 1+\epsilon) A_t)]$$



GRPO

为什么选择 GRPO 而不是 PPO?

PPO 在推理任务中表现较差的原因包括:

- **依赖评论者模型 (Critic Model)**: PPO 需要一个单独的评论者模型, 这会增加内存和计算开销。
- **训练复杂性**: 评论者模型在处理细致或主观任务时可能变得复杂。
- **高计算成本**: RL 流水线需要大量资源来评估和优化响应。
- **绝对奖励评估**: PPO 依赖于单一标准判断答案的好坏, 难以捕捉开放性任务的细微差别。

GRPO 如何解决这些挑战? GRPO 的贡献?

提出了一个不需要训练状态价值网络, 可以估算出每个 token 优势值的方法, 而且该方法更适合大模型生成强化学习的场景

GRPO 通过相对评估消除了对评论者模型的依赖。响应在一个组内进行比较, 而不是通过固定标准判断 (评论家模型)。可以将其类比为学生之间互相比较答案, 而不是由老师单独评

分。随着时间的推移，模型的表现会趋向于更高质量。

GRPO 的训练过程

GRPO 通过修改损失计算方式，保持其他训练步骤不变：

收集数据：

查询（问题）和响应（答案）。

旧策略（模型的旧快照）为每个查询生成多个候选答案。

分配奖励：

每个组内的响应都会被评分（即“奖励”）。

计算 GRPO 损失：

测量新策略生成过去响应的可能性。

评估这些响应的相对质量（更好或更差）。

应用裁剪以防止极端更新，最终得到一个标量损失。

反向传播 + 梯度下降：

反向传播计算每个参数对损失的贡献。

梯度下降更新参数以减少损失。

更新旧策略：

偶尔更新旧策略，使其与新策略匹配，为下一轮比较刷新基准。

GRPO 如何去掉评论家模型？为什么去掉？

PPO算法计算每个Token的Advantage



在 PPO-for-LLM 中，将 KL 散度作为逐 token reward 并用于训练 value network，往往会导致 value - reward 语义不匹配，从而使 GAE 优势估计偏差大、噪声高

GRPO 通过对同一 prefix 的并行采样构造 on-policy 的相对 baseline，避免了 PPO-for-LLM 中显式 critic 带来的 value - reward 语义不一致和非平稳问题。它并不是在学习一个更“科学”的价值函数，而是用条件化的 Monte Carlo baseline 替代了 value function，在 LLM 的 sequence-level reward 场景下更加稳定和一致

其实 PPO 和 GRPO 的本质区别就是 GRPO 在目标函数的计算上使用了自己提出的优势函数替换 PPO 目标函数中的 GAE 优势函数

3.8
5.2
6.1

$$\tilde{r}_1 = -1.06 \quad \tilde{r}_2 = 0.14 \quad \tilde{r}_3 = 0.92$$

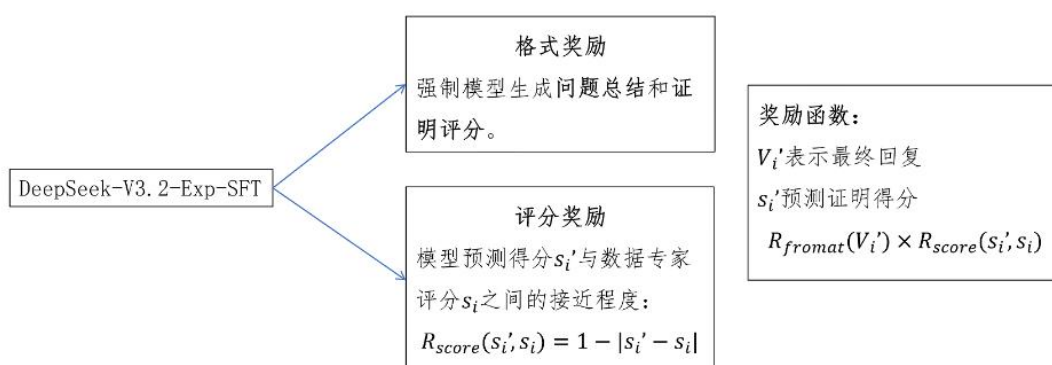
GRPO: Group Relative Policy Optimization
群体相对策略优化

2. 使用 DeepSeek-V3.2-Exp-Thinking 多轮迭代，生成候选证明。

3. 让数学专家按照评估标准对证明进行打分。得到数据 $D_v = \{(X_i, Y_i, s_i)\}, s_i \in \{0, 0.5, 1\}$

X_i 是数学题； Y_i 是模型生成的候选证明； s_i 是数学专家给这个证明的打分，1 表示证明完全没有问题，0.5 代表基本正确，但是有瑕疵，0 代表证明有明显错误；

4. 训练验证器：拿到数据集: $D_v = \{(X_i, Y_i, s_i)\}$ ，并不是训练证明的生成器，而是训练证明的验证器。验证器，可以给证明评分，是生成器的 Reward Model。GRPO 训练验证器，奖励函数由格式奖励 x 评分奖励组成；



验证器问题：

当验证器给出评分小于 1 时，给出的问题总结可能是错的。

目标：可以根据验证给出的问题描述，修改证明过程。

解决方法：



让根据 4 训练方法得到的初始验证器生成一批验证信息， $\{X_i, Y_i, V_i\}$ 中 X_i 表示数学题，

Y_i 表示数学候选证明， V_i 表示验证器给出的验证信息， ms_i 表示数学专家给验证信息所打的分(0,0.5,1)，从而生成元验证数据集 D_{mv} ，通过元验证数据集，训练一个元验证模型，也是采用 GRPO 训练，奖励函数和训练验证模型一致

5. 增强训练验证器：

加入元验证器：

$$R_v = R_{format} \times R_{score} \times R_{meta}$$

$$\mathcal{D}_v = \{(X_i, Y_i, s_i)\},$$

$$s_i \in \{0, 0.5, 1\}$$

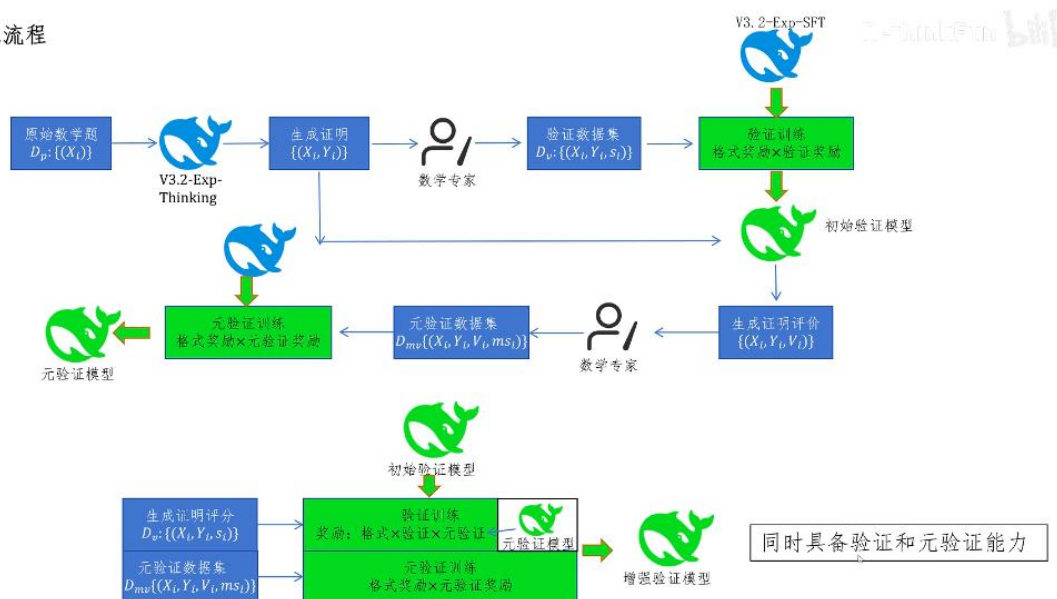
$$\mathcal{D}_{mv} = \{(X_i, Y_i, V_i, ms_i)\},$$

$$ms_i \in \{0, 0.5, 1\}$$

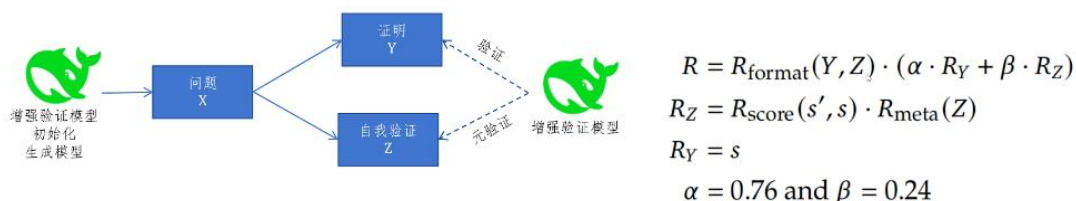
奖励函数为格式奖励 $\times$ 验证奖励 $\times$ 元验证奖励, 元验证奖励用于奖励其对证明过程的验证是否正确, 元验证奖励由元验证模型打分, 同时用了训练元验证器的数据集 $\mathcal{D}_{mv}$, 让验证器同时具备验证和元验证的能力

在 $\mathcal{D}_v$ 验证集上, 验证器的证明问题描述平均质量分从 **0.85 提升到 0.96**, 同时保持相同的证明得分预测准确率。

训练流程



6. 生成模型训练: 让模型在一次生成中, 生成证明 Y 和自我验证 Z 。验证格式与验证器相同。



其奖励函数中： $R_{\text{format}}(Y, Z)$ ：输出证明 Y 和自我验证 Z 的格式奖励

R_Y ：证明奖励 Y ，为增强验证器给出的打分 s ，权重 $\alpha=0.76$ 表明模型更鼓励给出正确的证明

R_Z ：自我验证的奖励，为生成模型给自己证明的打分与增强验证模型的打分的一致性奖励

$R_{\text{score}}(s, s)$ 乘以 $R_{\text{meta}}(Z)$

$R_{\text{meta}}(Z)$ ：生成模型找出自身问题的打分，是增强验证器的元验证给出的打分

总结：这个奖励函数鼓励模型给出正确的证明，在证明错误时，如果能够验证出自己的错误，也可以得到奖励，如果证明错误，并且没有验证出来，得分最低

7. 迭代循环：

生成模型和验证模型是一个互相促进的关系，验证模型提供监督信号，提升生成模型，生成模型提升证明的能力，让验证模型越来越难验证生成的证明是否正确了，利用这一点，可以生成更好的对验证模型的训练数据，从而提升验证模型，两个模型互相共享权重，迭代提升，但这个提升只能逼近其预训练的极限



迭代训练验证器的数据收集过程：

1. 对于生成器生成的每个证明，用验证器生成 n 份相互独立的验证分析。
2. 对于发现问题的分析，再生成 m 份元验证分析。
3. 对于一个证明，考察验证分析中找出问题，并给出最低分(0 或 0.5)的验证分析。如果有 k 条元验证判定证明确实有误，则给这个证明最低分(0 或 0.5)。（高置信错误样本）如果所有元验证都认为判定无效，则给这个证明 1 分。（高置信正确样本，纠错）否则丢弃该证明数据，或交给数学专家标注。（模型体系内部已经无法形成稳定共识）
4. 收集上述的数据用于迭代训练验证模型。

模型量化

什么是量化？

把 Float 类型（FP32，FP16）的模型参数和激活值，用整数（Int8，Int4）来代替。同时尽可能减少量化后模型推理的误差

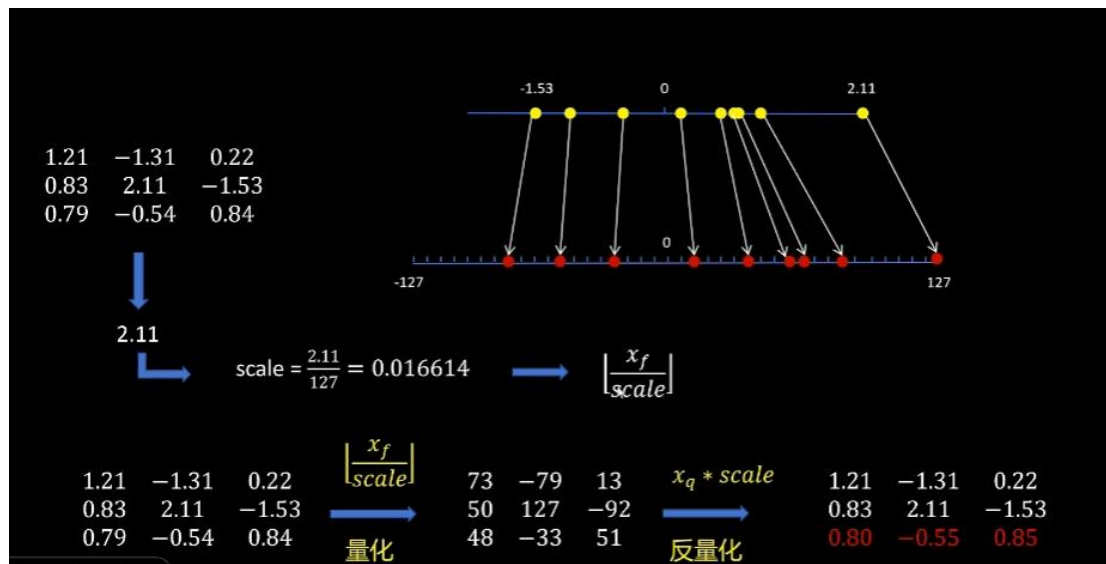
模型为什么需要量化？

1.为什么需要量化	
Llama 13B	FP32 52GB
	FP16 26GB
	Int8 13GB
	Int4 6.5GB

1. 量化可以减少模型的存储大小以及推理时占用的显存大小
2. 大模型推理过程中，制约模型推理速度的关键是显存带宽，而模型量化后，因为传输的数据小，所以可以减少推理过程中数据交换所占用的时间，从而提高推理速度
3. 量化后（如浮点->整型），显卡对整数的运算速度快于浮点型数据，从而提高推理速度

量化方式（包含量化和反量化）

对称量化



找到浮点数中绝对值最大的数，用其绝对值除以缩放后最大值得到比例系数

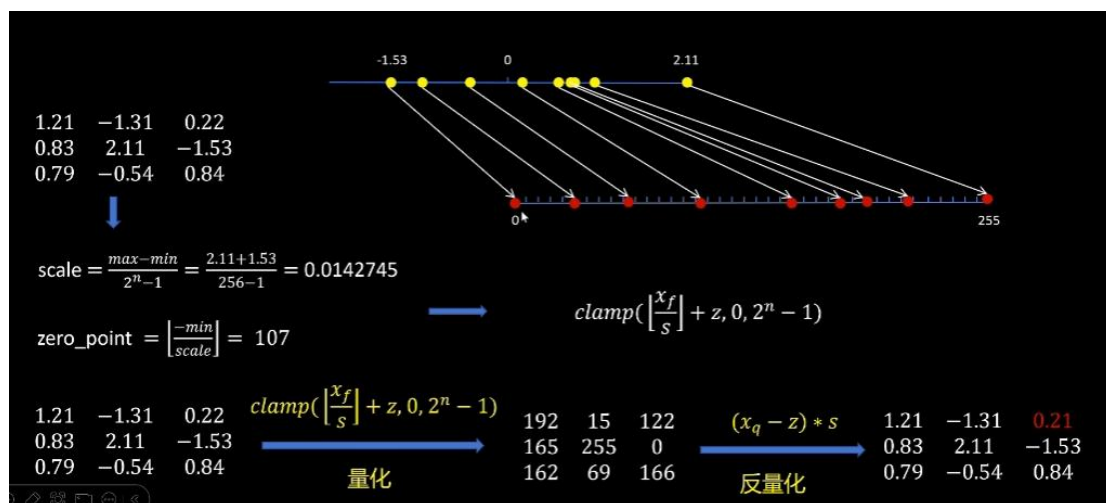
量化：浮点数除以缩放系数取整

反量化：整数乘以缩放系数

优点：计算简单

缺点：缩放后最小值端未对应最小浮点数，可能存在浪费，精度低

非对称量化



比例系数 = (浮点数最大值 - 最小值) / (2^n - 1)

n 表示量化位数

Zero_point 主要是对缩放后的值进行平移，让量化后变量处于 0~2^n - 1 之间

量化：浮点数除以缩放系数取整+zero_point，如果小于 0，则取 0，大于 2^n-1 ，取 2^n-1

反量化：（整数-zero_point）* 比例系数

优点：精度高

缺点：计算复杂

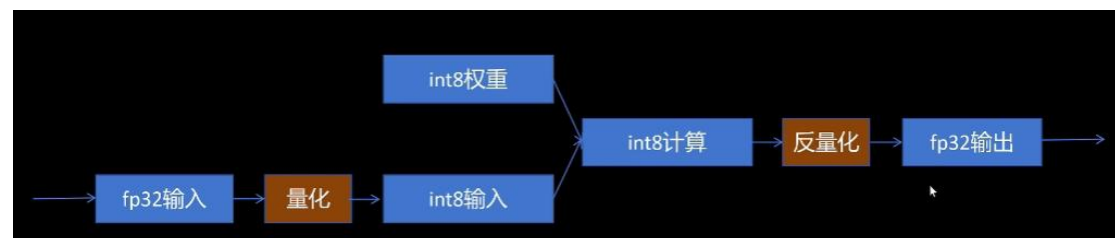
为什么量化对神经网络精度影响不大？

- 因为一般权重和输入都经过 **Normalization**，基本数值范围都不大。
- 因为激活函数，数值影响会被平滑。
- 因为对于绝大部分神经网络都是进行分类，最后都是概率值，只要最后某种类别概率高于其他类别就可以，不需要绝对数值。

训练后动态量化

过程：

- 1.将训练好的模型权重量化为 **int8**，并保存量化参数。
- 2.在模型推理时，对每一层输入的 **fp32** 激活值，动态进行进行量化为 **int8**。
- 3.在每一层对量化后的 **int8** 权重和 **int8** 激活值进行计算。
- 4.在每一层输出时将结果反量化为 **fp32**。
- 5.将 **fp32** 激活值传入到下一层。



代码实现：

```
model_int8 = torch.ao.quantization.quantize_dynamic(model, {torch.nn.Linear}, dtype=torch.qint8)

torch.qint8

tensor([[[-0.8211, 0.1416, 0.9627],
        [-1.9537, 0.5380, 1.6989],
        [-3.6243, 1.7555, 1.6423]], size=(3, 3), dtype=torch.qint8,
        quantization_scheme=torch.per_tensor_affine, scale=0.028314674273133278,
        zero_point=0])

torch.int_repr(qint8_tensor)

tensor([[[-29, -69, -128],
        [ 5, 19, 62],
        [ 34, 60, 58]], dtype=torch.int8])
```

训练后动态量化的问题：

- 每一次推理每一层都要对输入统计量化参数，耗时。
- 每一层计算完都转化为 fp32，存入显存，占用显存带宽。

训练后静态量化

解决了训练后动态量化的问题：

1. 每一次推理每一层都要对输入统计量化参数，耗时。
用有代表性的输入数据跑一遍整个网络，通过统计得到每层大概得量化参数。
2. 每一层计算完都转化为 fp32，存入显存，占用显存带宽。
这一层的输出是下一层的输入。下一层还是要量化，不如在这一层直接量化好再传给下一层。

过程：

1. 将训练好的模型权重量化为 int8，并保存量化参数。
2. 校准（calibration）：利用一些有代表性的数据进行模型推理，用这些数据在神经网络每一层产生的激活值计算出激活值的量化参数。这样就不用推理时每次根据实际激活值计算量化参数。
3. 在每一层对量化后的 int8 权重和 int8 激活值进行计算。
4. 在每一层输出时将结果反量化为 fp32，同时根据校准产生的激活值量化参数，把激活值量化为 int8，把量化参数放入量化后的激活值中。
5. 将 int8 的激活值和它的量化参数传入到下一层。



代码：

```
class MyModel(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.quant = torch.ao.quantization.QuantStub()
        self.linear1 = torch.nn.Linear(3, 3, bias=False)
        self.relu = torch.nn.ReLU()
        self.linear2 = torch.nn.Linear(3, 1, bias=False)
        self.dequant = torch.ao.quantization.DeQuantStub()

        model.qconfig = torch.ao.quantization.get_default_qconfig('x86')
        model_prepared = torch.ao.quantization.prepare(model)
        model_prepared(test_feature)
        model_int8 = torch.ao.quantization.convert(model_prepared)

    def forward(self, inputs):
        q_inputs = self.quant(inputs)
        outputs = self.linear1(q_inputs)
        outputs = self.relu(outputs)
        outputs = self.linear2(outputs)
        f_outputs = self.dequant(outputs)
        return f_outputs
```

模型结构中定义量化、反量化占位符，在 forward 中需要量化的部分前后分别调用量化和反

量化，在主函数中配置静态量化的配置，prepare 构建一个待量化模型，在构建好的数据集上运行一遍得到量化参数，之后调用 convert 即可得到量化后的 int8 模型

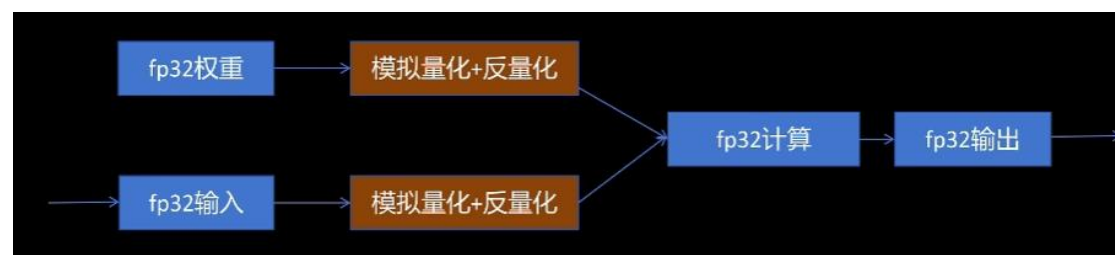
实验表明，同一个训练好的网络，量化前后 mse 的误差在 0.2 左右，针对训练好的模型进行量化总是会有误差，因此提出了量化感知训练

量化感知训练

在网络训练过程中，模拟量化，让模型在训练过程中就能调整参数，让它更适合量化，提高量化后模型的精度。

过程：

1. 加载 fp32 的模型参数。
2. 输入 fp32 的激活值。
3. 通过在网络里插入模拟量化节点 (fake_quantization) 来分别对模型参数和激活值进行量化和反量化。从而引入量化误差。
4. 模型在 fp32 精度下进行计算。
5. 计算后的激活值传入下一层。



代码实现：

```
class MyModel(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.quant = torch.ao.quantization.QuantStub()
        self.linear1 = torch.nn.Linear(3, 3, bias=False)
        self.relu = torch.nn.ReLU()
        self.linear2 = torch.nn.Linear(3, 1, bias=False)
        self.dequant = torch.ao.quantization.DeQuantStub()

    def forward(self, inputs):
        q_inputs = self.quant(inputs)
        outputs = self.linear1(q_inputs)
        outputs = self.relu(outputs)
        outputs = self.linear2(outputs)
        f_outputs = self.dequant(outputs)
        return f_outputs

model.qconfig = torch.ao.quantization.get_default_qconfig('x86')
model_prepared = torch.ao.quantization.prepare_qat(model)
train_loop(model_prepared)
model_int8 = torch.ao.quantization.convert(model_prepared)
```

结构定义与训练后静态量化相同，主函数中调 prepare_qat 创建待量化感知训练模型，在数据集上进行训练后，convert 得到 int8 模型，实验表明，量化感知训练后的 mse 要低于训练后静态量化

量化感知训练（QAT）的核心缺陷是：

它在训练时“假装”量化、在推理时“真的”量化，两者始终不完全一致；

同时训练成本高、稳定性差、对大模型（尤其是 LLM）并不友好。

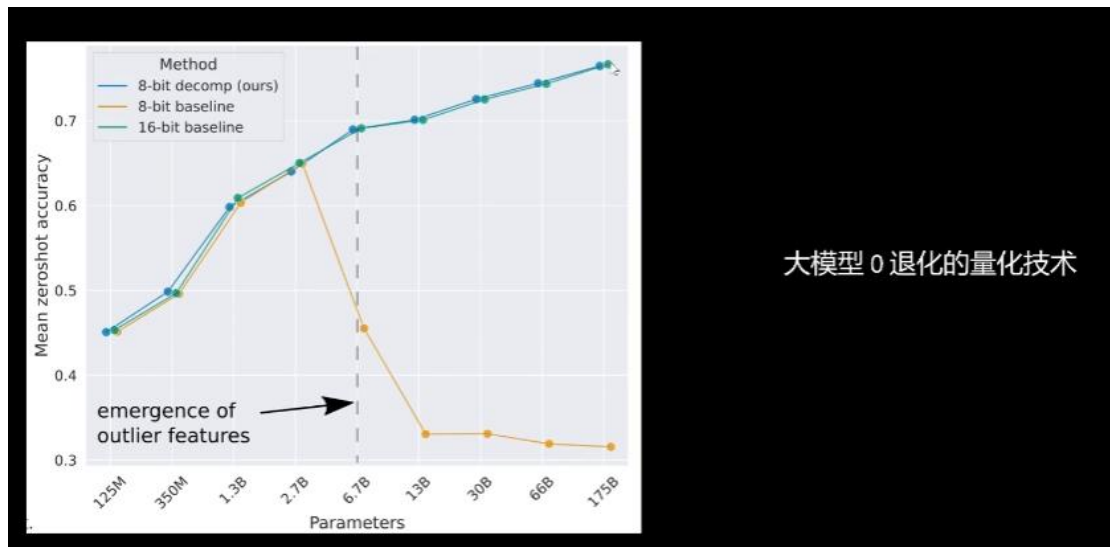
训练时：

- 在前向传播中模拟量化
- 但反向传播还用浮点数（靠 STE）

推理时：

- 真正用低比特整数算

LLM.int8() Bitsandbytes HuggingFace 默认大模型量化方法



传统量化方法在模型参数达到一定量级后基本失效，导致这一现象的主要原因是 **Emergent Features**，即在原始特征中某些特征突然变大，一般是其他特征的几十倍，而对于这种包含异常值的特征经过传统方法的量化与反量化后，许多其他不同的特征就变成同一个值了，即出现了信息丢失，模型表现下降，而如果去除异常值，也就忽略了模型中原始的重要特征，模型的性能也会下降

原始特征: [-0.10, -0.23, **57.0**, 0.08, -0.38, -0.28, **-45**, -0.29, -2.11, 0.34, -0.53, **-67.0**]

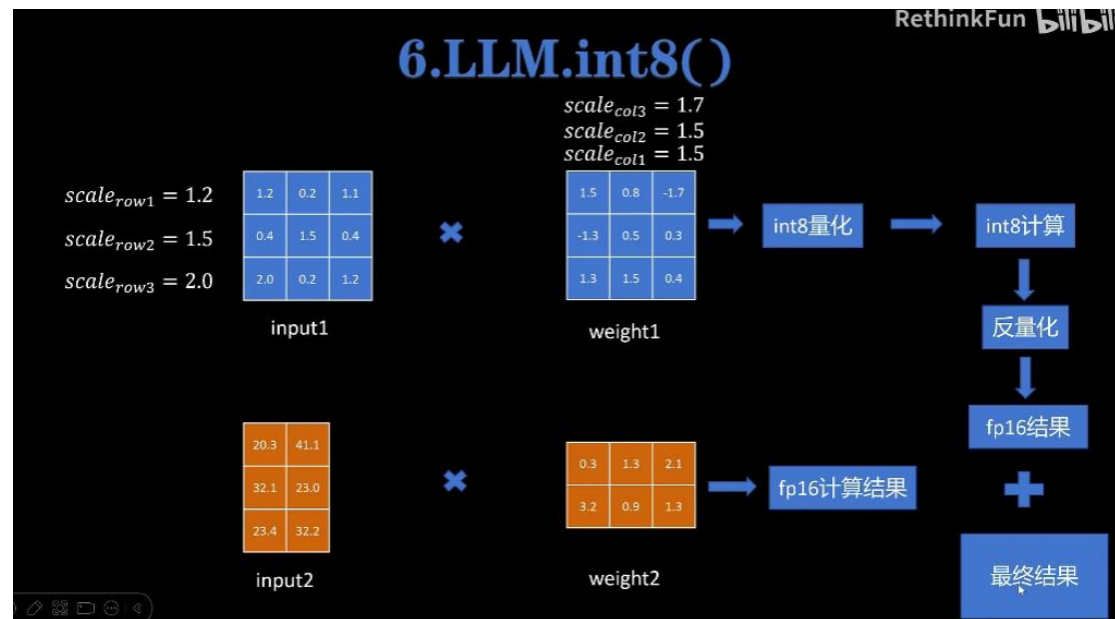
量化+反量化: [-0.00, -0.00, **57.0**, 0.00, -0.53, -0.53, **-45**, -0.53, -2.11, 0.53, -0.53, **-67.0**]

去除异常值后
量化+反量化: [-0.10, -0.23, **0.33**, 0.08, -0.38, -0.28, **-2.11**, -0.28, -2.11, 0.33, -0.53, **-2.11**]

Emergent Features

Hugging Face LLM.int8()则是将异常特征与其他普通特征分别进行处理，然后再汇总结果
具体而言，将输入矩阵中的异常列提取出矩阵，对应的权重矩阵提取出要相乘的行矩阵，原始的正常输入特征可以进行传统的量化与反量化计算得到 FP16 的结果，而异常特征则在

FP16 下进行计算（不会丢失精度），最后将两者的结果相加即可



- Emergent Feature 仅占有所有特征的 0.1%
- Weight 在加载模型时量化，显存占用比 float16 减半
- 使用 LLM.int8() 对精度几乎没有影响
- 模型推理速度会变慢 20%左右
- Emergent Features 的分布是有规律的，对于一个参数量为 67 亿的 transformer 模型来说，每个句子的表示中会有 150000 个 Emergent Features，但这些 Emergent Features 只分布在 6 个维度中。

代码实现：

```
bnb_config = BitsAndBytesConfig(load_in_8bit=True)
model = AutoModelForCausalLM.from_pretrained(model_id, device_map="auto", quantization_config=bnb_config)
```

加载时指定量化配置即可

QLoRA 4bit 量化 NormalFloat4 量化

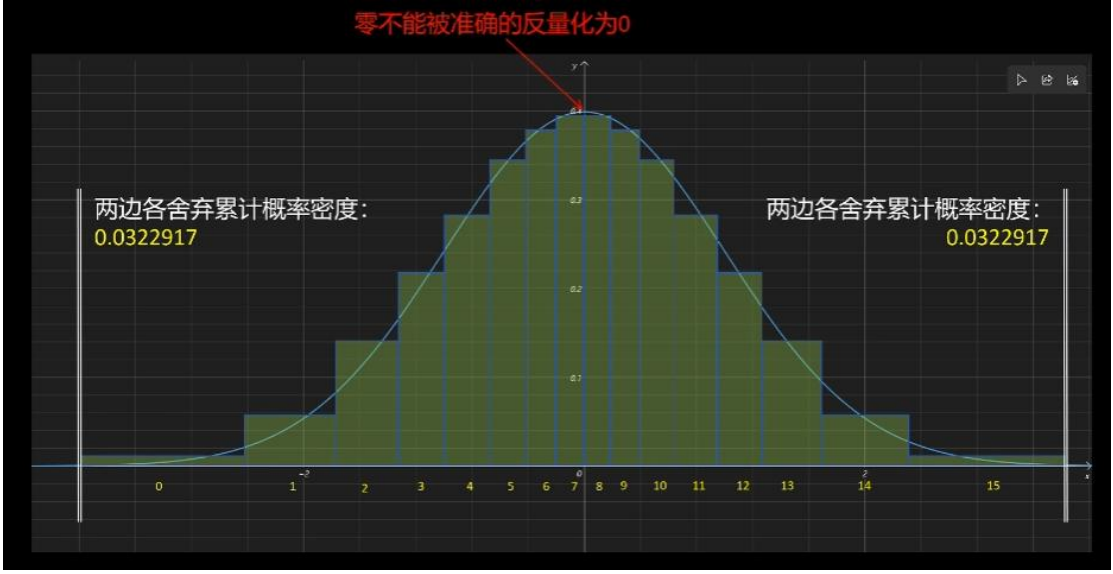
这是一种非线性的量化方法

思想：神经网络一般参数值都是符合均值为 0 的正态分布的，因此均匀的分配量化后的整数值就存在浪费，因此就引出了在原始浮点数密集的范围多分配一些量化后的整数值，密度小的地方少分配一些，从而减少量化后的误差

计算方式：

由于 4bit 量化后的整数有 16 个，将正态分布累计概率密度分为 16 个区域，越接近 0 值，分配的整数 1 越密集，越远离 0 值，分配的整数越稀疏

注意点:



将所有x除以最大绝对值，归一化到[-1,1]之间。

-1.0	0
-0.6961928009986877	1
-0.5250730514526367	2
-0.39491748809814453	3
-0.28444138169288635	4
-0.18477343022823334	5
-0.09105003625154495	6
0.0	7
0.07958029955625534	8
0.16093020141124725	9
0.24611230194568634	10
0.33791524171829224	11
0.44070982933044434	12
0.5626170039176941	13
0.7229568362236023	14
1.0	15

Normal Float 4-bit, NF4

NF4 16 个值的计算

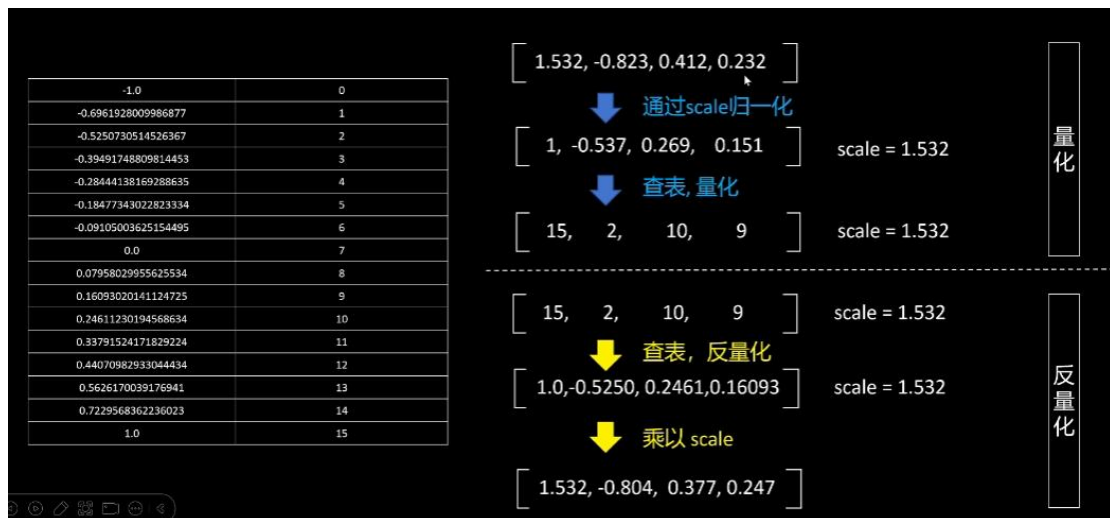
```

1 import torch
2 from scipy.stats import norm
3
4
5 def create_normal_map(offset=0.9677083):
6     # one more positive value, this is an asymmetric type
7     v1 = norm.ppf(torch.linspace(offset, 0.5, 9)[:1]).tolist() # 正数部分
8     v2 = [0]
9     v3 = (-norm.ppf(torch.linspace(offset, 0.5, 8)[:1])).tolist() # 负数部分
10    v = v1 + v2 + v3
11
12    values = torch.Tensor(v)
13    values = values.sort().values
14    values /= values.max()
15    return values

```

量化:

先除以最大值 scale 归一化后查表, 选择最贴近的量化整数



反量化:

查表找出整数对应的浮点数, 再乘以最大值 scale

分块量化

Qlora 里每 64 个值作为一个块进行 NF4 4bit 量化

分块量化带来的问题

64 个 4bit = 256 bit

1 个 32 位的 scale

额外占用: $32/256 = 12.5\%$

解决办法: 双重量化

每 256 个分块的 scale 值进行一次 8bit 量化。

额外占用从 12.5% 降低到 3.174%

双重量化减少了显存占用, 但是反量化时要进行两次反量化, 先对 scale 值进行反量化, 然后对 tensor 值进行反量化。

```
[ ] import torch
    from transformers import BitsAndBytesConfig

    bnb_config = BitsAndBytesConfig(
        load_in_4bit=True,
        bnb_4bit_use_double_quant=True,
        bnb_4bit_quant_type="nf4",
        bnb_4bit_compute_dtype=torch.bfloat16
    )

    model_4bit = AutoModelForCausalLM.from_pretrained(model_id, quantization_config=bnb_config)
```

因此在实际使用时，BitsAndBytesConfig 中的配置从上到下依次含义为：
4bit 量化开启，是否启用双重量化，量化类型，计算类型（NF4 量化后的整数不能直接参与计算，还是需要反量化才能计算）

DeepSeek V3 和 R1 有什么区别？

V3 对标的是 GPT-4o，属于通用的自然语言处理模型，倾向处理泛化的一些任务，而在一些逻辑性比较强的任务中，比如数学推导，长文本推断下的表现性能有限

R1 对标的是 GPT-o1，属于侧重于推理任务的模型

表现上而言，针对一个用户的 query 输入，推理模型会有一个思考的 think 过程，而通用模型直接会输出答案，而因为推理模型存在思维链，所以同等参数规模下回复更慢，但准确率更高

为什么 R1 的推理能力强于 V3？

R1 是在 V3 模型的基础上，经过下面的步骤训练得到的：

- **冷启动数据微调：**利用精心设计的冷启动数据对模型进行微调，增强其初始能力，也让思考过程的可读性更好
- **大规模强化学习训练：**在微调后，模型通过与环境的交互，采用强化学习方法，GRPO，去掉 PPO 中的价值模型评价阶段，改为对高价值策略进行平均的方式，特别是在编码、数学、科学和逻辑推理等推理密集型任务上，进一步提升其能力
- **语言一致性奖励：**在强化学习训练中，引入语言一致性奖励，确保模型生成的内容语言一致，解决语言混杂问题，提升可读性

CPU 和 GPU 的区别？

CPU 和 GPU 的核心区别在于其设计和架构。

CPU 采用少量功能强大、通用性高的核心，配备大容量缓存和复杂控制单元，擅长快速处理顺序性、逻辑复杂的通用任务。

GPU 则集成数千个简化、专用的计算核心，通过极高的内存带宽和并行架构，专为处理大规模数据并行计算（如图形渲染、矩阵运算）而优化，在特定计算密集型任务上可实现数十至数百倍的吞吐量提升

为什么用英伟达 GPU 做深度学习？

英伟达建立了 CUDA 平台，使得个人用户可以控制 GPU 硬件，比如 `pytorch`，提供了很多可以调用 CUDA 平台的函数

讲一下 BM25 算法

BM25 是一种概率检索模型，旨在根据查询词（Query Terms）与文档（Document）的相关性对文档进行排名。BM25 的核心思想是：

- **词频（Term Frequency, TF）**：查询词在文档中出现的频率越高，相关性越高，但高频词的贡献会逐渐饱和。
 - **逆文档频率（Inverse Document Frequency, IDF）**：查询词在整个文档集中的稀有程度越高，权重越大。
 - **文档长度归一化**：较短的文档通常比长文档更相关，因为词频在短文档中更集中。
- 简单来说，一个词在一个文档中出现次数越多，但在所有文档中越稀有，它的权重就越高。

什么是融合检索？

为什么要融合检索？

在构建检索增强生成（RAG）系统时，我们常常陷入一个困境：**如何确保检索到的上下文既“语义相关”又“关键词精确”？**

想象一下这个场景：

当用户搜索“苹果公司发布的 M3 芯片评测”时，一个纯粹依赖向量搜索的 RAG 系统可能会返回一篇关于“苹果公司最新财报”的文章。从语义上看，这没错，两者都与“苹果公司”高度相关。但用户最关心的核心关键词——“M3 芯片”——却被忽略了。

反过来，当用户搜索“好用的笔记本电脑”时，一个传统的关键词搜索引擎（如 BM25）可能会因为无法理解“好用”这个主观词汇，或者因为“笔记本电脑”这个词在太多文档中出现，而返回一大堆不相关的结果。它无法领会用户寻找“高性能”、“轻薄”或“长续航”的真实意图。

这两种情况都指向了一个核心问题：**单纯依赖一种搜索范式，无论是基于稠密向量的语义搜索，还是基于稀疏向量的关键词搜索，都有其局限性。**

为提升整体检索效果，研究人员提出了一种检索后增强技术——融合检索（Hybrid Retrieval），其核心思想是：

将不同检索方式的结果进行融合排序，以获得更准确、鲁棒的检索结果。

难点在于对于两种搜索返回的、不可直接比较的分数，该如何融合？

怎么融合？

目前，业界最推崇的方法之一是**倒数排名融合 (Reciprocal Rank Fusion, RRF)**。

- **核心思想:** RRF 不关心原始的相关性分数，只关心文档在每个列表中的排名。一个文档在任何一个列表里排名越高，它的最终得分就越高。

- **计算公式:** $\text{Score}(\text{doc}) = \sum (1 / (k + \text{rank}_i))$ 其中， rank_i 是文档在第 i 个搜索结果列表中的排名， k 是一个小的平滑常数（通常设为 60），用于降低排名靠后结果的权重。

- **为什么有效:** RRF 非常鲁棒且简单。它优雅地绕开了归一化不同搜索引擎得分的难题，让排名决定一切，使得两种完全不同的搜索范式可以公平地“投票”。

在 RRF 的基础上，业界又推出了加权倒数排名算法 WRRF，引入了不同检索系统的权重方便调整不同检索系统在检索中的占比，增强鲁棒性和检索的灵活性

大模型训练

模型训练的显存占用如何计算？

1.输入输出

Llama13B

Embedding后输入：

b Batch Size : 1

$$b * s * h * 2 / 1024 / 1024$$

s Sequence Len : 1024

FP16: 10MB

h Hidden Size: 5120

输入输出： 20MB

2 是指的 fp16 要占 2 个 Byte

2.模型参数

Llama13B

$$1B = 1000^3$$

$$1GB = 1024^3 \text{ byte}$$

$$FP16: 13 * 2 = 26GB$$

3.优化器

Llama13B	梯度指数平滑值: $13 * 4 = 52\text{GB}$
Adam	梯度平方指数平滑值: $13 * 4 = 52\text{GB}$
	模型参数: $13 * 4 = 52\text{GB}$
	一共156GB

为什么优化器这里都是FP32?

因为用 FP16 来计算大量小值的累加操作会丢失精度。Pytorch 的自动精度混合训练时中对于 sum, min 这样的操作也是自动转化为 FP32 进行的

4.激活值

Llama13B	FP16:
s 序列长度: 1024	$s * b * h * (34 + 5 * a * s/h) * L / 1024 / 1024 / 1024 \text{ GB}$
b batch size: 1	=14.5GB
h 隐藏层大小: 5120	
a Attention头: 40	bach size=4000000
L 层数: 40	$14.5\text{GB} * 4000000 = 58000000\text{GB}$

5.梯度值

Llama13B	FP16:
	$13\text{B} * 2 = 26\text{G}$

Llama13B

BatchSize=1, Seq_Len=1024, FP16

模型参数: 26GB

优化器: 156GB

激活值: 14.5G (和BatchSize, Seq_len相关)

梯度值: 26GB

合计: 222.5GB

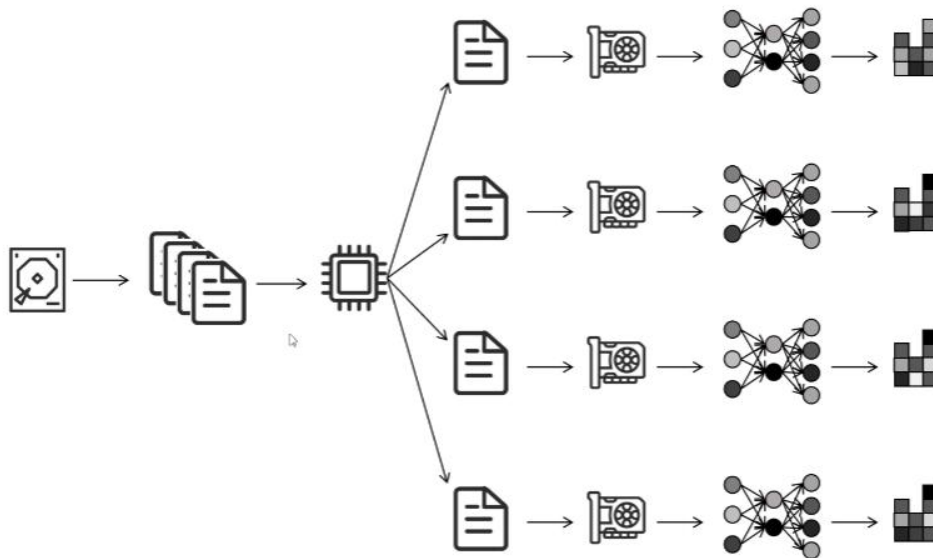
单机单卡情况下的训练流程



从硬盘读取数据->CPU 处理数据组成 batch->传入 GPU->进行前向网络传播->算出 loss->进行后向传播->计算梯度->梯度更新网络参数->完成一次训练

数据并行多卡训练框架

DP(Data Parallel)

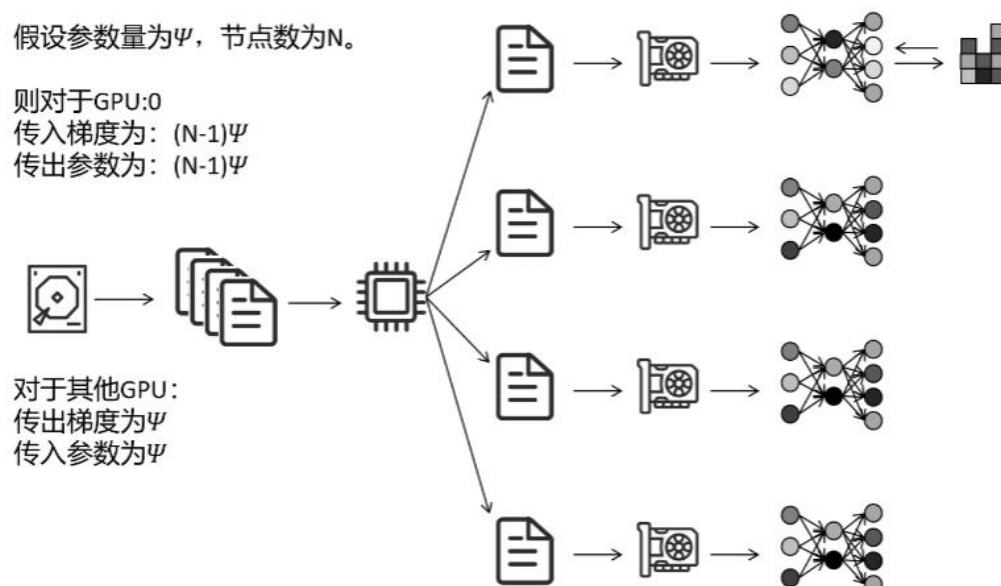


训练流程:

从硬盘读取数据->CPU 处理数据分成多份给每个 GPU->每个 GPU 独立进行网络的前向传播后向传播训练->计算出各自的梯度->所有其他的 GPU 都将计算出的梯度传递到 GPU0 上进行平均->GPU0 用全局平均的梯度更新自己的网络参数->将更新后的参数广播到其他 GPU 上

关键点:

如何减少多卡之间的通信量以提高训练效率



通信的参数量取决于模型本身参数和使用的 GPU 数量（线性相关）

问题:

- 单进程-多线程模式，python GIL 只能利用一个 cpu 核
- GPU0 负责收集梯度，更新参数，同步参数。通信，计算压力大

DDP(Distributed Data Parallel)

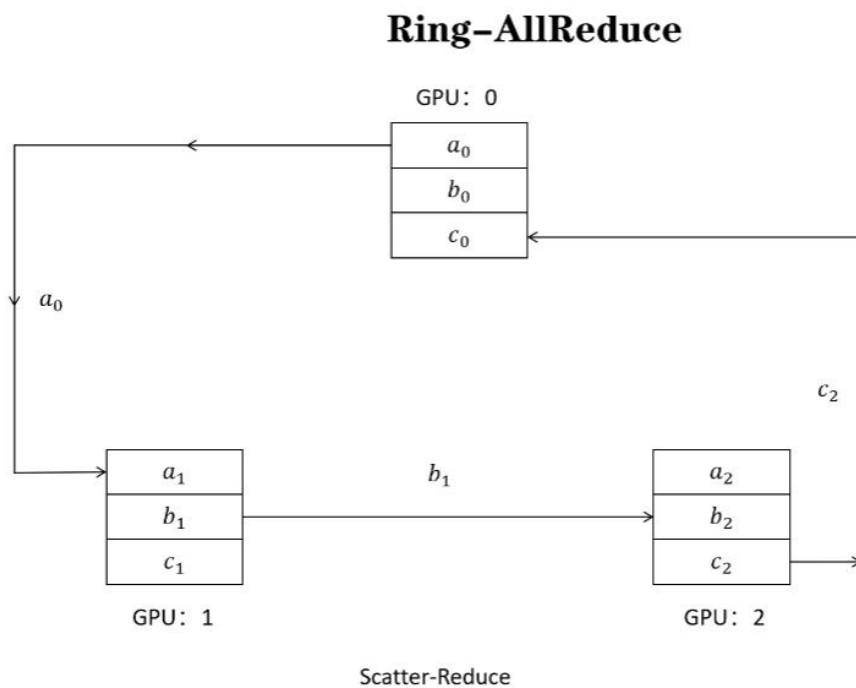
集群通讯方式 Ring-AllReduce

将多个节点连成一个环进行通讯，每个 GPU 通讯时同时收发数据，下面图中 abc 均表示参数，目标是每个 gpu 都能有各自的同步和

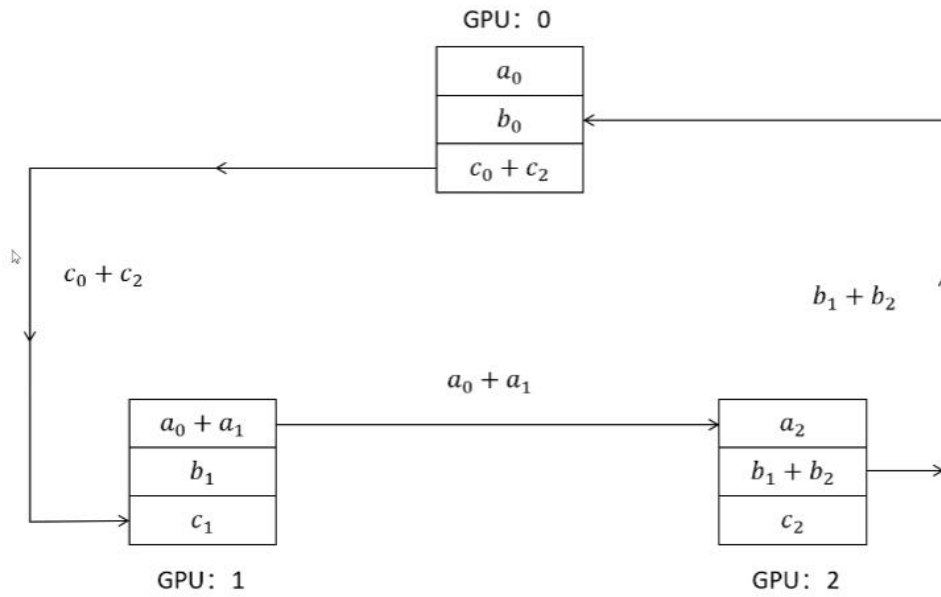
第一阶段：scatter-reduce

scatter 通过分发，让每个节点有不同的值

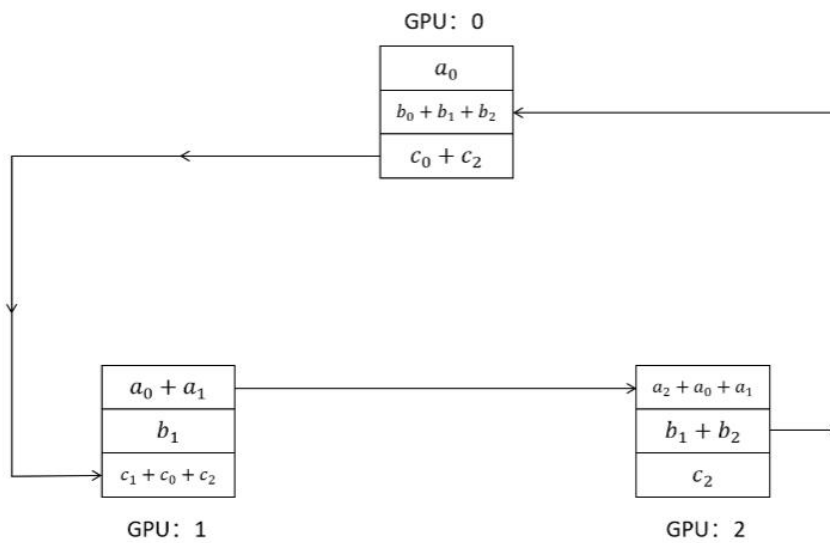
Reduce 收集所有节点的值并进行计算



Ring-AllReduce



Ring-AllReduce



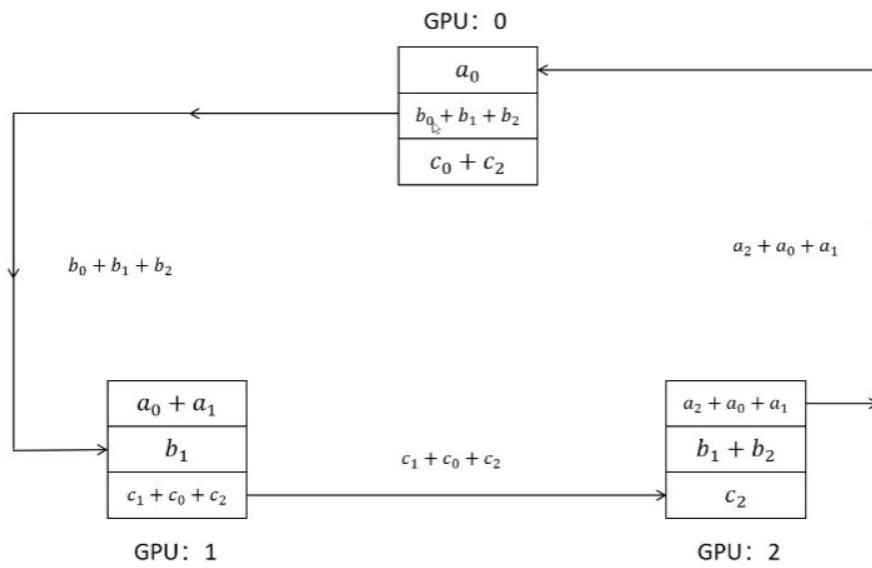
Scatter-Reduce

第二阶段: AllGather

将各参数梯度求和值同步到其他 gpu

Ring-AllReduce

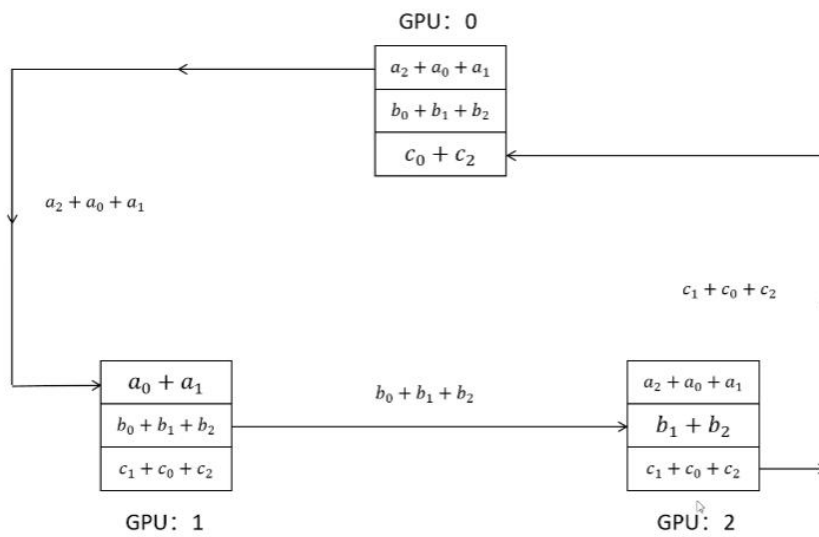
2.5x faster



AllGather

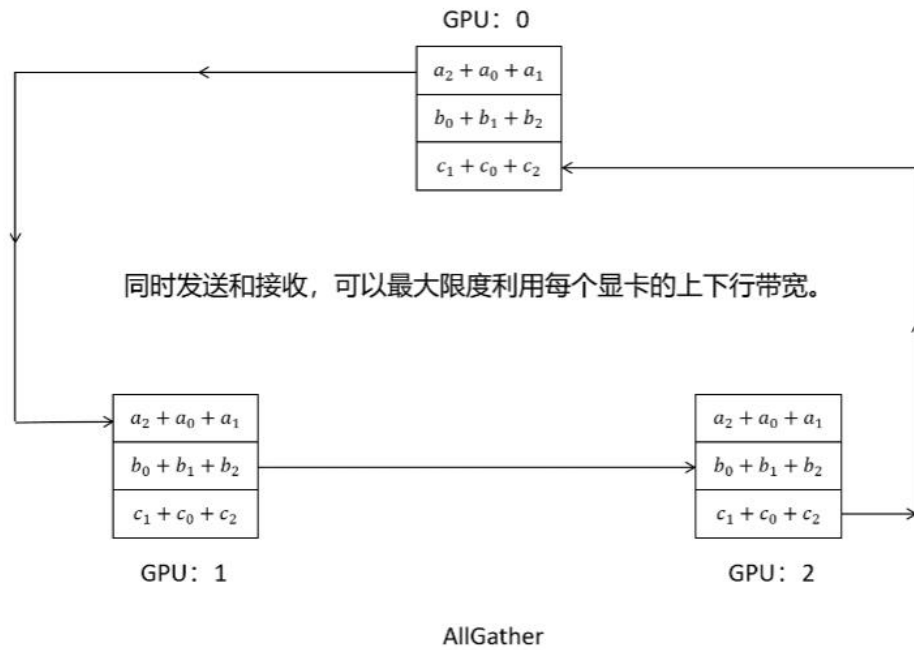
Ring-AllReduce

2.5x faster



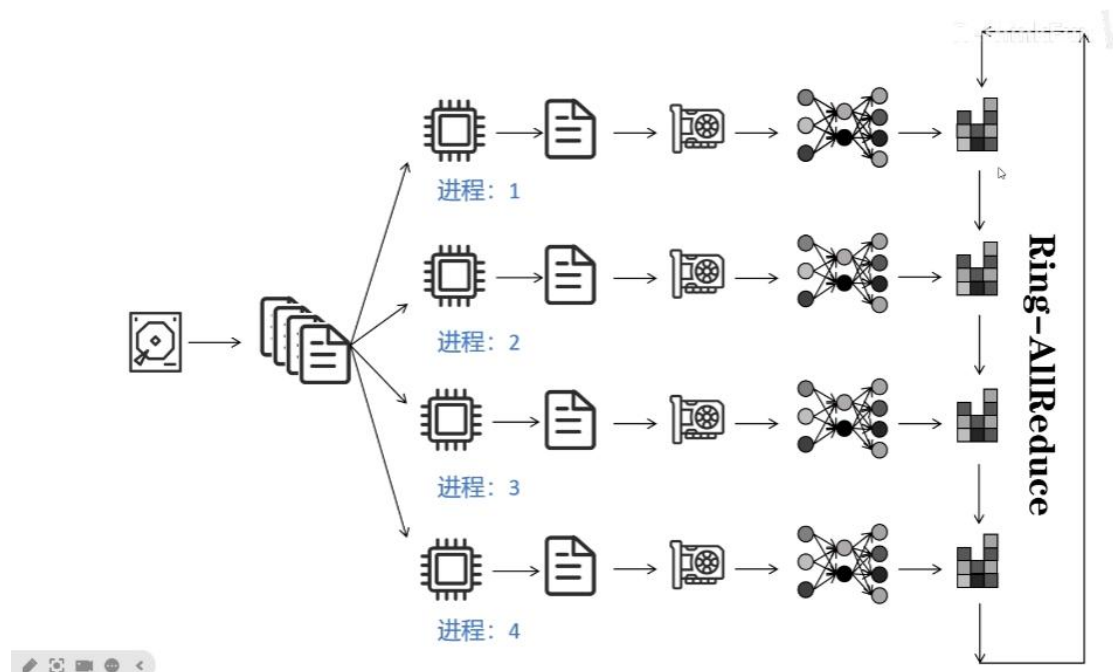
AllGather

Ring-AllReduce



DDP 训练流程:

多进程，每个进程为自己的 GPU 准备数据并和其他 GPU 通信，每个进程计算出自己的梯度后，通过 Ring-AllReduce 来同步多个 GPU 计算出的梯度



DDP 通讯量分析

参数量为 Ψ ，进程数为 N

对于每一个GPU进程：

Scatter-Reduce 阶段传入/传出： $(N - 1) \frac{\Psi}{N} \approx \Psi$

AllGather 阶段传入/传出： $(N - 1) \frac{\Psi}{N} \approx \Psi$

总传入/传出： 2Ψ

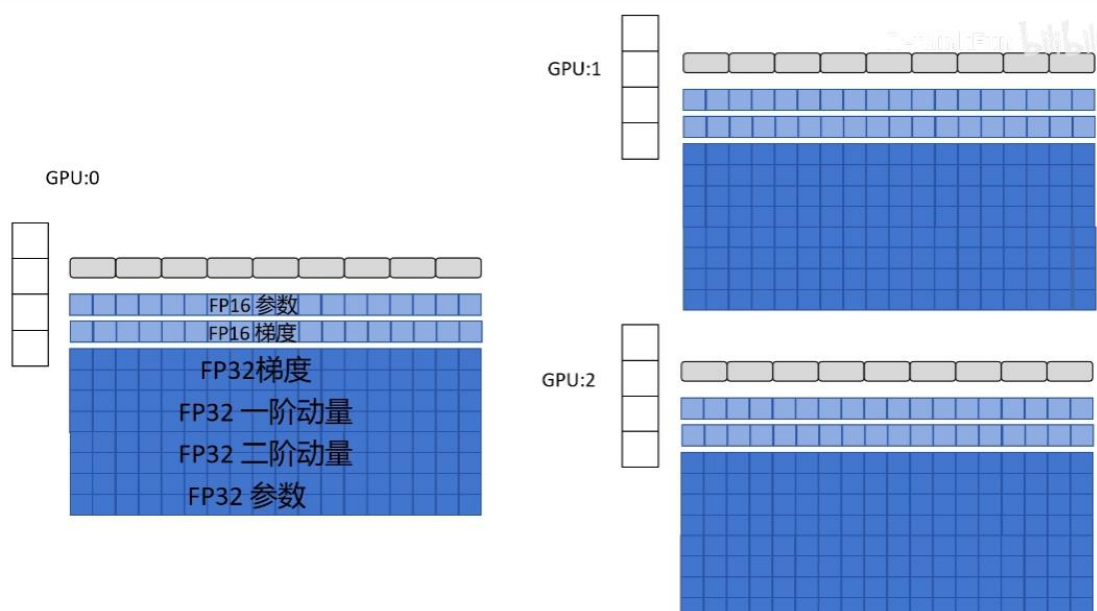
与集群大小无关

缺点：

- 每个 GPU 都要存完整的神经网络，优化器，可能会导致 GPU 显存不够

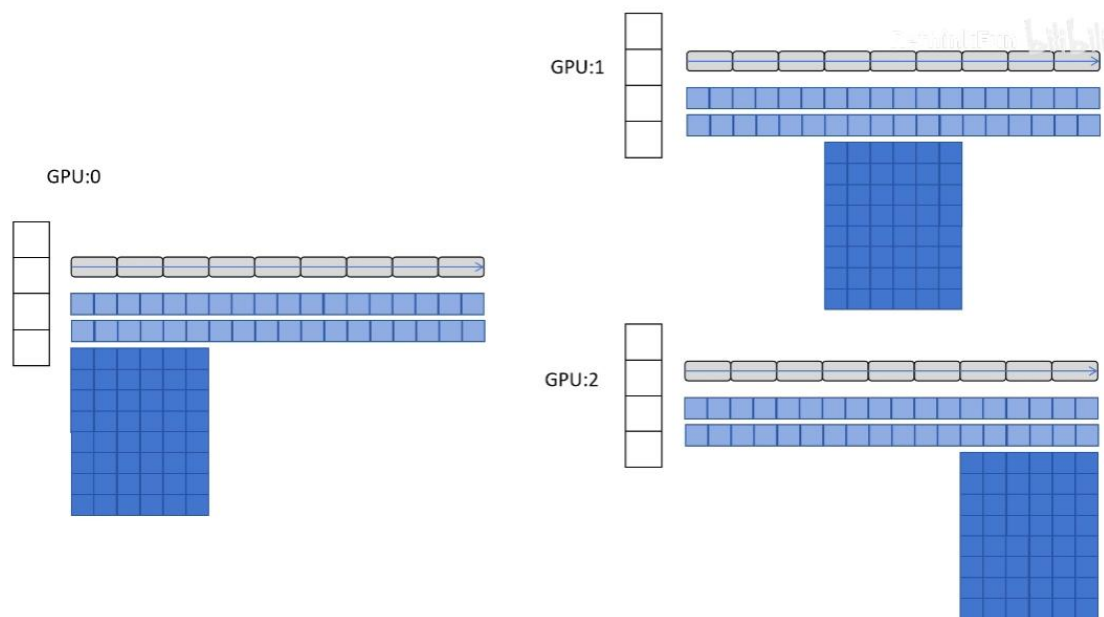
DeepSpeed ZeRo-1

模型训练时每个 GPU 都要存完整的优化器状态，造成大量的空间占用



以 9 层神经网络为例，GPU0 存储前三层神经网络对应的优化器状态，负责前三层网络参数的更新，GPU1 存储中三层，GPU2 存后三层，因为每个 GPU 都存储了 FP16 的网络参数，因此前向传播照常进行，后向传播时，每个 GPU 都从后向前计算出每一层的参数的梯度，由于 GPU0 和 1 不负责后三层梯度的计算，因此他们将自己计算出的后三层梯度发送给 GPU2（发送时仍然会继续计算其它层的梯度），GPU2 负责聚合三个 GPU 计算出的最后三层的梯度，并算出梯度平均值，反向传播结束后，每个 GPU 都拿到自己优化器对应部分参数的梯度均值，然后将梯度转化为 FP32，进行缩放，然后更新优化器的一阶和二阶动量，最后优

化器更新 FP32 参数，接下来，更新各自优化器对应部分的 FP16 的网络参数，然后将各自更新后的 FP16 的网络参数广播给其他 GPU，完成一次训练



DeepSpeed ZeRO-1 通讯量分析

参数量为 Ψ ，进程数为 N

对于每一个GPU进程：

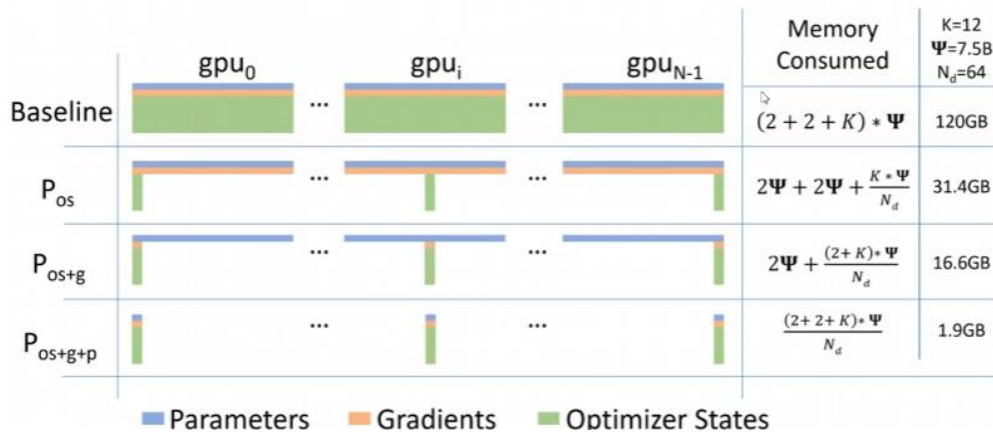
梯度收集阶段传入/传出： $(N - 1) \frac{\Psi}{N} \approx \Psi$

参数广播阶段传入/传出： $(N - 1) \frac{\Psi}{N} \approx \Psi$

总传入/传出： 2Ψ

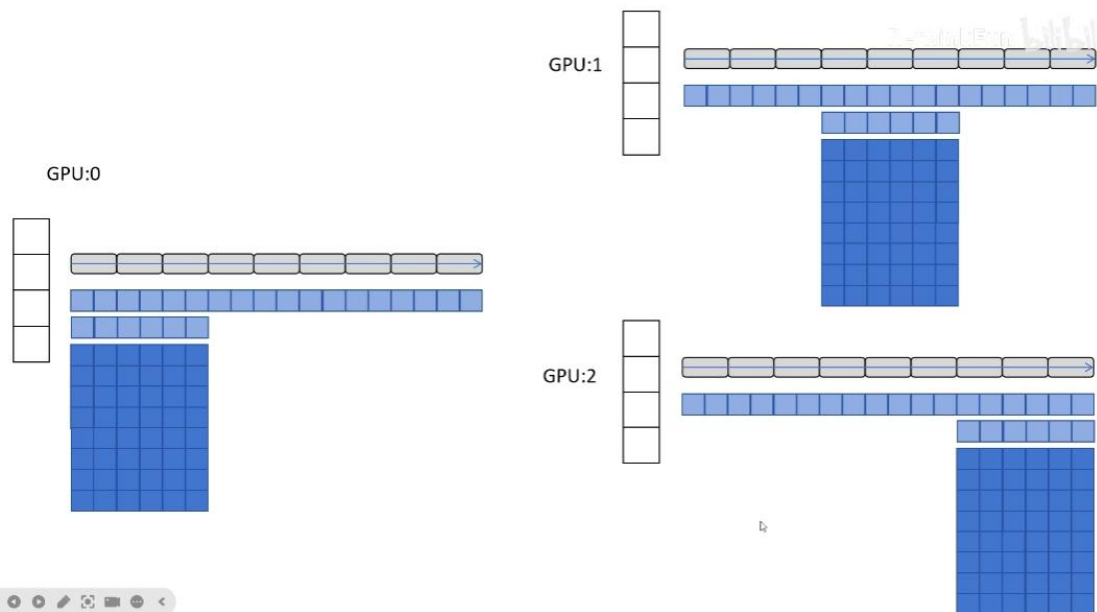
与DDP通讯量相同

DeepSpeed ZeRO-1 显存节省分析



DeepSpeed ZeRo-2

动机来源于既然每个 GPU 只负责更新一部分参数，则也只需要保存这一部分参数的梯度即可，将 FP16 的梯度也按 GPU 进行划分



具体而言，前向传播时，计算出每一层的梯度并将梯度发送给负责更新这一部分的 GPU，自己直接释放掉这部分参数占用的显存

DeepSpeed ZeRO-2 通讯量分析

参数量为 Ψ ，进程数为 N

对于每一个GPU进程：

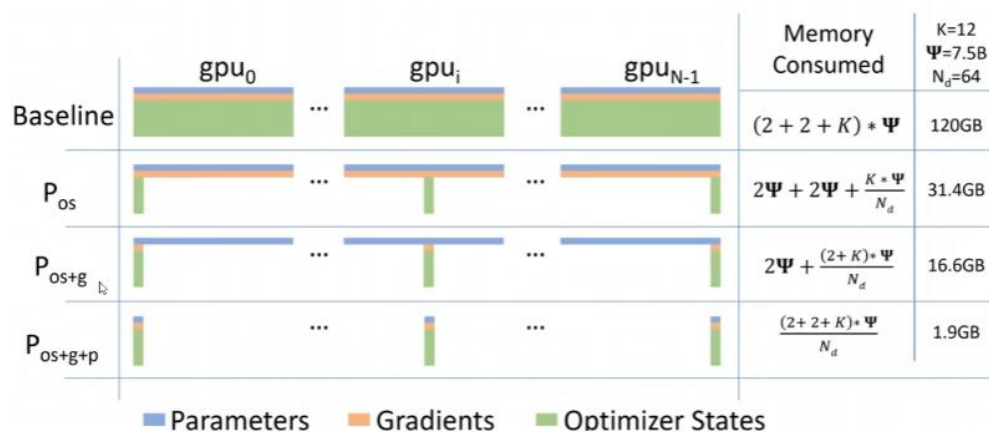
梯度收集阶段传入/传出： $(N - 1) \frac{\Psi}{N} \approx \Psi$

参数广播阶段传入/传出： $(N - 1) \frac{\Psi}{N} \approx \Psi$

总传入/传出： 2Ψ

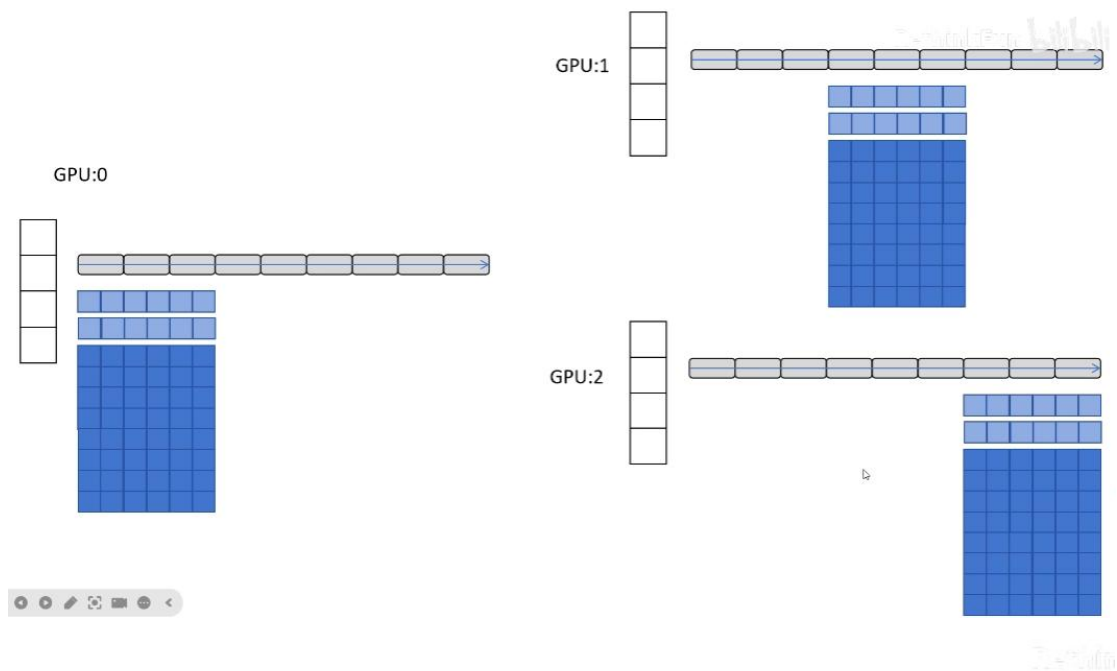
与DDP通讯量相同

DeepSpeed ZeRO-2 显存节省分析



DeepSpeed ZeRo-3

在 ZeRo-2 的基础上继续对参数进行划分，让参数零冗余，前向/后向传播时，遇到当前 GPU 没有的参数时依靠对应的 GPU 进行广播，计算完梯度后立即释放掉对应的显存占用



DeepSpeed Zero-3 通讯量分析

参数量为 Ψ ，进程数为 N

对于每一个GPU进程：

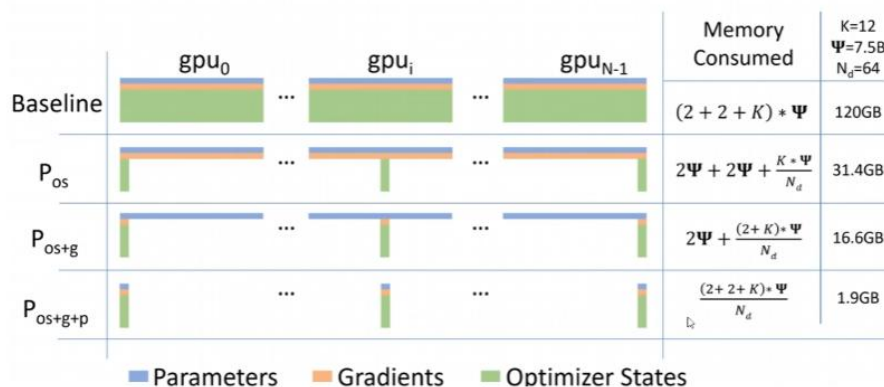
梯度收集阶段传入/传出： $(N - 1) \frac{\Psi}{N} \approx \Psi$

参数广播阶段传入/传出： $2 * (N - 1) \frac{\Psi}{N} \approx 2\Psi$

总传入/传出： 3Ψ

是DDP传输量的1.5倍

DeepSpeed Zero-3 显存节省分析



一般实际建议使用 zero-2 没有增加通信量，但大大减少了显存占用，仅为原来 DDP 的 10%

为什么多卡训练不一定线性加速？

多卡训练不一定线性加速，主要原因是通信和同步开销。

在 Data Parallel 中，每一步反向传播都需要做梯度 all-reduce，当卡数增加时，通信成本会上升。

如果 batch size 较小，单卡计算时间很短，通信时间反而成为主要瓶颈，导致 GPU 等待同步。

如果数据加载、预处理或 CPU IO 跟不上，也会限制整体吞吐，导致算力无法被充分利用。

可以通过增大 batch size、使用梯度累积、优化通信策略，或者采用 ZeRO、FSDP 等减少通信和显存开销。

训练时 GPU 利用率很低，你会怎么排查

第一，检查数据加载是否成为瓶颈，比如 DataLoader 的 num_workers 是否过小，是否存在 IO 阻塞。

第二，检查 batch size 是否过小，导致 GPU 计算时间不足，算力喂不饱。

第三，在多卡训练中，会关注是否存在频繁的同步操作，比如 all-reduce 阻塞。

最后，我会通过 profiling 工具或日志观察 CPU、GPU 和 IO 的时间占比，定位具体瓶颈。

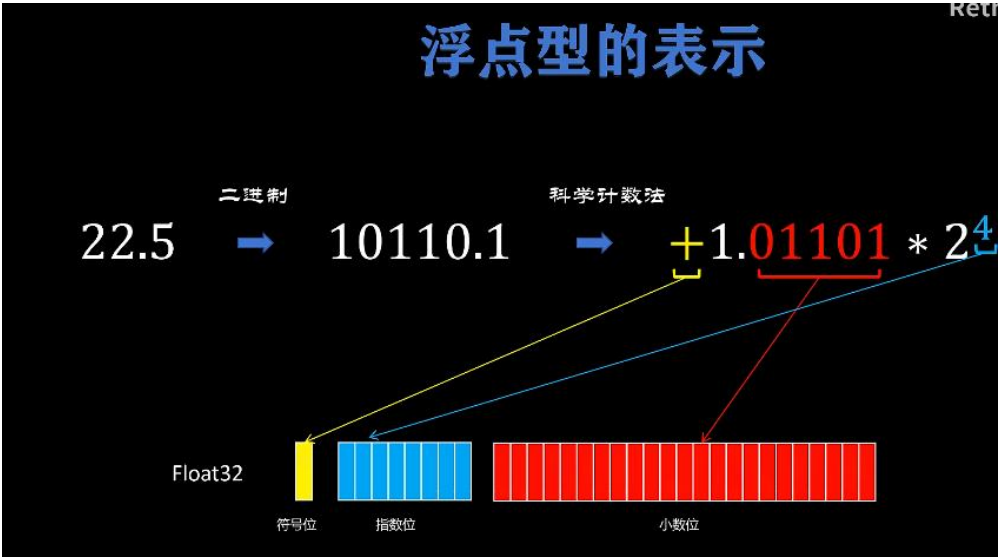
你怎么看是数据加载慢？

如果 GPU utilization 周期性掉到很低，同时 CPU 利用率偏高，通常是数据加载或预处理成为瓶颈。

混合精度训练

AMP(Automatic Mixed Precision)

浮点型的表示



低精度的优势

低精度带来的好处

A100 算力

	A100 40GB PCIe	A100 80GB PCIe	A100 40GB SXM	A100 80GB SXM
FP64	9.7 TFLOPS			
FP64 Tensor Core	19.5 TFLOPS			
FP32	19.5 TFLOPS			
Tensor Float 32 [TF32]	156 TFLOPS 312 TFLOPS*			
BFLOAT16 Tensor Core	312 TFLOPS 624 TFLOPS*			
FP16 Tensor Core	312 TFLOPS 624 TFLOPS*			
INT8 Tensor Core	624 TOPS 1248 TOPS*			

- 一. 16倍的计算速度提升。
- 二. 显存占用变小。
- 三. 数据传输带宽占用小。

以 A100 为例，从 FP32->FP16，算力能提升 16 倍，其次 FP16 的空间占用小，不仅能够降低显存占用，还能降低数据传输的带宽占用，因此能够提升模型的训练速度

低精度的缺点

1. 表示范围问题，低精度类型不能表示太大或太小的高精度类型，会导致数据类型溢出
2. 大数吃小数的问题：例如 pytorch 中用 float16 的 2048+float16 的 0.5 依然是 2048，出现这个现象的原因是小数和大数相加时，小数要转为和大数一样的指数表示， $2048=1.0 \times 2^{11}$ ，而 $0.5 = (1.0 \times 2^{-12}) \times 2^{11}$ ，而因为 float16 的小数位只有 10 位，无法表示，因此溢出为 0

二. 大数吃小数问题

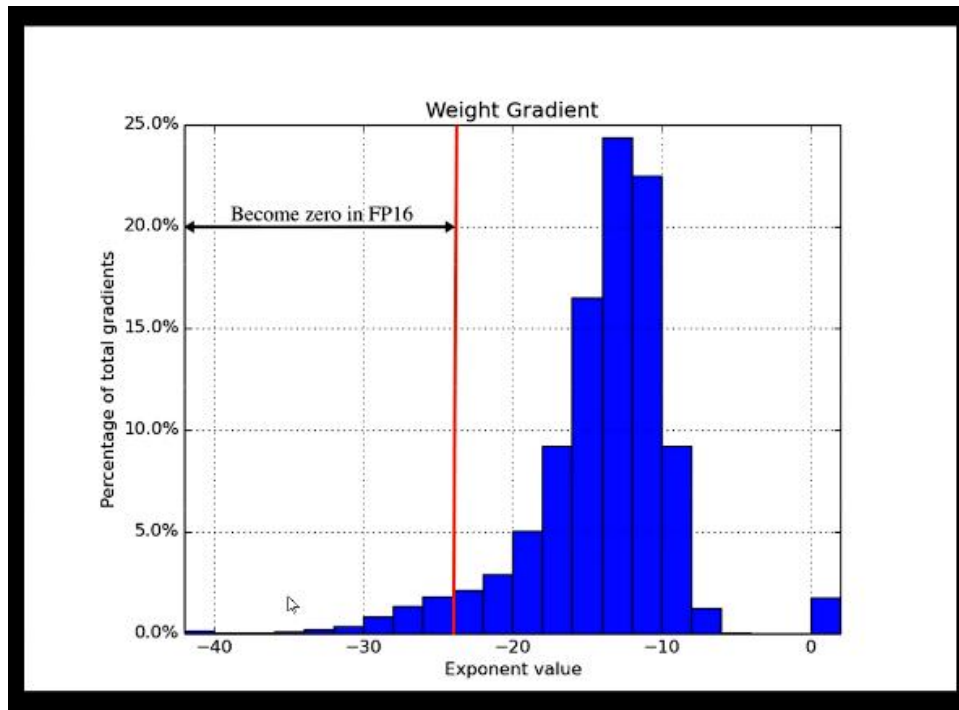
$$2048 + 0.5 = 2048$$

$$2048 \quad 1.0 * 2^{11}$$

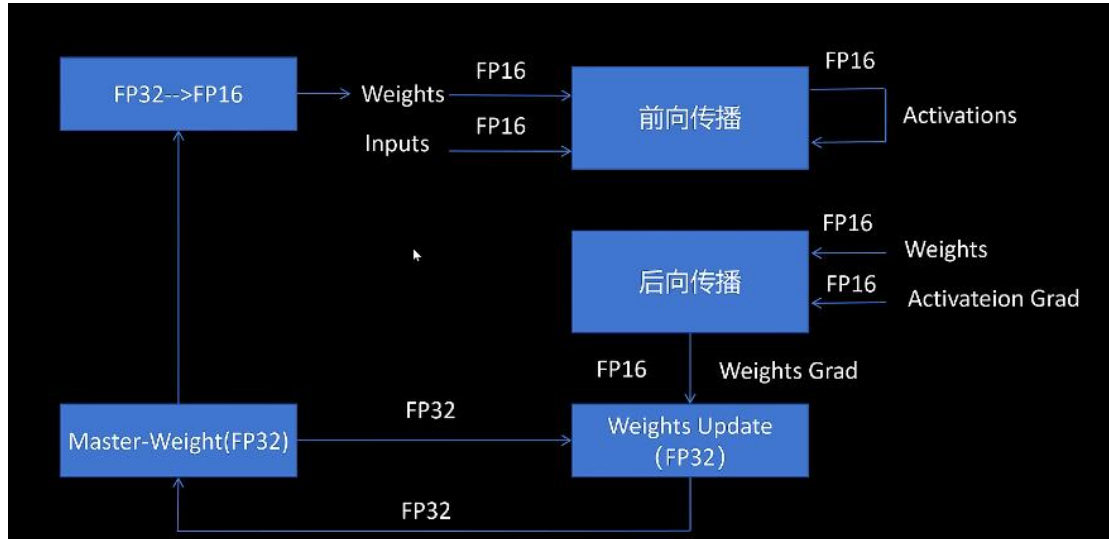
$$0.5 \quad 1.0 * 2^{-1} \quad 0.00000000000001 * 2^{11}$$

神经网络大部分的计算都是在 float16 下进行，但优化器会同时保存一份 float32 的参数值，即 Matser-Weight，原因如下：

在计算梯度时，由于梯度可能很小，导致溢出 FP16 的表示范围，梯度更新时，如果在 FP16 精度下计算，就可能会有部分梯度下溢为 0，起不到参数更新的作用，此外，还存在大数吃小数的问题，因为参数值相对梯度来说都很大，如果用 FP16 精度，则会存在参数值吃掉梯度值的问题，因此需要使用 FP32 来保存一份模型的参数值

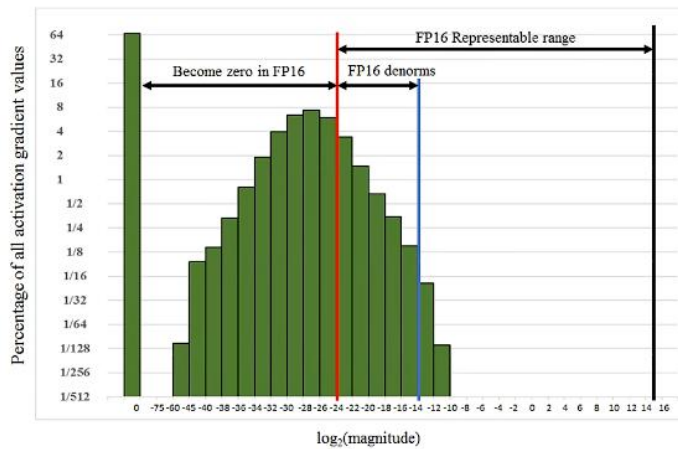


下图讲述了网络的训练过程中参数的精度变化，一开始从优化器中传出 FP32 的参数，转换为 FP16 进入前向传播计算，反向传播也是 FP16，反向传播计算出的 FP16 梯度转换为 FP32 传入优化器进行权重更新

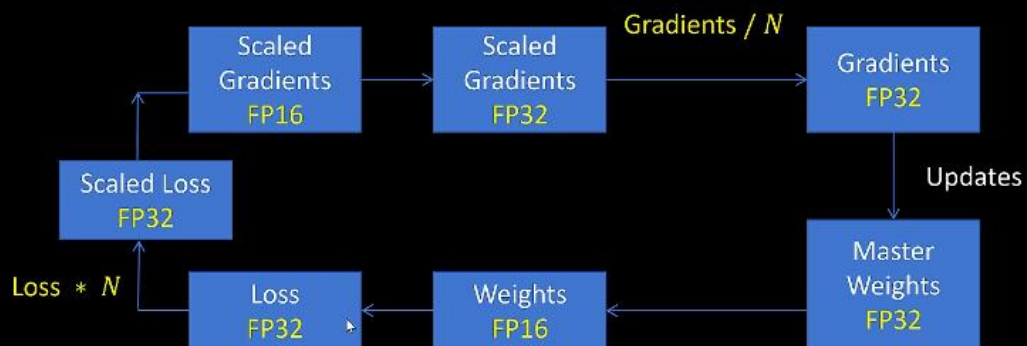


我们刚才讲了计算出的梯度都很小，因此实际情况中 FP16 中数据较大的范围就没有被使用，而小的梯度又会溢出 FP16 的表示范围，因此可以对梯度进行缩放，将其移动至 FP16 可以表示的范围内

Loss Scaling



Loss Scaling



前向传播得到 FP32 的 loss，通过乘以缩放系数 N 放大 loss（仍为 FP32），反向传播从放大后的 FP32 loss 出发，在涉及 FP16 权重和激活的算子中计算梯度；随后在优化器更新前对梯度进行反缩放并以 FP32 形式更新参数，从而完成一次稳定的混合精度训练。

代码：


```

15 net = create_model() # 创建模型
16 opt = torch.optim.SGD(net.parameters(), lr=0.001) # 创建优化器
17 scaler = torch.cuda.amp.GradScaler(enabled=use_amp) # 定义梯度缩放器
18
19 for epoch in range(epochs):
20     for input, target in zip(data, targets):
21         # 进行自动缩放计算, 缩放类型
22         with torch.autocast(device_type=device, dtype=torch.float16):
23             output = net(input)
24             loss = loss_fn(output, target)
25             scaler.scale(loss).backward() # 对loss进行缩放
26             scaler.step(opt) # 缩放梯度, 进行参数更新
27             scaler.update() # 调整缩放比例
28             opt.zero_grad()

```

为什么 BF16 不需要 loss scaling

BF16 不需要 loss scaling，因为它的指数范围和 FP32 一样，不容易发生梯度下溢（underflow）。

loss scaling 的目的只有一个：

防止梯度太小，在低精度里直接变成 0

BF16 通过牺牲尾数位的精度来换取指数位的范围，使得 BF16 和 FP32 的最大/最小值数量级一样，因此不容易出现梯度下溢或者梯度上溢

那 BF16 精度低不会有问题吗？

这是一个非常自然的疑问。

答案是：不会（对深度学习）

原因：

- 梯度本来就有噪声
- SGD 对尾数精度不敏感
- 神经网络对小误差极其鲁棒

训练过程中出现 OOM，你会怎么分析

OOM 本质上是模型参数、激活、梯度或优化器状态占用过大。

训练过程中出现 OOM，我会从几个方面分析：

1. 首先检查 `batch size` 是否过大，因为激活占用通常和 `batch size` 成正比。
2. 如果 `batch size` 已经很小，我会考虑使用 `gradient accumulation` 或 `activation checkpointing` 来降低显存峰值。
3. 在大模型场景下，还可以使用 `DeepSpeed ZeRO` 或 `FSDP`，将参数、梯度和优化器状态分片。
4. 同时我也会关注显存碎片问题，比如是否频繁创建临时张量

activation checkpoint 是干嘛的？

它通过在前向时不保存部分中间激活，在反向传播时重新计算，来换取计算时间降低显存占用。

如何做训练日志和监控？

日志和监控主要是为了**观察训练状态、排查问题和复现实验**。

在训练中我通常会记录核心指标，比如 `loss`、`learning rate`、`step`、`epoch`，以及 GPU 显存使用情况。

同时会使用 `TensorBoard` 或类似工具对不同实验进行对比，观察收敛趋势。

这样在出现训练不稳定、`loss` 异常或性能问题时，可以快速定位问题

为什么要记录 `learning rate`？

因为学习率变化会直接影响 `loss` 曲线，很多不稳定问题其实和 `scheduler` 配置有关。

训练中断了，如何恢复？

需要通过 `checkpoint` 机制恢复。

训练中断后，我会通过 `checkpoint` 机制恢复训练。

`checkpoint` 中通常需要保存模型参数、优化器状态，以及 `AMP` 场景下的 `scaler` 状态。

恢复时从上一次保存的 `step` 继续训练，保证训练过程的连续性和可复现性。

为什么要保存 `optimizer` 状态？

因为像 `Adam` 这种优化器依赖历史梯度信息，如果只恢复模型参数，训练轨迹会发生变化。

如果让你设计一个模型训练平台，你会关注哪些点？

如果设计一个模型训练平台，我会重点关注几个方面：

第一是稳定性，确保训练任务在异常情况下可恢复；
第二是可扩展性，支持多卡、多机和不同规模模型；
第三是性能，合理利用混合精度和分布式训练提升资源利用率；
第四是自动化，比如自动启动、监控和实验管理；
最后是可观测性，提供完善的日志和指标，方便排查问题。

什么是 Skills？

Skills 是 Claude 提出的功能，是将一类可复用的任务能力，封装成结构化、可调用的行为单元。skill = “模型 + 约束 + 执行逻辑”组成的一个能力模块

本质上是一种解耦上下文的方法，ai 目前的核心还是怎么解决上下文有限的问题

Skills 和 Tool 有什么区别？

Tool 是系统层面的原子执行接口，负责完成具体的一步操作；
而 Skill 是面向 Agent 的能力抽象，通常封装了一类任务的执行逻辑，内部可能组合多个 Tool 并加入状态管理和约束。LLM 在决策时更多是选择 Skill 并给出参数，而系统通过 Skill 的实现来控制执行路径，从而在保证灵活性的同时提升整体稳定性和可维护性。

Tool:

- 一次调用
- 无状态
- 无上下文目标

Skill:

可以是:

- 多步
- 有状态
- 有目标约束

以“写爆款小红书标题”为例，如果只用 Tool 模式，通常是让 LLM 自行决定是否搜索、搜索什么、如何组织上下文，再根据这些信息生成标题。这个过程中，任务拆解和执行路径几乎完全依赖模型的推理能力，因此结果波动较大，稳定性不高。

而在 Skill 模式下，“写爆款小红书标题”本身被抽象成一个独立的能力单元。LLM 只需要选择这个 Skill 并给出输入参数，比如主题或关键词，具体如何生成标题则由 Skill 内部封装好的执行流程来完成。

这个流程中，系统可以预先定义好标题生成的 **Prompt** 约束、可访问的本地参考素材、允许调用的搜索或分析函数，以及必要的反思或过滤步骤。LLM 和系统在 **Skill** 内协作执行，但对外只暴露一个稳定的能力接口。

通过这种方式，任务结构被固化在 **Skill** 层，而不是完全依赖模型的临场发挥，从而显著提升了生成结果的一致性、可控性和整体表现。

本质上，**Tool** 是把执行能力暴露给模型，而 **Skill** 是把“如何完成一类任务”的知识固化到系统中，用工程结构约束模型的不确定性。

Agent、multi-Agent、Skills 之间有什么区别？

最初的单 agent 架构，把所有的 tool（包括 mcp 接入的 tool）的信息（描述、参数、调用规则）都放在提示词/上下文中，且每轮调用都携带，当 tool 太多就会造成提示词爆炸问题（1 是费 token，2 是造成污染）。

随后又提出了多 agent 架构，不同的 subagent 配备不同的 tool，由 supervisor 决定调用哪些 subagent，这一定程度上解决了提示词爆炸的问题，但是 supervisor 调用一个 subagent 往往就是一条指令，并不包含之前的完整上下文信息，可能会造成上下文丢失问题，且开发太多 agent 有些太重了。

现在的 agent skills 的做法是仅把 SKILL.md 的元数据加载到提示词中，而后按需加载 SKILL.md 的主体部分（使用 tool，例如 Bash 工具），核心思想就是渐进式披露，解决了提示词爆炸问题，且全程只使用一个 agent、一个对话窗口，比较连贯，比多 agent 更轻量。

agent skills 并不是取代、多 agent，各有各的使用场景，例如并行任务就更适合多 agent 架构。且二者可以结合使用